

תוכן העניינים:

7.....	מבוא למערכות הפעלה.
8.....	פירוט תתי – המערכות.
9.....	מבנה מערכת המחשב.
10.....	מבנה מערכת ההפעלה.
11.....	צורות שונות למבנה מערכת ההפעלה.
12.....	מטרות מערכת ההפעלה.
13.....	שירותי מערכת ההפעלה.
14.....	תהליך ה-Boot – טעינת מעה"פ.
14.....	User mode ו- Kernel mode.
14.....	תוכניות מערכת.
15.....	System-Calls.
16.....	פסיקות – Interrupts.
19.....	סיגנלים – Signals.
22.....	תהליכים – Processes.
25.....	מצבי התהליך.
26.....	Context Switch.
27.....	תזמון תהליכים – Scheduling.
27.....	ה-Dispatcher.
28.....	מבוא לאלגוריתמים לתזמון תהליכים.
29.....	אלגוריתמים לתזמון תהליכים.
33.....	Threads.
34.....	User Thread ו- Kernel Thread.
35.....	שיקולי Performance בתהליכונים.
36.....	מודלים לריבוי תהליכונים.
36.....	טיפול בסיגנלים בתהליכונים.
37.....	יצירת תהליכונים – Thread Pool.
37.....	ביטול תהליכונים – Thread Cancellation.
37.....	fork() ו- exec() בתהליכונים.
38.....	הספרייה <pthread.c>.

39.....	שיטות חלוקת עבודה בין מעבדים.....
40.....	תבניות לעיצוב אלגוריתמים מקביליים.....
42.....	מקבול וסנכרון.....
44.....	פתרונות תוכנה ברמה נמוכה.....
44.....	פתרונות ל-2 תהליכים.....
45.....	אלגוריתם פטרסון.....
45.....	פתרונות לתהליכים מרובים.....
45.....	אלגוריתם המאפיה.....
47.....	חסימת פסיקות.....
48.....	פתרונות חומרה.....
48.....	פקודת TSL.....
48.....	פקודת Swap.....
49.....	פקודת CAS.....
50.....	פתרונות תוכנה ברמה גבוהה.....
50.....	Semaphore.....
53.....	הספרייה <semaphore.h>.....
54.....	פירוט גרסאות הסמאפור באופן תחילי.....
57.....	Mutex.....
59.....	CV.....
61.....	Critical Region.....
62.....	Monitor.....
63.....	Readers/Writers Lock.....
64.....	בעיות סנכרון קלאסיות.....
64.....	בעיית היצרן – צרכן = בעיית החוצץ המוגבל.....
66.....	בעיית הקוראים – כותבים.....
68.....	בעיית הפילוסופים הסועדים.....
69.....	Deadlock.....
70.....	מניעה.....
71.....	התחמקות.....
73.....	גילוי.....

74.....	התעלמות
74.....	היחלצות
75.....	מבוא לניהול הזיכרון
77.....	ניהול הזיכרון
78.....	הקצאה רציפה
80.....	Virtual Memory – זיכרון וירטואלי
80.....	Paging – אפשרות א
90.....	Segmentation – אפשרות ב
92.....	Segmentation + Paging – אפשרות ג
93.....	Virtual Memory – הרחבה
93.....	Demand Paging
94.....	Copy On Write – COW [העתקה בעקבות כתיבה]
95.....	Page Replacement – החלפת דפים
96.....	אלגוריתמים להחלפת דפים
99.....	דרוג האלגוריתמים להחלפת דפים
100.....	הקצאת frame'ים
101.....	אלגוריתמים להקצאת frame'ים
105.....	Trashing – דשדוש
106.....	Working Set – מודל קבוצת העבודה
107.....	Page Faults ניטור
107.....	Prepaging
107.....	Page Faults-הקשר בין מבנה התוכנית ל-
108.....	נעיצת חוצצים
109.....	IPC – תקשורת בין תהליכים
109.....	Channel
109.....	Pipes
111.....	Named Pipe ('משויים') / צינור בעל שם
112.....	Message Queue
114.....	Sockets
118.....	shared memory

120.....	memory mapped files
122.....	מבנה מערכת האחסון המסיבי – Mass-Storage Structure
125.....	אלגוריתמים לתזמון זרוע הדיסק
128.....	דיסק בזיכרון – RAM DISKS
129.....	מערכי דיסקים – RAID
132.....	ניהול שטח הדיסק
133.....	File System – מערכת הקבצים
134.....	קובץ
135.....	מדריך
135.....	סוגי קבצים ב-UNIX
138.....	שיטות גישה לדיסק
139.....	Directories ו- Partitions – ניהול קבצים
141.....	מודלים למבנה מערכת הקבצים
142.....	File System Implementation – מימוש מערכת הקבצים
143.....	ארגון מערכת הקבצים
143.....	שיטות להקצאת בלוקים בדיסק
146.....	Virtual File System – מערכת הקבצים הווירטואלית
149.....	Mounting – עגינת מערכת הקבצים
150.....	טעינת מערכת הקבצים
151.....	פעולות על קבצים

שיתוף קבצים

הרשאות

ביצועים

מערכות קבצים מתועדות

מערכות קבצים מבחורות

.....I/O מערכות

.....I/O ניהול קלט פלט

.....User Applications

.....OS

.....Device Driver

.....	Interrupt Controller
.....	Polling – תשאול
.....	Interrupts – פסיקות
.....	Direct Memory Access (DMA) – גישה ישירה לזיכרון
.....	Device
.....	BIOS ו- Bootstrapping
.....	סיווג מערכות
.....	וירטואליזציה

מבוא למערכות הפעלה

תפקיד מעה"פ הוא לתווך בין התוכנה לחומרה, תוך שמירה על יעילות ונוחות בהרצת תוכניות. כמו כן, היא תבטיח פעילות תקינה של המחשב: לנהל תוכניות באופן בו הן לא תתנגשנה זו בזו וכדו'. כדאי שהיא תדע גם לנהל את משאבי המערכת באופן מיטבי.

נוכל להגדיר את מעה"פ כתוכנית בקרה ושליטה שמתפעלת את השימוש במערכת.

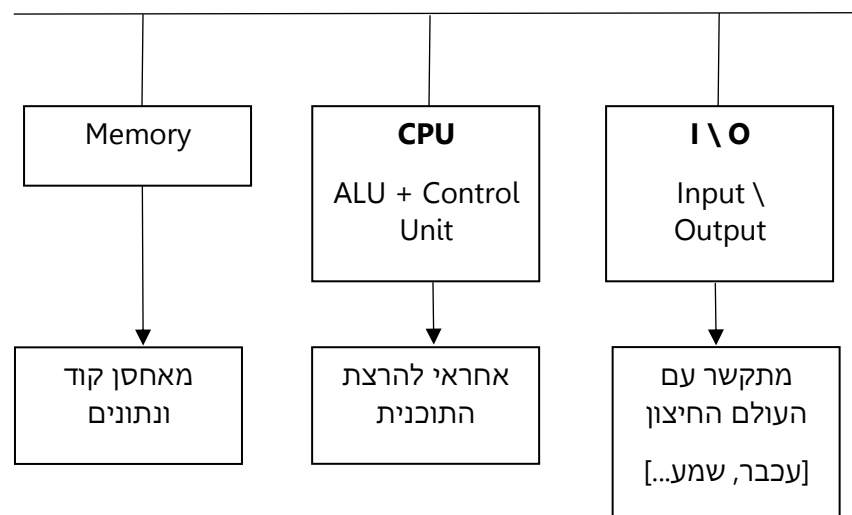
כידוע עולם המחשבים מתפתח בהתבססות על עולם המתימטיקה.

כיום נסתמך ונשען על מודל מתמטי שהומצא ע"י וואן ניומן – זהו מודל לעיצוב ותכנון מחשבים המכיל:

א. 4 תתי מערכות:

1. Memory – זיכרון
 2. ALU – יחידת העיבוד המתמטית של המחשב
 3. Control Unit – יחידת הבקרה
 4. ניהול וביצוע I/O
- ב. התוכנית מאוחסנת בזיכרון הראשי בזמן הריצה שלה
- ג. פקודות התוכנית מורצות ברצף

הערה: כיום יחידת ה-ALU מגיעה יחד עם ה-Control Unit ברכיב אחד המכונה CPU.



פירוט תתי – המערכות

1. Memory – היררכיית הזיכרון:

הזיכרון במחשב מורכב מסוגי זיכרונות שונים, כאשר: ככל שזמן הגישה לסוג זיכרון מסוים מהיר יותר- כך עלותו הכספית גבוהה יותר, וממילא גם הכמות ממנו ברכיבי המחשב קטנה יותר.

הזיכרון מתחלק ל-2 סוגים:

1. Volatile Memory – זיכרון נדיף: זיכרון שברגע הפסקת החשמל נמחק.
2. Non-Volatile Memory – זיכרון שאינו נדיף: נשאר צרוב גם בעת נתק חשמלי.

רכיבי הזיכרון – לפי סדר הקרבה למעבד, ולפי הגודל בביטים:

1. CPU Registers – אוגרי המעבד

אלו יחידות זיכרון קטנות בגודל של 32 או 64 ביטים.
זמן הגישה אליהם: 0.5ns (nano second : ננו שניה = 10^{-9})

2. Cache Memory – זיכרון מטמון

זמן הפניה לזיכרון מגיע ל-200ns, ואנחנו עלולים להגיע למצב בו המעבד מושבת כליל עד שיקבל את הנתונים שזקוק להם.
לכן נשתמש בזיכרון מטמון, שהפניה אליו מתבצעת לא דרך הפס (bus), אלא באופן יעיל יותר.
זמן הגישה אליהם: 1-10ns

3. D-RAM – הזיכרון הראשי

זהו זיכרון בעל זמן ריצה קבוע בכל מצב. מורכב מיחידות אחסון בגודל קבוע – גודל של תא בזיכרון הוא בערך 8 ביטים (Byte).
לכל תא בזיכרון יש תוכן וכתובת. כמות הביטים הנדרשת לייצג כתובת היא: מספר התאים²
על הזיכרון מוגדרות שתי פעולות:
1. fetch() – שליפת נתונים על הזיכרון
2. store() – אחסון נתונים בזיכרון
אין פעולת מחיקה, מכיוון שמבצעת דריסה לאחר שחרור מצביעים...
זמן הגישה אליו: 100ns

4. SSD (storage device) – הזיכרון המשני, הדיסק הקשיח

זמן הגישה אליהם: 100,000ns או 3-15ms

2. CPU – המעבד:

המעבד מורכב משני חלקים:

1. ALU (Arithmetic Logic Unit) – יחידת העיבוד המתמטית. יחידה האחראית על החישובים הלוגיים והמתמטיים. מורכבת משערים לוגיים.
2. Control Unit – יחידת הבקרה.

המעבד מבצע באופן בלתי פוסק 3 פעולות:

1. fetch() – פעולה הקוראת את הפקודה הבאה מהזיכרון
2. decode() – פעולה המפענחת את הפקודה.

3. execute() – הרצה בפועל של הפקודה.

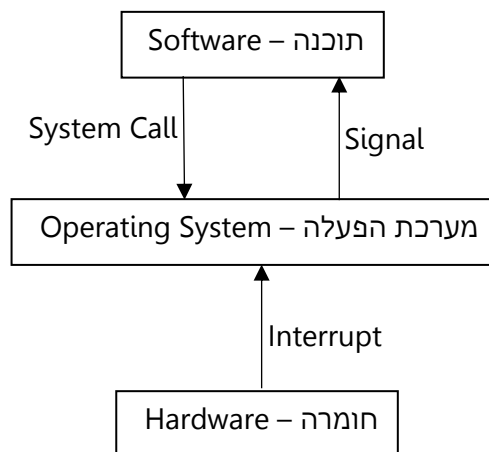
3. O \ I – ניהול קלט ופלט:

מפורט ב'שירותי מעה"פ'.

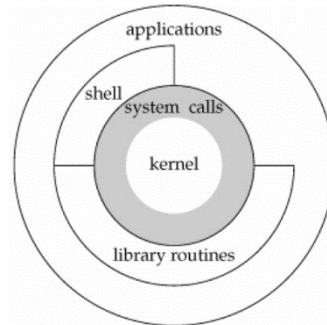
מבנה מערכת המחשב

מערכת המחשב בנויה מ-4 חלקים:

1. **Hardware** – חומרה – הכוח המבצע
2. **Operating System** – מערכת ההפעלה – תוכנית המשמשת כחוצץ בין המשתמש במערכת המחשב לבין החומרה
3. **Applications** – אפליקציות – כל התוכנות בהם אנו משתמשים
4. **Users** – משתמשים – בנ"א או מחשבים אחרים



מבנה מערכת ההפעלה



מערכת המחשב בנויה ממספר שכבות, כאשר לכל שכבה תפקיד משלה. צורת המבנה מספקת הגנה.

1. Kernel – גרעין:

התהליך שרץ כל הזמן על המחשב, ומבצע את הניהול בפועל. אחראי לניהול התהליכים, הקצאת הזיכרון הראשי וניהול הציוד – שירותים בסיסיים בלבד.

2. System Calls – קריאות מערכת:

מנגנון המספק את הממשק בין התהליך למערכת ההפעלה, ומהווה נקודת כניסה לקוד ה-kernel.

כלומר תהליך שולח למערכת ההפעלה System Call, והיא דואגת לביצוע הפעולה. מבחינתנו לא ניכר הבדל בין ביצוע ישיר של System Call לבין פונקציית ספריה המזמנת System Call [כמו scanf] או לפונקציית ספריה שלא מזמנת System Call [כמו strlen]. פונקציית הספרייה תשלח את הפרמטרים ל-kernel [מעמיסה אותם על המחסנית / על האוגרים / אחסון במקום כלשהו בזיכרון]. מדובר בפעולת שלא ניתנות לביצוע ב-User Mode.

3. Library – ספריות:

ספריות המגיעות כחלק ממערכת ההפעלה ומספקות שירותים שונים. כלומר: מטעמי נוחות- מערכת ההפעלה מבצעת משימות עבור המשתמשים (כמו: שליפת תאריך, הצגת תיקיות וכו') – אלו תוכניות מערכת.

4. Shell – שורת הפקודה:

זוהי מעטפת המספקת למשתמש גישה לשירותי ה-kernel. המעטפת יכולה להיות כ-CLI (שורת פקודה) או GUI – תצוגה למשתמש.

5. Applications – אפליקציות:

אפליקציות שהמערכת מספקת למשתמש (לדוגמה: מחשבון).

צורות שונות למבנה מערכת ההפעלה

1. מערכת הפעלה מונוליטית (Monolithic):

- מערכת הפעלה הכוללת את כל השירותים האפשריים
- + זמן קימפול אופטימלי כי הכול כבר יחד
- + אין צורך להשלים שירותים, מאותו הסיבה
- מכביד על המערכת שירותים מיותרים
- קושי בתחזוקה (חייב לוודא ששינוי לא פוגע בשירות קיים), וממילא משפיע על הניידות
- עומס על הזיכרון וירידת ביצועים

2. מערכת הפעלה מינימליסטית (Micro Kernel):

- מערכת הפעלה המכילה את השירותים הבסיסיים ביותר ורק אותם. שאר השירותים נחשבים לרכיבי תוכנה, וירוצו ב-User Mode. [יוצר overhead – תקורה]
- מתאימה למערכות זמן אמת, בקרה וכדו'.
- + ביצועים טובים
- + קל לתחזוקה
- קושי בניידות – נדרשות התאמות בכל שדרוג
- קושי בפיתוח תוכנות למערכת זו – כל מערכת תהיה שונה מאחרת
- תקורה בעקבות ריבוי context switch

3. מערכת הפעלה מודולרית (Modular):

- מערכת הפעלה בסיסית הניתנת לפירוק והרכבה.
- זוהי הגישה הנפוצה ביותר.
- + תחזוקה קלה ונוחה
- + דרישה פחותה של משאבים
- + קוד בסיסי קטן
- האטה בביצועים בעקבות צורת המימוש העקיפה
- אופן התחזוקה עלול לייצר באגים וקריסת מערכת [כל אחד מתקן איך שהוא חושב...]

Micro-Kernel

כפי שהזכרנו, מטרתנו היא למזער את גודלו של ה-kernel, ולהשתמש בשיטה המודולארית, ולא בשיטה המונוליטית.

בגישת ה-micro-kernel נבצע זאת, נשאיר רק את המינימום ההכרחי ביותר ב-kernel.

החלקים שנשארים מחולקים למודולים נפרדים, המתקשרים ביניהם באמצעות העברת הודעות, [מעה"פ מעבירה את ההודעות]. כך נשיג מודולאריות גבוהה, תחזוקה קלה, ניידות וביטחון.

מטרות מערכת ההפעלה

1. **Abstraction – הפשטה:**

ישנם התקני חומרה רבים, ולא ייתכן שכל מתכנת יצטרך להתאים את עצמו לכל יצרן חומרה באשר הוא. לכן באה מערכת ההפעלה ודואגת בעצמה לתאימות הזו, והופכת את השימוש לנוח יותר.

2. **Protection – הגנה:**

איננו רוצים שמידע על תהליך יהיה חשוף לגישה מתהליכים אחרים, לכן נצטרך לבצע הגנה על מרחב זיכרון פרטי לכל אחד.

3. **Utilization of resources – ניצול נכון של משאבים: (יעילות המערכת)**

ניהול חלוקה הוגנת של החומרה בין התוכניות והתהליכים השונים. מערכת ההפעלה אחראית על הקצאת משאבים, ומחליטה כמה משאבים להקצות לכל תהליך. במצבים של קונפליקט – מחליטה על פתרון, על מנת לשמור על שימוש יעיל והוגן. מערכת ההפעלה מנהלת את הרצת התוכניות – על מנת למנוע שגיאות ושימוש חריג במערכת, שעלולים לגרום לקריסת המחשב.

שירותי מערכת ההפעלה

1. **ממשק משתמש – UI:**
כך נוכל להזין פקודות למעלה"פ באופן טקסטואלי (CMD: פקודות והוראות, קובץ אינטראקטיבי – script), או באופן גרפי (GUI: חלונות, תפריטים, סמן עכבר וכדו')
2. **הרצת תוכניות:**
טעינת תוכנית לזיכרון, הכנות לביצוע, הביצוע עצמו וכו' הם מתפקידיה המובהקים של מעלה"פ.
3. **ביצוע I/O:**
נצטרך להתאים את ביצוע ה-I/O לציווד הרצוי (מסך, מקלדת וכו'). כמובן שמטעמי אבטחה לא ניתן הרשאה לתוכנית המשתמש לבצע את ה-I/O באופן עצמאי, אלא רק באמצעות system-call.
4. **ניהול מערכת הקבצים – File System:**
יצירת / מחיקת / עדכון / חיפוש קובץ – מתבצעים רק ע"י מעלה"פ, בביצוע system-call.
5. **תקשורת בין תהליכים – IPC:**
תהליכים רוצים לשתף ביניהם מידע או משאבים, ותפקיד מעלה"פ לאפשר להם לתקשר זה עם זה.
6. **מעקב וטיפול בשגיאות:**
תוכנית עלולה לייצר שגיאות, או אפילו החומרה נכשלת (בנפילת מתח, כשלון בכתיבה לדיסק וכדו') – ומתפקידי מעלה"פ לדאוג לנהל את הטיפול בבעיה. (אחת הדרכים – ע"י signal).
7. **הקצאת משאבים:**
מעלה"פ אחראית על החומרה, וממילא מנהלת את הקצאת המשאבים לכל תהליך בהתאם לדרישות ותוך שימת לב לא לקפח אף תהליך.
8. **רישום חיובים:**
רישום מדויק של השימוש במערכת, והקצאת משאבים לכל תהליך, זאת ע"מ לשפר ביצועים.
9. **הגנה ובטיחות:**
הגנה על הזיכרון מפני תוכניות פנימיות או התקפות חיצוניות.

תהליך ה-Boot – טעינת מעה"פ

בעת הדלקת המחשב מורצת תוכנית המכונה bootstrap loader/programmer. תפקידה לזהות את החומרה והזיכרון שבמערכת המחשב. תוכנה זו מכונה גם BIOS – Basic I/O System, והיא מאוחסנת ב-ROM.

בסיום ביצוע אתחולים – ה-BIOS יטען לזיכרון RAM את ה-boot block של מעה"פ.

ה-boot block יטען את שאר ה-kernel OS.

כעת נוכל לומר שמעה"פ רצה. היא תבצע אתחולים משלה (עדכון טבלת הפסיקות וכדו'), וממתינה לאירועים שיבואו. אירועים = פסיקות (תוכנה או חומרה. יפורט בהמשך).

Kernel mode

ליבת מעה"פ נמצאת תמיד בזיכרון, ואחראית לטפל בתהליכים, ניהול הזיכרון, ביצוע system-calls, וכן טיפול בציווד ובהתקנים. מטרתנו תמיד להקטין את גודלו של ה-kernel למינימום ההכרחי, וששאר השירותים יסופקו ע"י תוכניות מערכת. ה-kernel אינו תהליך, אלא למנהל התהליכים כולם.

בזמן שה-CPU מבצע קוד של ה-kernel מצבו מוגדר כ-kernel mode, ויש לו הרשאות לבצע פקודות מיוחסות, ולגשת לכל מקום בזיכרון. הקוד שרץ נמצא בכתובות השייכות ל-kernel.

כמובן שבכל פעם שנעבור לביצוע ב-kernel mode – ערכו של האוגר PC ישתנה.

User mode

בזמן שה-CPU מריץ קוד שאינו שייך ל-kernel מצבו מוגדר כ-user mode, והרשאותיו מוגבלות. לדוגמה – לא נוכל לטפל בווקטור הפסיקות, אלא נבצע system-call שיבצע זאת עבורנו.

תוכניות מערכת

תוכניות מערכת הן תוכניות שאינן שייכות ל-kernel, אך מסופקות מהמעטפת (shell, מחשבון וכו'). כשהן זקוקות לשירות ממעה"פ – יבצעו system call.

בין תוכניות המערכת:

ה-shell, כל פקודה שאפשר להריץ ב-shell היא תוכנית מערכת בפני עצמה, מנהל שולחן העבודה, תוכניות לעריכת קבצים, תומכי שפות תכנות, דפדפן ועוד.

System-Calls

כשתוכנית מעוניינת לבצע I/O או לעדכן נתונים – היא צריכה לבקש ממעלה"פ לבצע זאת עבורה, כלומר: התוכנית מועברת ל-kernel mode, וכעת רץ קוד של מעלה"פ שמבצע עבורה את הבקשה הרצויה.

הבקשה ממעלה"פ לבצע עבורנו פעולה מיוחדת נקראת system-call. בעקבות זימון הפונקציה מתבצעת פסיקה [פקודת אסמבלר מיוחדת] המפעילה את מעלה"פ עבורנו.

כדי לבצע system-call ב-UNIX נריץ את פקודת האסמבלר interrupt 0x80.

דוגמה- ביצוע exit לתוכנית: נריץ את הפקודה הנ"ל. לאוגר %eax נכניס '1', ולאוגר %ebx נכניס את ה-exit status הרצוי.

בכל אופן, תמיד נזמן את הפונקציה exit() שתבצע עבורנו הכול – ע"י system-call.

נוכל לחלק את ה- System Calls ל-5 קבוצות עיקריות:

א. בקרת תהליכים:

יצירת וסיום תהליך [באופן תקין או לא], טעינת תוכנית ע"י exec(), שליפה ועדכון תכונות של ה-PCB, המתנה X זמן או לסיום תהליך בן, שליחת signal לתהליך אחר, וכמובן להרצת פקודות התוכנית.

ב. הטיפול בקבצים:

יצירת / מחיקת / העתקת / העברת קובץ, פתיחת / סגירת קובץ לקריאה / כתיבה, תזוזה למיקום X בקובץ, שליפה ועדכון תכונות הקובץ. כל הנ"ל תקף גם לתיקיות directory.

ג. הטיפול בצידוד:

טיפול בצידוד חומרה כללי, ובעיקר צידוד I/O.

ד. אחזקת מידע:

קריאת התאריך והשעה, שמירת מספר המשתמשים, נתוני מערכת ההפעלה, שטח הדיסק הפנוי וכדו'.

ה. תקשורת:

בהעברת מידע בין תהליכים [באותו מחשב / במחשבים שונים], ברור כתובת IP, פתיחת / סגירת ערוץ תקשורת, קבלת / שליחת מידע על הערוץ, הקצאות ושחרורים ל-shared memory (זיכרון משותף) או כלים אחרים להעברת הודעות.

הפקודה לצפייה בכל ה- System Calls שהמערכת ביצעה: **strace -c**

פסיקות – Interrupts

כדי לאפשר למחשב להגיב לשינויים לא סטנדרטיים ו/או פתאומיים – נשתמש בפסיקות. פסיקה זהו אירוע הגורם למעבד להשהות את פעולתו הרגילה, ולעבור לבצע פעילות מיוחדת של מעה"פ ע"י שליחת איתות (signal) למעה"פ.

לדוגמה: בעת טיפול במשאבי חומרה, ביצוע system-calls, טיפול בתקלות בעת ריצה [כמו חלוקה באפס, וגישה שגויה לזיכרון].

מי שולח את הפסיקות?

התקני חומרה חיצוניים, תוכניות השולחות system-calls לביצוע פקודות מיוחדות, וכן מעבדים יכולים לשלוח פסיקות זה לזה. רק מעה"פ אחראית לטפל בפסיקות.

סוגי פסיקות:

פסיקות א-סינכרוניות (חיצוניות)	פסיקות סינכרוניות (פנימיות)	
רכיב חומרה, שאינו קשור דווקא לריצת התוכנית, בד"כ מבקר הפסיקות. וכן פסיקות שא"א להתעלם מהן. השליחה- ע"י איתות חשמלי לבקר הפסיקות.	פקודה בתוכנית הגורמת לחומרה או לתוכנה לייצר פסיקה. ניתן לדעת מראש שפסיקה כזו תתבצע. • פסיקה יזומה - <u>Trap</u> : משמשת להעברת מידע בין תהליך למעבד • חריגה - <u>Exception</u> : משמשת לדיווח על שגיאות שנוצרו ע"י פקודות.	השולח:

מבני נתונים לניהול הפסיקות:

1. ווקטור הפסיקות – Interrupts Vector או Interrupt Descriptor Table (IDT):
זוהי טבלה השומרת את מספרי הפסיקות, ועבור כל מספר שומרת מצביע לפונקציית הטיפול של הפסיקה. מספר השורות בטבלה יהיה כמספר ווקטורי הפסיקות.
קיימות 2 טבלאות נפרדות, לפסיקות תוכנה ולפסיקות חומרה.
הטבלה נשמרת בתחילת הזיכרון הראשי במחשב, ונטענת ע"י ה-BIOS. בד"כ מעה"פ לא מסתמכת על שגרות הטיפול הקבועות בטבלה, ומחליפות חלק ממנה.

2. רגיסטר דגל הפסיקות – IF (Interrupts Flag register):
זהו ביט שניתן לחסום באמצעותו קבלת פסיקות. כמובן שיש פסיקות שא"א לחסום אותן, כמו לדיווח על נפילת מתח, לחצן אתחול וכדומה.

אופן הטיפול:

המעבד יבדוק אם נשלחה אליו פסיקה, בין ביצוע של כל 2 הוראות עוקבות, תוך שהוא אינו קוטע ביצוע הוראת מכונה בסיסית.

נטפל בפסיקה כמה שיותר מהר (בהזזת עכבר עד 150ms שהמשתמש לא ירגיש) כמו כן, תמיד נטפל בפסיקות ב-context של התהליך שהמעבד מריץ כעת, גם אם אין קשר בין הפסיקה לבין התהליך. לכן לעיתים מעה"פ תחסום פסיקת context-switch ע"מ למנוע בעיות סנכרון עתידיות.

כשנסיים לבצע את הפסיקה, ה-CPU יחזור לבצע את הקוד מהמקום שעצר בו.

כשנעבור לביצוע פסיקה- נעבור ל-kernel mode, ובעת הצורך גם נחליף את המצביע למחסניות הרלוונטיות. שגרת (פונקציית) הטיפול בפסיקות נקראת Interrupt Service Routine – ISR [רוטינה לטיפול בפסיקות].

פסיקות חומרה – פירוט השלבים:

- בקר המבקש לשלוח פסיקה- שולח אות IRQ (Interrupt ReQuest) לעבר קו הפסיקה המחובר אליו. [הבקר יודע לרוב בין הפסיקות ולאפשר חוקיות מסוימת – כמו עדיפות לפסיקות ספציפיות].
- המעבד בודק אם התקבלה פסיקה בין כל 2 הוראות מכונה (כאשר IF דולק).
- כשמזוהה פסיקה – ה-CPU מבכה את הרגיסטר IF, ומבקש מההתקן אות – מהו מס' הפסיקה.
- ההתקן יחזיר את מס' הפסיקה הרצויה.
- ה-CPU ישמור את ערכי הרגיסטרים (כתובות חזרה) המסמנים את המקום בו נעצרנו מביצוע התוכנה, ויטען את שגרת הטיפול מווקטור הפסיקות.
- ה-CPU יבצע את שגרת הטיפול.
- ה-CPU יטען חזרה את ערכי הרגיסטרים בהם נעצרנו.

פסיקות תוכנה:

לא ניתן להתעלם מפסיקות תוכנה כלל!

כשה-CPU מזהה שמגיעה אליו פקודה מיוחסת (בעקבות system-call) – נעבור ל-kernel mode ונשלח trap למעה"פ.

Exceptions:

1. Faults – תקלות שאפשר לסדר ע"י חזרה על שורת הקוד האחרונה. לדוג': page-fault.
2. Aborts – תקלות חמורות שנגרמו כתוצאה מהרצת השורה האחרונה בקוד.

חשוב לדעת ש-Exceptions חשובים יותר מפסיקות חומרה רגילות.

ההבדלים בין פונקציות לפסיקות:

פונקציות רגילות:	פסיקות מערכת:	
1. סינכרוניות בלבד	א-סינכרוניות וסינכרוניות	
2. רצות ב-user mode	רצות ב-kernel mode	
3. לא מבצעות החלפת מחסניות	מבצעות החלפת מחסניות	
4. שומרות את מצב האוגרים	שומרות כתובת חזרה בלבד	

בעיות בטיפול בפסיקות:

המנגנון שתואר למעלה- כללי מידי, ועלולות להיווצר בו תקלות. לדוגמה: נרצה לדחות את הטיפול בפסיקות כאשר מריצים קטעים קריטיים, נרצה לוודא שאיתור שגרת הטיפול תתבצע באופן יעיל ועוד.

לכן כיום, המעבדים המודרניים **פותרים** את התקלות באופן מתקדם.

מעה"פ קובעת עדיפות לכל פסיקה, באופן שהפסיקות בעלות המספרים הנמוכים חשובות יותר. ואף אם ה-CPU טיפל בפסיקה ממספר גבוה – הוא יעבור לביצוע הפסיקה הנמוכה, ולאחר מכן ישוב לפסיקה במקום שעצר לבצע אותה.

נוכל להגדיר מסיכת פסיקות – mask register שתסמן אלו פסיקות ייחסמו ואילו יאופשרו תמיד. [0-לא מאופשר, 1-מאופשר]. וכן נוכל גם להגדיר את עדיפות הפסיקות: קריטיות פחות או יותר.

נדאג לסנכרן את מבני הנתונים, כך שבעת קיומן פסיקות נמנע אפשרות של deadlock.

Revectoring – אפשר לעדכן את ווקטור הפסיקות לשינוי שגרות טיפול.

סיגנלים – Signals

Signal הוא מנגנון להעברת הודעות לתהליכים ממעלה"פ על אירוע חריג שהתרחש בקשר אליהם, או להעברת הודעות בין תהליכים.

בהמשך נפרט על סיגנלים בין תהליכים, בפרק על IPC.

לכל סיגנל יש מספר ושם פרטי. הוא יכול להיות סינכרוני או א-סינכרוני. סינכרוני- אם נשלח כתוצאה מהרצת התוכנית, וא-סינכרוני אם הוא נשלח מחוץ לתוכנית (מתהליך אחר).

תהליך יכול לקבל סיגנל רק כאשר הוא רץ ב-user mode. אם הוא ב-kernel mode הסיגנל ימתין, ויכונה pending signal. בכל פעם שנתחיל לרוץ ב-user mode (לפעמים מספר פעמים באותו context!!) – נבדוק אם ממתנים לו סיגנלים.

תהליך המקבל סיגנל בד"כ מועף. אך נוכל להגדיר צורת טיפול אחרת: התעלמות מהסיגנל / תפיסת הסיגנל וטיפול בו באופן מסוים / חסימת הסיגנל בד"כ באופן זמני.

סיגנלים נפוצים:

- SIGFPE – שגיאה אריתמטית [חלוקת שלם באפס, גלישת מחסנית וכו']
- SIGSEGV – גישה לזיכרון לא חוקי עבור התהליך [זיכרון לא מאותחל, מחוץ ל address space- וכו']
- SIGBUS – גישה לזיכרון באופן שגוי [פניה למשתנה long עם כתובת בעייתית]
- SIGILL – ניסיון לביצוע פקודה לא חוקית [פקודה מיוחסת ב-user mode, פקודה שאינה קיימת וכו']
- SIGTERM – בקשה מתהליך כלשהו לסיים את פעולתו, אך התהליך יכול לדחות את הבקשה. [הפקודה kill p_pid שולחת באופן ב"מ סיגנל זה]
- SIGKILL – עצירת התוכנית בכוח. [הפקודה 9-kill תשלח סיגנל זה], מבלי אפשרות להתעלם ממנו.
- SIGINT – עצירת תהליך ע"י Ctrl+C. (Interrupt של INT) (קיצור של Interrupt)
- SIGQUIT – ע"י Ctrl+/. (קיצור של Interrupt)
- SIGALRM – טיימר להקצבת זמן אמיתי.
- SIGVTALRM – טיימר להקצבת זמן ריצה ב-CPU (סה"כ זמן ריצה כולל).

סיגנלים המיועדים לבקרת תהליכים:

- SIGSTOP – עצירת התהליך המרכזי שרץ כעת, מבלי אפשרות להתעלם ממנו.
- SIGTSTP – דומה לסיגנל הקודם, רק שהוא 'חלש' יותר, וניתן לתפוס אותו. ע"י Ctrl+Z
- SIGCONT – גורמת לתהליך שהושהה להמשיך לרוץ
- SIGCHLD – נשלח ל'אבא' של התהליך, כאשר התהליך מסתיים או מושהה.

סיגנלים לשגיאות מערכת:

- SIGPIPE – כאשר תהליך פתח PIPE או תור הודעות לכתיבה, ואין קורא בצד השני / נעלם.
- SIGXCPU – כאשר תהליך עבר את זמן הריצה ה-soft במעבד, אם הוא גם יעבור את זמן הריצה ה-hard במעבד יישלח לו SIGKILL.
- SIGXFSZ – כאשר תהליך מנסה לכתוב לקובץ ולהגדילו מעבר למותר במערכת.

קיימים גם סיגנלים לשימוש המתכנת – SIGUSR1 ו-SIGUSR2.

מבני נתונים לניהול סיגנלים:

1. ווקטור הסיגנלים של מעה"פ:
מכיל את מספרי הסיגנלים, ומצביעים לפונקציות ברירת המחדל לכל סיגנל.
2. ווקטור הסיגנלים – פרטי לכל תהליך:
נשמור בווקטור סיביות דלוקות בהתאם לסיגנלים שנשלחו.
3. ווקטור תגובה לסיגנלים:
נגדיר בו האם להשתמש בפונקציית הסיגנל המקורית ששייכת למעה"פ [SIG_DFL], או שנגדיר פונקציה אחרת [SIG_IGN].

רשימת הסיגנלים המלאה תוצג בהרצת הפקודה: **man -7 signal**

תפיסת סיגנלים:

יש סיגנלים שכדאי לתפוס אותם, ולטפל באופן מיטבי מבלי לנסות להתאושש מהם, ולגרום לתוכנית לסיים באופן נקי: לסגור משאבים, לשלוח הודעות שגיאה ולצאת.

פונקציית `signal()` יכולה לשנות את פונקציית הטיפול בסיגנל באופן חד פעמי. לאחר הפעם הראשונה שייתפס הסיגנל- התוכנית תבצע את פונקציית הטיפול שבחרנו, ותחזיר את ההגדרה לפונקציית הטיפול בסיגנל זה – לברירת מחדל. נשלח לפונקציה כפרמטרים את שם הסיגנל, ושם הפונקציה הרצויה לביצוע.

חסימת סיגנלים:

נוכל לחסום סיגנלים, כלומר: לגרום להם להמתין במאגר עד שנוכל לטפל בהם.

- הגדרה: 'מסיכה', היא מערך של סיביות בו כל סיבית מייצגת סיגנל. 1- דלוק, 0- כבוי]

כיצד נחסום? באמצעות אחת מהפונקציות הבאות:

1. `sigprocmask(int how, const sigset_t* set, const sigset_t* old_set)` – הפונקציה תקבל מספר שלם המסמן כיצד להתייחס לשני הפרמטרים הבאים. האם להוסיף / להסיר סיגנלים מהמסכה הקיימת, או להחליף את המסכה לחלוטין.
שני הפרמטרים הנוספים מכילים: מצביע למסכה החדשה שלפיה נעבוד, ומצביע למסכה הקודמת – למקרה שנרצה לשחזרה בהמשך.

פונקציות עזר לייצור המסכה:

- `sigemptyset(&set)` – ריקון המסכה
- `sigfillset(&set)` – מילוי המסכה
- `sigaddset(&set, SIGINT)` – הוספת סיגנל למסכה
- `sigdelset(&set, SIGINT)` – הסרת סיגנל
- `sigismember(&set, SIGINT)` – בדיקה אם סיגנל שייך למסכה

הבעיה בפונקציה זו: חוסר אטומיות, אם מגיע סיגנל לפני ביצוע הפונקציה `sigprocmask` – החסימה כבר לא תהיה שלמה. [ועלול להיווצר מצב מרוץ – race condition].
לכן הפונקציה `sigaction` מבצעת חסימת סיגנלים באופן אטומי.

2. `sigaction(int signum, const struct sigaction* act, struct sigaction* oldact)` –

הפונקציה תקבל את מספר הסיגנל שיש לתפוס, מבנה המכיל את מסיכת הסיגנלים החדשה + הפונקציה לכל סיגנל + מסיכת סיגנלים בכל פונקציה, ומבנה נוסף המכיל את אותם הפרמטרים – עם הערכים שהיו בהם עד כה. אם לא נרצה לשחזר נוכל לשלוח NULL.

סיכום פונקציות סיגנל נפוצות:

הסיגנל:	תפקיד:	ערכים מתקבלים:
<code>kill(process_id, sig_name)</code>	שולחת סיגנל כלשהו	PID של התהליך אליו נשלח הסיגנל, ושם הסיגנל הרצוי.
<code>signal(sig_name, handler_func)</code>	שינוי פונקציית התגובה	שם הסיגנל, ופונקציה חילופית.
<code>sigprocmask()</code>	מיסוך סיגנלים באופן אטומי	מפורט למעלה
<code>sigaction()</code>	שינוי פונקציית התגובה + מיסוך סיגנלים באופן אטומי	מפורט למעלה
<code>sigint()</code>	עצירת התוכנית	
<code>sigurg()</code>	קריאת חומר רב מ-socket לקובץ	
<code>sigttou()</code>	התהליך הבא מוכן לכתיבה	
<code>sigxfsz()</code>	גודל הקובץ הגיע לגבול	

השוואה בין סיגנלים לפסיקות:

סיגנלים:	פסיקות:	
אירועים המופעלים על ידי המעבד והתוכנה הפועלת עליו	אירועים שנוצרו מבחוץ, לא מהמעבד. I/O, טיימר וכו'.	1.
נקבע ע"י מעה"פ	נקבע ע"י הפלטפורמה הפיזית	2.
סינכרוניים ואסינכרוניים		3.
בעלי ID ייחודי		4.
ניתנים למיסוך		5.

תהליכים – Processes

בעבר, מערכות מחשב הריצו תוכנית יחידה בכל נקודת זמן, והיא שלטה על כל המחשב והחומרה. כיום, המחשב מריץ מספר תוכניות הנמצאות בו זמנית בזיכרון, כך שהמעבד מריץ כל תוכנית לפרק זמן $X ==$ מתבצע שיתוף זמן – Time Sharing.

כדי לאפשר מצב בו התהליכים לא פוגעים זה בזה למרות הימצאותם בו זמנית בזיכרון, נצטרך למנוע מתוכנית לבצע פעולות מסוימות באופן עצמאי, והיא תצטרך לבקש ממעלה"פ לבצע אותן עבורה – ע"י בקשת System Call.

הגדרה: Process – תהליך = תוכנית שקיימת בזיכרון וכעת רצה. (יש כאלה שמכנים זאת כ'ישות').

בעבר, במערכות אצווה (batch) הריצו ג'ובים – כל ג'וב מתחילה ועד סוף, בזמן שג'וב היה ב-I/O המעבד היה מושבת. לא חבל?

הפתרון: Multi – Programing (ריבוי תוכניות). נחזיק בזיכרון מספר תוכניות (ג'ובים) במקביל, ובזמן ביצוע I/O הג'וב יוותר מרצונו הטוב על המעבד – וכך נגיע לשיתוף זמן.

לכל תהליך יש:

PCB – טבלה בזיכרון המכילה את מצב התהליך והזיכרון שלו. הטבלה תחומה בין כתובת V_0 ל- V_{max} . התחום כולו מכונה address space – מרחב כתובות. התהליך רשאי לפנות רק לכתובות שבטווח זה. [הוא יודע מה הטווח באמצעות אוגר הבסיס ואוגר הגבול – base ו-limit]. שמורים בו:

- | | |
|--|--|
| <div style="display: flex; align-items: center;"> <div style="writing-mode: vertical-rl; transform: rotate(180deg); margin-right: 10px;">address space</div> <div style="border: 1px solid black; padding: 5px; text-align: center;"> <div style="display: flex; justify-content: space-between; margin: 0;"> V_{max} מחסנית </div> <div style="text-align: center; margin: 5px 0;">↓</div> <div style="display: flex; justify-content: space-between; margin: 0;"> ↑ ערמה </div> <div style="border: 1px solid black; padding: 2px; margin: 2px 0;">bss</div> <div style="border: 1px solid black; padding: 2px; margin: 2px 0;">data</div> <div style="border: 1px solid black; padding: 2px; margin: 2px 0;">text</div> <div style="display: flex; justify-content: space-between; margin: 0;"> V_0 </div> </div> </div> | <ul style="list-style-type: none"> • text / code – פקודות התוכנית • stack – עבור הפרמטרים של התוכנית, קריאות לפונקציות ומשתנים מקומיים. • heap – עבור הקצאות דינאמיות במהלך התוכנית. • data – עבור משתנים סטטיים וגלובליים מאותחלים. • bss – block started by symbol משתנים גלובליים וסטטיים שאינם מאותחלים. • מידע נוסף על התהליך: מבנה נתונים המכונה 'מתאר התהליך' ומכיל כמאה שדות: רשימת קבצים שהתהליך פתח, עדיפות, זמן ריצה, מתעדכן בכל context switch – מצבי האוגרים (PC, IR, PSW, Base, limit), ה-pid שלו (כמו ת"ז לתהליך), מיקום זיכרון משותף, מצב התהליך וכו'. <p>המידע מסונכרן עם מבני הנתונים של מערכת ההפעלה.</p> |
|--|--|

יצירת תהליך:

יצירת תהליך מתבצעת תמיד ב-kernel mode. כאשר נייצר תהליך בן ב-UNIX בעת הולדת הבן הוא זהה [כפיל] של אביו. דוגמאות:

1. התהליך `Swapper \ Idle` מנהל את הזיכרון, והוא יוצר את `INIT`
2. בטעינת מעה"פ נוצר התהליך `INIT` שרץ לנצח, וה-`pid` שלו הוא 1.
3. בביצוע `login` – התחברות לחשבון משתמש במעה"פ. תהליך זה הוא בן של `INIT`, והוא עובר 'מוטציה' כלומר: נטעין על גביו תוכנית אחרת-`shell`.
4. בכל הרצת פקודה ב-`shell` נוצר תהליך המממש את התוכנית שהפקודה מבצעת.
5. בהרצת `script` המייצר תהליכים סדרתית.
6. ייצור תהליכים מסיבות נוספות.

הפונקציה לייצור תהליך בן: `fork()`

הפונקציה מפעילה `System Call` ייעודי, שייצור תהליך בן לתהליך שביצע את הקריאה. במערכות מסוימות הבן חולק עם אביו את משאבי האב [← הפחתת העומס על המערכת בעקבות ילדים רבים]. וכן בד"כ הבן מקבל נתונים מאביו. מבחינת ה-`address space` יכול להיות שהבן הוא העתק של האב או שהילד מבצע תוכנית אחרת משל עצמו – ע"י הפונקציה `exec()`.

חשוב להדגיש: כאשר תהליך יוצר תהליך בן – מועתק ה-`PCB` של האב לבן, הכול מועתק כולל הקוד והנתונים. מזה יוצא שגם הערך של ה-`PC` מועתק לבן, והבן ירוץ מהשורה שאחרי ה-`fork()`.

במספר פרמטרים יש הבדל, לדוגמה: ה-`pid` של הבן שונה משל האב, סטטוס חזרה מה-`fork()`.

הפונקציה `fork()` מחזירה מספר שלם המעיד על תוצאת ההרצה. בכישלון יוחזר -1, ובמקרה של הצלחה יוחזר לתהליך האב ה-`pid` של תהליך הבן החדש, ולתהליך הבן יוחזר 0. כך יש לנו מידע מי האב ומי הבן, והאב יוכל להמתין לבן מסוים לצורך כלשהו.

בעת יצירת התהליך – האב יכול להמשיך לרוץ במקביל לבנו או לעבור למצב השהיה והמתנה לסיום הבן ע"י הפונקציה `wait()`.

קיימת הגבלה על כמות התהליכים שהמערכת יכולה לייצר.

הפונקציה יצירת תהליך בן שחולק עם אביו את משאביו: `clone()`

אם נקבע שהתהליך החדש יחלוק עם אביו את מרחב הכתובות, מצביעי הקבצים, ומיסוך הסיגנלים – בעצם נייצר `kernel thread`. האב יהיה חייב להעביר לבנו מצביע לקצה המחסנית [כי ב-Linux מחסניות גדולות כלפי מטה] – ע"מ למנוע התנגשות

הפונקציה לטעינת תוכנית חדשה: `exec()`

הפעולה גורמת ליצירת מוטציה של תהליך הבן מהאב. כלומר: כשנפעיל פונקציה זו- כל ה-`address space` של התהליך שרץ עד כה מוחלף לגמרי, ונעבור להריץ תוכנית אחרת שטיטען לזיכרון ותרוץ מתחילתה. הפונקציה מקבלת כפרמטרים את: שם התוכנית, ומערך ארגומנטים עבורה.

סיום תהליך:

מתבצע תמיד ב-kernel mode. כשתהליך מסתיים הוא חייב לקבל אישור מההורה שלו על סיום הביצוע, ורק כשמתקבל האישור – משאביו מפונים מהזיכרון!. אם האב סיים את פעילותו לפני סיום הבן- התהליך INIT יאמץ אותו. דוגמאות:

1. התהליך סיים את פעולתו בהצלחה או בכישלון (ע"י הפונקציה `exit()`)
2. התהליך ביצע פסיקת תוכנה חמורה (כמו חלוקה ב-0)
3. מעה"פ מעיפה אותו מהמערכת בעקבות קיפאון (deadlock)
4. אביו שלח לו פקודה לסיים עכשיו
5. המשתמש ביצע `logout` – התנתקות מהחשבון.

הפונקציה לסיום תהליך: `exit()`

הפונקציה מחזירה status – ערך המציין את סיום התהליך, ערך זה נשלח לאביו של התהליך. היא שולחת סיגנל SIGCHLD לאב. בד"כ האב יטפל בסיגנל זה ע"י זימון פונקציית טיפול בסיגנל- שתכלול קריאה ל-`wait()` ואז נמנע חסימה או המתנה של האב לשווא, אלא רק כאשר יש סיבה לבצע `wait()`.

אם האב לא רוצה לקבל דיווח על סיום ילדיו- הוא יכול לשנות את פונקציית התגובה שלו לסיגנל זה ע"י ביצוע: `signal(SIGCHLD, SIG_IGN)`

הפונקציה להמתנה לסיום של אחד מילדי התהליך הנוכחי: `wait()`

הפונקציה תחזיר:

- 1- במקרה שאין ילדים כלל, או שכולם כבר סיימו את פעילותם
- Pid של תהליך בן הנמצא במצב zombie. נוכל לשלוח NULL כפרמטר לפונקציה אם לא נרצה לקבל מידע הכולל דגלים על סיום התהליך
- כאשר עדיין יש בנים פעילים- האב ייחסם (יעבור ל-blocked) עד לקבלת ערך חזרה.

גרסה נוספת לפונקציית המתנה לסיום תהליך בן: `waitpid()`

נוכל להמתין לילד מסוים שיסתיים, או לא להמתין לו ולהחזיר כישלון.

הפונקציה לקבלת מזהה התהליך: `getpid()`

ע"מ לקבל את ה-pid של התהליך הנוכחי נשתמש בפונקציה זו.

הפונקציה לקבלת מזהה ההורה של התהליך: `getppid()`

ע"מ לקבל את ה-pid של אביו של התהליך הנוכחי נשתמש בפונקציה זו.

הפונקציה לזימון X בעת ביצוע `exit()` לתהליך: `atexit(func)`

הפונקציה תזמן את ה-func שנשלחה כפרמטר כאשר מתבצע `exit()` לתהליך [destructor]

קשרי משפחה בין תהליכים:

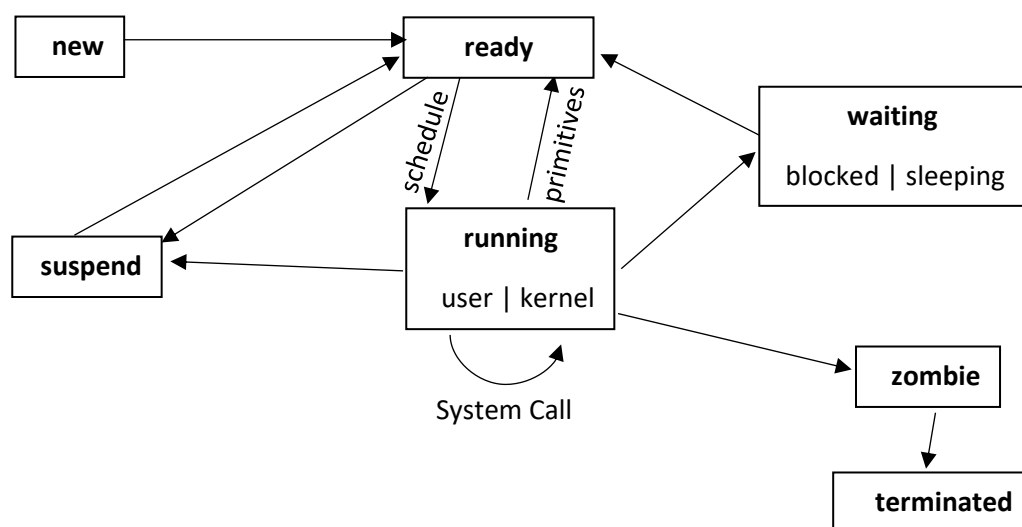
קשר משפחתי מתבצע באמצעות שמירת מצביע למתארי התהליכים. לכל תהליך יש:

שם המצביע:	הסבר:	מצביע ל:
P_CPTR	CHILD	בן הצעיר ביותר של התהליך
P_OPTR	OLDEST PARENT	אב התהליך
P_OSPTTR	OLDEST SIBLING	אח שנולד לפניו
P_YSPTR	YOUNGEST SIBLING	אח שנולד אחריו

מצבי התהליך:

מצב התהליך – מתאר מה התהליך עושה כעת.

1. new – התהליך 'נולד', והוא מקבל את מבני הנתונים הדרושים שיאפשרו לו לרוץ במערכת.
2. ready – התהליך מוכן להרצה, יש לו את כל המשאבים שהוא זקוק להם, והוא ממתין לתזמון מה-scheduler, כלומר: שיוקצה לו זמן לריצה על המעבד. זהו תור של תהליכים במצב זה.
3. running – התהליך זכה במעבד, ורץ עליו כעת.
מצב זה מתחלק כמובן לריצה ב-user mode או ב-kernel mode.
4. waiting – התהליך ממתין לאירוע כלשהו.
מצב זה מתחלק ל:
blocked – התהליך ממתין ל-I/O, ממתין לאיתות, או למנעול שישתחרר וכדו'.
sleeping – התהליך מרדם את עצמו לזמן קצוב = מחכה שתגיע נקודת זמן X.
יש כאן תורים לפי האירוע שממתינים אליו. תור ל-I/O, תור למנעולים וכו'.
5. suspend – המערכת הפעלה משהה תהליכים מסוימים, ומורידה אותם מה-RAM לדיסק.
6. zombie – התהליך סיים את פעולתו, וממתין שאביו יתעניין בגורלו.
7. terminated – התהליך מועף מהזיכרון, נשחרר את המצביעים אליו וכו'.



סוגי תהליכים:

1. תהליכי אצווה – מורצים ברקע, ואין חשיבות לזמן התגובה שלהם.
 2. תהליכים אינטראקטיביים – יש חשיבות רבה לזמן התגובה למשתמש. ממתנים הרבה ל-I/O. תמיד נקצה עוד זמן לתהליך כזה, אם נגמר לו הזמן, אלא א"כ יש תהליך אחר עם עדיפות גבוהה ממנו, או שיש זקן ממנו.
- המעבד מבחין ביניהם באופן דינאמי – לפי כמות זמן ההמתנה של התהליך.

Context Switch

זהו מצב בו מתחלפים התהליכים שרצים על המעבד.

כיצד מתבצעת הפעולה?

נשמור את מצב התהליך שיורד, ונטען על גביו תוכנית חדשה. יש כאן תקורה נלווית – overhead.

התקורה מתחלקת ל-2:

1. תקורה ישירה – יש השפעה ישירה על כמות העבודה למעבד כתוצאה משמירת וטעינת האוגרים של התהליכים המתחלפים.
 2. תקורה עקיפה – כאשר תהליך רץ, הרבה מהנתונים נשמרים בזיכרון המטמון (Cache) כשתהליך חדש רץ על המעבד הוא יביא את הנתונים שלו למטמון, ומה שיקרה זה שהנתונים הקודמים יידרסו. לכשישוב התהליך הראשון לריצה על המעבד הוא יצטרך להביא שוב את הנתונים שלו, ובכך איבדנו את רוב המטרה בזיכרון מטמון... וכן תהיה ירידה בביצועים.
- ננסה למזער את התקורה ככל שביכולתנו:

- עלות הפעולה תלויה בחומרה: בכמה אוגרים נטפל, והאם קיימות פקודות מכונה שמבצעות שמירה וטעינה של אוגרים באופן אטומי.
- למנגנון ההחלפה של תרגום הכתובות יש השפעה רבה על התקורה (כמה תהליכים יעבדו באופן יעיל או סלקטיבי).
- ביצוע context switch ע"י S/W מתבצע באופן מהיר יותר, כי שמירת האוגרים נעשית באופן סלקטיבי.

הפקודה להצגת התהליכים שרצים כרגע במערכת: **ps -a**

תזמון תהליכים – Scheduling

לאורך חיי התהליך – הוא נודד בין תורי תזמון שונים, כשהוא ממתין לקבלת שירותים מגוונים. תפקידה של מעה"פ הוא לבחור איזה מבין כל התהליכים יקבל את השירות שהוא ממתין לו. התוכנית במעה"פ שמבצעת זאת נקראת – Scheduler (מתזמן).

סוגי תזמון:

1. תזמון ארוך טווח – בוחר מי יהיו התהליכים שיכנסו להמתין למעבד. (באצווה) שולט על רמת ריבוי התוכניות, ומשימתו לאזן בין סוגי התהליכים: תהליכי I/O ותהליכי CPU ע"מ להגיע לניצולת גבוהה של הציוד.
 2. תזמון קצר טווח – בוחר מי יהיה התהליך שירוצץ עכשיו במעבד. (ב-UNIX)
 3. Swapper – מתזמן ביניים / משחלף. מעביר תהליכים למצב suspend (כשהמעבד עמוס בתהליכים, חלק מהם מועברים ל-disk) זה מחליף חלקית מתזמן ארוך טווח ב-UNIX.
- מתי נבחר תהליך להרצה?** כשהתהליך שהורץ סיים / התבצע System Call / תהליך שהיה blocked עבר ל-ready / התהליך שרץ עבר ל-ready כי נגמר לו הזמן.

ה-Dispatcher

תפקידו של המשגר הוא **לממש** את החלטת המתזמן: לבצע את ה-context switch, לעבור ל-user mode בעת הצורך, לטעון את האגרים הנכונים של התהליך שנבחר שימשיך לרוץ עם הנתונים שלו. חובה עלינו לדאוג שגם ה-scheduler וגם ה-dispatcher יהיו מהירים, מכיוון שיש לנו שימוש תדיר בהם.

חביון השיגור = הזמן שעובר בין הפסקת תהליך A עד תחילת ריצת תהליך B. זו תקורה שנשאף למזער תמיד...

מבוא לאלגוריתמים לתזמון תהליכים

אלגוריתם שאינו חוטפני – ייתן לתהליך שתוזמן את מלוא הזמן שהוא נזקק לו ע"מ לסיים את ביצועו.
אלגוריתם חוטפני – ניתן לכל תהליך זמן מוקצב מראש לפי פרמטר מסוים. נצטרך לדאוג שלא תתבצע חטיפה באמצע עדכון DS כלשהו או באמצע ביצוע System Call – כי אם כן תתבצע חטיפה בזמן כזה התוצאה עלולה להיות הרת אסון 😞. גישת שיתוף הזמן היא חוטפנית במהותה.

לפי מה נחליט איזה אלגוריתם תזמון טוב יותר?

1. נמדוד את ניצולת המעבד – CPU Utilization – תמיד נעדיף להעסיק את המעבד כמה שיותר.
2. נמדוד את התפוקה – throughput – נעדיף להספיק כמה שיותר תהליכים ב-X יחידות זמן
3. נמדוד את זמן הביצוע הממוצע – Average Turnaround Time – כמה זמן לקח למעבד להשלים את ביצועו של כל תהליך.
4. נמדוד את זמן ההמתנה הממוצע – Average Waiting Time – כמה זמן המתין התהליך סה"כ עד סיומו במערכת.
5. נמדוד את זן התגובה – Response Time – כמה זמן לקח עד שהמשתמש קיבל פלט ראשון.
6. נבדוק את רמת ההגינות: עד כמה יש אפליה בין תהליכים?
כפרמטר השוואה – אנו בעיקר נתעניין בזמן ההמתנה הממוצע.

חישובים:

תפוקה – כמה משימות הושלמו בשנייה?

$$\frac{\text{מספר המשימות}}{\text{הזמן הכללי}}$$

Completion – זמן ממוצע לסיום משימה.

$$\frac{\text{סכום כל זמני המשימות, מהרגע שהגיעו עד שהסתיימו}}{\text{כמות המשימות}}$$

Wait – זמן המתנה ממוצע של כל משימה.

$$\frac{\text{סכום זמני ההמתנה של כל המשימות}}{\text{כמות המשימות}}$$

אלגוריתמים לתזמון תהליכים

1. First Come First Served – FCFS

נריץ את התהליכים לפי סדר הגעתם לתור ה-ready, ללא חטיפות.

[ממומש ע"י תור]

הבעיה: ייווצר Convoy Effect 'אפקט השיירה' – תהליכים קצרים ימתינו זמן רב לתהליך ארוך עד שיסתיים.

לדוגמה – נתון כי בתור הכניסה נמצאים: [השלימי את החישוב]

תהליך:	CPU burst time:	תפוקה:	Completion	:Wait
P1	5			
P2	17			
P3	9			
P4	18			

2. Shortest Job First – SJF

נריץ את התהליך הבא מתור ה-ready שזקוק להכי מעט זמן ב-CPU, ללא חטיפות.

[ניתן למימוש ע"י AVL, ערמת מינימום, רשימה ממוינת, מערך ממוריד]

הבעיה:

א. קל למימוש רק למערכות אצווה – רק בהן המשתמש מגביל את זמן ריצת התהליך.

ואילו במערכת אינטראקטיבית נוכל רק לנסות לחזות את הזמן הדרוש.

ב. גורם להרעבה לתהליכים ארוכים.

לדוגמה – נתון כי בתור הכניסה נמצאים: [השלימי את החישוב]

תהליך:	CPU burst time:	תפוקה:	Completion	:Wait
P1	5			
P2	17			
P3	9			
P4	18			

3. Shortest Remaining Time First – SRTF

זהו המימוש החוטפני של SJF. כלומר: במצב בו מגיע תהליך עם זמן קצר יותר מהתהליך הנוכחי המעבד ייחטף ויעבור אליו.

לדוגמה – נתון כי בתור הכניסה נמצאים: [השלימי את החישוב]

תהליך:	CPU burst time:	זמן הגעה:	תפוקה:	Completion	:Wait
P1	5	0			
P2	17	1			
P3	9	2			
P4	18	3			

4. Priority – עדיפות

ב-SJF נתנו עדיפות לכל תהליך לפי זמן הריצה שהוא דרש על המעבד. כאן ניתן לתהליך את המעבד לפי עדיפות שתיקבע ע"י מעה"פ או ע"י המשתמש. נוכל לממש אלגוריתם זה באופן שאינו חוטפני, או באופן חוטפני. הבעיה: נגרם להרעה לתהליכים בעדיפות נמוכה. הפתרון: Aging – הזדקנות: ככל שתהליך מזדקן וממתין במערכת – עדיפותו תעלה.

לדוגמה – נתון כי בתור הכניסה נמצאים: [השלימי את החישוב]

תהליך:	CPU burst time:	עדיפות:	תפוקה:	Completion	:Wait
P1	5	5			
P2	17	1			
P3	9	7			
P4	18	3			

5. Round Robin

זהו אלגוריתם מצוין למערכות שיתוף זמן. האלגוריתם מממש את FCFS עם חטיפת המעבד לאחר X זמן, והתהליך שנחטף ממנו המעבד מועבר שוב להמתנה למעבד. הזמן המוקצב לכל תהליך נקרא: Quantum \ Time Slice

- כיצד נקבע את ה-Quantum?
- אם הוא אינסופי = נקבל את FCFS כרגיל
- אם הוא קטן מידי = ניכנס למצב trashing כלומר- יתבצע בלולאה כל הזמן:

1. Interrupts

2. Scheduling

3. Context Switch

לכן ניקח את כלל האצבע הבא: נקבע את Q כך שב-80% מהמקרים התהליך יוותר על המעבד מרצונו, וב-20% נבצע חטיפה. ובעצם - ל-CPU bound ניקח Q גדול, ול-i/o bound ניקח Q קטן.

לדוגמה – נתון כי בתור הכניסה נמצאים: [השלימי את החישוב]

Wait:	Completion	תפוקה:	CPU burst time:	תהליך:
			5	P1
			17	P2
			9	P3
			18	P4

6. Multi Level Queue – MLQ

לעיתים נרצה לחלק את התהליכים לקבוצות, לדוגמה: לפי סוג התהליך. אלגוריתם זה מחלק את כל התהליכים למספר תורים נפרדים, ולכל תור מוגדר אלגוריתם תזמון משלו. ואילו התזמון בין התורים מתבצע לפי עדיפות.

הבעיה: יש הבדל גדול מאוד בין זמן ביצוע CPU לבין זמן ביצוע I/O – לוקח הרבה יותר זמן ל-I/O להתבצע. כך תהליכי I/O יוגבלו מאוד בעקבות מס' נקודות:

- הם לא ישתמשו בכל הזמן שהוקצה להם
- ימתינו זמן רב בתור ל-I/O
- שירות ציוד ה-I/O איטי יחסית וזמן הביצוע שלו גם לוקח זמן

הפתרון: נרצה להעביר תהליכים כאלו לתור 'אקספרס' שיעביר אותם מתור לתור ללא בעיות.

Multi Level Feedback Queue – MLFQ .7

באלגוריתם זה נוכל להעביר תהליכים בין התורים עצמם. כך שתהליך הזקוק לזמן רב- ירד לתור בעדיפות נמוכה, כמובן שתוך התחשבות במצב הזדקנותו. כלומר: כולם מתחילים מהתור בעל העדיפות הגבוהה ביותר. אם הוא לא הסתפק בזמן שהוקצב לו- הוא 'יוענש' ויועבר לתורים נמוכים יותר. בעת הצורך- הוא יועלה בחזרה מעלה. [לתור בעדיפות גבוהה- ניתן קוואנטום קטן, ולהפך. אפליה מתקנת למי שסיים מוקדם. עדיפות דינאמית לפי השימוש במעבד שמורכבת גם מהעדיפות ההתחלתית].

Threads

כשנרצה לכתוב תוכנית המבצעת מספר פעילויות במקביל – תהליך יחיד ייתקע מידי פעם בהמתנה ל-I/O, למידע מתהליך אחר וכדו', ולא נוכל לבצע אפילו פעולות אחרות שלא זקוקות ל-I/O בזמן ההמתנה מכיוון שכל התהליך נחסם.

נוכל לנסות לפתור זאת ע"י חלוקת התהליך למס' תהליכים בנים. אך אז יהיו הגבלות כמו:

- המידע שלהם לא יהיה משותף מלכתחילה, ואם נשתמש בכלי שיתוף IPC עדיין נשאר עם תקורה והאטה בביצוע התוכנית.
- כל יצירת תהליך היא פעולה יקרה.
- יהיו תהליכים שיבצעו פעולות דומות, וה-code שלהם יהיה זהה = בזבוז זיכרון.
- נוכל להגיע להצפת המערכת בתהליכים

הפתרון: Thread – תהליכון. יהיו כאלה שיכנו אותו חוט / LWP (Light Weight Process)

באופן עקרוני, תהליך יחיד יוכל להריץ פונקציות במקביל, וכולן יתבססו על המידע שב-PCB של התהליך המכיל אותן. במילים אחרות תהליכון הוא: יחידת ביצוע פעילה של תהליך, ועובד במקביל לתהליכונים נוספים ← נהיה חייבים לספק תיאום.

כדי שתהליכונים יבצעו את הפונקציה שלמענה הם נוצרו, כל אחד ישמור PC וקטע מהמחסנית לעצמו בלבד. מישע זה נשמר ב-TCB (Thread Control Block) – מבנה נתונים השומר מידע על התהליכון. לדוגמה: אוגר ה-PC, רישומי רגיסטרים, ומצביע ל-stack של התהליכון, id ייחודי עבורו, ומידע על המשאבים שזקוק להם.

אם נרצה שלכל תהליכון יהיו משתנים משלו, ולא יישמרו ע"ג ה-stack אך עדיין יהיו גלובליים לכל התהליכון – בכל הפונקציות שהוא יריץ – נוכל לאפשר זאת ע"י הקצאת המשתנה ב-TSD (thread specific data). נגדיר משתנה גלובלי מסוג pthread_key_t ונאתחל אותו בכל תהליכון לפי מה שנרצה.

יתרונות ה-Threads:

1. מקביליות <= מהירות
2. שמירה על מטמון חם
3. יעילות בדרישות הזיכרון וגם הקלה על התקשורת בין התהליכונים.
4. החלפת קשר מהירה: סה"כ שינוי של מספר רגיסטרים.

הרווח למעשה"פ: יכולה לסנכרן בין התהליכים בקלות ואף בין מערכות חומרה שונות.

הרווח לתהליכים: יתרון מספר 1

אתגרים:

1. נצטרך להגן על מידע השייך לתהליכון
2. נצטרך לסנכרן בין התהליכונים כאשר הם מבצעים משימה משותפת או טיפול במשתנה משותף. זאת כדי להימנע מחשש ל-race condition

User Thread

מסופק ע"י ספרייה **חיצונית**, ומנוהל בה. מבחינת מעה"פ מדובר בתהליך אחד, ואילו פונקציית הספרייה יודעת לתת חלק מהזמן שהוקצה לתוכנית כולה לתהליכון זה. כל ה-thread switch נמצא באחריות הספרייה.

תהליכון תמיד יוותר על המעבד ע"י pthread_yield().

יתרונות: מהירות עבודה – context switch מהיר, חלוקת זמן הוגנת, קל לבצע porting, יצירת תהליכון זולה מאוד, שליטה על context switch ע"י המתכנת, וכן גמישות עם המעה"פ.

חסרונות: קריאת מערכת תחסום את כל התהליכונים של התהליך המקורי [כי מבחינת המעה"פ מדובר בתהליך אחד], וכן בריבוי מעבדים אין מקביליות. כלומר: התהליכונים ירוצו על מעבד אחד בלבד.

פתרון לבעיית חסימת החוטים בביצוע קריאת מערכת חוסמת: הספרייה [כבר בזמן הקמפול!] תתפוס את ה-system call, ונשלח למעה"פ במקומו system call שאינו חוסם. את ה-thread ששלח את הבקשה- נשהה, ובינתיים נמשיך להריץ את שאר ה-threads שלא מבצעים פעולות I/O.

מבנה הנתונים המתחזק את ה-user thread הוא: **ULT** (user level thread), המכיל: id של התהליכון, ערך הרגיסטר UL, ומצביע ל-thread_stack.

בתוך ה-PCB ישנו מבנה נתונים בשם **LPS** (light process state) והוא שומר מצביעים ל-ULT ול-KLT של ה-threads ששייכים לתהליך, מסיכת הסיגנלים, פרמטרים ל-system calls ועוד.

Kernel Thread

מסופק ע"י **מערכת ההפעלה**, ומנוהל בה.

יתרונות: מסופק ע"י מעה"פ ומוכר לה. קריאות מערכות לא חוסמות את שאר התהליכונים, יש תמיד מקביליות.

חסרונות: איטיות ביצירת התהליכונים.

מבנה הנתונים המתחזק את ה-kernel thread הוא: **KLT** (kernel level thread), המכיל מצביע למחסנית, רישומי רגיסטרים ועוד.

בתוך ה-PCB ישנו מבנה נתונים בשם **HPS** (hard/heavy process state) והוא שומר את מיפוי הכתובות הווירטואליות של התהליך.

היתרונות בהפרדה בין ה-HPS ל-LPS הם:

תקורה - overhead, ביצועים - performance, גמישות - flexibility,

מדרגיות/ סילומיות - scalability

מכיוון שאם היינו שומרים את כל המידע ב-PCB ללא הפרדה למבני נתונים מסודרים- היה צורך לשמור את **כל** המידע **בכל** context switch ולבצע עדכון על כל שינוי שהתבצע.

אמצעים לסנכרון בין תהליכונים:

חשוב לדעת כי לעיתים קרובות נצטרך לסנכרן בין התהליכונים, העובדים על אותו תהליך – וממילא גם על אותם נתוני בסיס, קבצים פתוחים וכו'.

בעז"ה בהמשך, בפרק על מנגנוני מקבול וסנכרון נלמד על מנגנונים המיועדים לסנכרון בין תהליכונים – mutex ועוד.

שיקולי Performance בתהליכונים

נמדוד את הנכס (התהליכון) של המערכת, ואף נוכל להשתמש בתוצאות לחקר התנהגות המערכת.

דוגמאות לשיקולים נפוצים:

- זמן המתנה
- תפוקה כוללת
- שיעור ביצוע הבקשה
- זמן ריצה
- ניצול ושימוש במעבד
- יעילות הפלטפורמה
- ביצועים פר דולר (רמת הביצועים חלקי העלות הכספית של החומרה)
- ביצועים פר משקל החומרה

Multi – Threading VS Multi – Processing:

יתרונות:	+ מרחב כתובות אישי לכל תהליך + משאבים פרטיים לתהליך	+ מרחב הכתובות משותף + שמירה קלה על מצב משותף + context switch זול ומהיר
חסרונות:	- שימוש גבוה בזיכרון - Context switch יקר - קושי לשמור על מצב משותף	- יישום ומימוש מורכבים - נדרש סנכרון - מעה"פ חייבת לתמוך בריבוי תהליכונים

מודלים לריבוי תהליכונים

1. רבים ליחיד – Many To One
תהליכונים משתמש רבים ממופים לתהליכון גרעין יחיד.
2. יחיד ליחיד – One To One
תהליכון משתמש יחיד ממופה לתהליכון גרעין יחיד.
3. רבים לרבים – Many To Many
תהליכונים משתמש רבים ממופים לתהליכונים גרעין רבים.
במקרה שתהיה הצפה של תהליכונים משתמש המערכת לא תקרוס כי יש הגבלה על מספר התהליכונים שתהליכון גרעין יכול להתייחס אליהם.

טיפול בסיגנלים בתהליכונים

כאשר נשלח סיגנל לתהליך שבו מס' תהליכונים, מי מהם יקבל את הסיגנל?

1. התהליכון שהסיגנל רלוונטי עבורו
 2. כל התהליכונים שלא מסכו את הסיגנל הזה
 3. תהליכון ייעודי – שזהו חלק מתפקידו
- מעה"פ Linux מטפלת בסיגנל לפי סוגו:
1. סיגנל **סינכרוני** ישלח לתהליכון שביצע את הפעולה. אם הוא לא יטפל בסיגנל כראוי ייתכן והתהליך יסתיים.
 2. סיגנל **א-סינכרוני** ישלח לתהליכון כלשהו, בד"כ הראשי בתהליך.
 3. כל התהליכונים יטפלו בסיגנלים באותו אופן.
 4. ניתן למסך סיגנלים לכל תהליכון בנפרד, ע"י הפונ' `pthread_sigmask()`
- שימו לב: שליחת סיגנל לתהליכון משתמש עשויה להיות מאתגרת, מכיוון שמעה"פ מכירה רק תהליך אחד.

יצירת תהליכונים – Thread Pool

יצירת תהליכון היא פעולה זולה יחסית ליצירת תהליך, אך גם יצירת וחיסול תהליכון – הן פעולות בעלות תקורה כלשהי שתמיד נרצה להקטין.

הפתרון: Thread Pool – מאגר / בריכת תהליכונים:

הרעיון המרכזי הוא שבעת יצירת תהליך הוא ייצר N תהליכונים שיישמרו במאגר. בכל פניה לשרת – הוא יעיר תהליכון מהמאגר, וכשהתהליכון יסיים את פעולתו הוא יחזור לישון במאגר. טם אין תהליכון פנוי לטפל בבקשה – הבקשה תמתין.

לפי מה נקבע את N ?

1. באופן סטטי – כתלות במס' המעבדים, גודל הזיכרון, ומס' פניות אפשרי.
2. באופן דינאמי – לפי העומס במערכת

הנוסחה למציאת כמות threads מקסימלית לניצול המעבדים היא:

$$Y \text{ threads} = Y \text{ cores} / (1 - \text{block_coefficient})$$

$\text{block_coefficient} = \text{ערך בין } 0 \text{ ל-} 1 - \text{כמה אחוזים מהזמן ה-thread ישן בין פעולות I/O.}$

ביטול תהליכונים – Thread Cancellation

ביטול תהליכון [לפני שסיים את ריצתו] שקול להריגת תהליך. נקרא לו 'תהליכון מטרה' – target thread. כיצד נבצע את הביטול?

1. באופן א-סינכרוני: התהליכון יופסק מיידית, בעת קבלת הביטול.
2. באופן סינכרוני: 'ביטול דחוי' – התהליכון יתבטל רק כשיגיע למקום מתאים – 'נקודת ביטול' [נקודת ביטול יכולה להיות: מקום בו מבוצע System Call / פונ' טיפול כמו: pthread_join() / כשהתהליכון מבצע pthread_testcancel() ובודק האם ממתינה לו בקשת ביטול].

fork() ו-exec() בתהליכונים

?- מה יקרה כאשר תהליכון מבצע fork()?

1. ייווצר תהליך המכיל את אותם תהליכונים של אב התהליכון
 2. ייווצר תהליך שיריץ תהליכון אחד בלבד – התהליכון שביצע את ה-fork()
- במע"פ Linux הפעולה fork() תשכפל רק את התהליכון שביצע אותו, כלומר אופציה 2 מהנ"ל.

?- מה יקרה כאשר תהליכון מבצע exec()?

1. רק התהליכון הנוכחי יחליף את ה-PCB שלו, השאר ירוצו כרגיל
2. כל התהליך, על כל התהליכונים שבו יוחלפו לתוכנית חדשה.

מע"פ Linux אכן מבצעת את אופציה 2 מהנ"ל.

הספרייה <pthread.c>

זוהי ספרייה הקיימת בשפת C ומאפשרת לנו להתנהל עם תהליכונים, להגדיר ולבצע עליהם פעולות שונות ומגוונות, בהתאם להגדרות המתכנת.

[בשפות רבות יש ספריות דומות להתנהלות בתהליכונים, כמו ב-C#, JAVA, Python ועוד]

- מצביע לתהליכון – יהיה מסוג pthread_t. [לחילופין ניתן להשתמש גם ב-int] לדוגמה:

pthread_t t1;

- הפונקציה מייצרת את התהליכון, שולחת את המצביע לתהליכון, פונקציה לביצוע ופרמטרים עבורה. הפרמטר השני אמור להכיל דגלים המסמנים כיצד לייצר את התהליכון [משתנה מסוג pthread_attr_t]. בד"כ נעביר NULL. הפונקציה תחזיר 0 כשהצליחה, וערך השונה מ-0 בכישלון.

Int status = **pthread_create**(&t1, NULL, do_func, (void*) &args);

- כשנרצה לסיים את פעולת התהליכון:

pthread_exit(NULL);

- לקבלת ה-tid של התהליכון נשתמש ב- **pthread_self()**
- ע"מ לגרום לתהליך האב להמתין לכל בניו, נשתמש ב- **pthread_join(&t, NULL)**
- להשוואה בין תהליכונים נשתמש ב- **pthread_equal(t1, t2)** [כשהם שווים – מחזירה שונה מ-0]
- לסיום/ביטול תהליכון נשתמש ב- **pthread_cancel(tid)** איך התהליכון יטפל בפונקציה זו?
 1. יתעלם מהבקשה
 2. יסתיים באופן א-סינכרוני [מיידי]
 3. יסתיים באופן סינכרוני [כשיגיע לנקודת ביטול]

לפי מה נדע איך לטפל בפונקציה זו? לפי הפונ' הבאה:

- נגדיר באמצעות הפונ' **pthread_setcancelstate()** מה תהיה תגובת התהליכון לביטול. נשלח כפרמטר את המצב החדש, ואת המצב הקודם. [ב"מ: אפשר ביטול]
- כדי להגדיר האם הסיום יהיה סינכרוני או א-סינכרוני נשתמש ב- **pthread_setcanceltype()** ונשלח פרמטרים בהתאם. [ב"מ: סינכרוני]
- כדי לקבוע פונקציה שתבצע בעת סיום התהליכון נשתמש במחסנית clean_up המיועדת בדיוק לצורך זה, וקיימת בכל תהליכון. כלומר: כשנרצה להוסיף פונ' לביצוע נכתוב: **pthread_cleanup_push(func, args)** ובסיום התוכנית נשתמש ב- **pthread_cleanup_pop(0)**
- המתנה בין תהליכונים: כשנרצה שתהליכון X יחכה לתהליכון Y נשתמש ב- **pthread_join(tid, val)** שתחזיר 0 בהצלחה, ומספר שונה מ-0 בכישלון
- ניתוק תהליכון – שלא יהיה JOINABLE יתבצע ע"י **pthread_detach(t1)**
- אתחול עבור כל התהליכונים פעם אחת – ע"י **pthread_once()**
- שליחת סיגנל לתהליכון – ע"י **pthread_kill(tid, sig_num)**
- מיסוך סיגנלים לתהליכון – ע"י **pthread_sigmask()**

שיטות חלוקת עבודה בין מעבדים

1. **גישה סימטרית – SMP:**
מעה"פ אחת שולטת בכל המעבדים ומקצה להם תהליכים / תהליכונים לביצוע באופן דינאמי, ע"מ לאזן את העומס.
אתגרים: העברת התהליך מליבה לליבה מורכבת- נצטרך להעתיק את מצב האוגרים, הגדרות ה-MMU וזיכרון המטמון. כמו כן, מעה"פ צריכה לסנכרן בין דרישות המשאבים.
2. **שיטה א-סימטרית – AMP:**
נתייחס לכל ליבה בנפרד, כך שלכל ליבה יכולה להיות מעה"פ שונה ← כל ליבה תפעל במעה"פ המתאימה ביותר לתפקידה.
אתגרים: יש צורך לספק תקשורת בין הליבות השונות.
3. **גישת הביניים – BMP:**
מעה"פ אחת שולטת בכל הליבות. אין מעבר של תהליכונים בין ליבות כלל. כל תהליכון מתנהל בליבה בה הוא נוצר.
יתרונות: ניהול הזיכרון קל ← מהירות הביצועים משתפרת.

תבניות לעיצוב אלגוריתמים מקביליים

תלויים בעיקר בתכנון ועיצוב מקדים. הקשיים: סתירה בין חלוקת המשימות לריבוי המעבדים (נוצרת תקורה בתקשורת ובסנכרון), תלויות נתונים ומשימות, תכנות מקבילי אי דטרמיניסטי (לא נדע מראש את התוצאה)

1. Boss – Worker

- הבוס מקצה משימות, וכל עובד מבצע משימה שלמה.
 חשוב לשים לב: מכיוון שכל המערכת תלויה הבוס $\leftarrow$ הבוס חייב להיות יעיל!
יתרונות: העובדים לא זקוקים לסנכרון (משימה אחת לא תתבצע פעמיים).
חסרונות: הבוס יעבוד קשה, עלול להיווצר חוסר התאמה בין העובד למשימה.
 פתרונות לחסרונות:
 - חלוקת עובדים לקבוצות עבודה לפי התמחות
 - הגדלת מספר התהליכים בשיטת thread-pool (הגדלה במספר קבוע או בהכפלה)

2. Pipeline: שיטת 'הסרט הנע'

- כל תהליכון אחראי למשימת משנה במערכת, כך שכולם יחד יוצרים את המשימה השלמה.
 כולם עובדים במקביל. התפוקה תלויה בעובד החלש ביותר.
יתרון: לכל תהליכון יש עבודה קבועה $\leftarrow$ יעילות בביצועים.
חסרון: סנכרון ואיזון העומס בין העובדים.

דוגמה לחישוב זמן העבודה:

מספר thread	מוצר מספר
1	1 2 3 4 5 6 7 8 9 10 11
2	1 2 3 4 5 6 7 8 9 10 11
3	1 2 3 4 5 6 7 8 9 10 11
4	1 2 3 4 5 6 7 8 9 10 11
5	1 2 3 4 5 6 7 8 9 10 11
6	1 2 3 4 5 6 7 8 9 10 11
	100 220
	0 360
	N works J levels K threads T time to level
	$(N-1) * T + (J * T)$ $11-1 * 20 + 6 * 20 = 320$
	$10 - 1 * 20 + 6 * 20 = 300$

3. **Layered Pattern:** מודל השכבות

כל שכבה מורכבת מסוגים שונים, ומתתי משימות נפרדות. כל משימה עוברת את כל השכבות
יתרונות: לוקאליות, משימה מוגדרת לכל תהליכון, תקורה נמוכה, גרנולריות, חלוקה גסה.
חסרונות: סנכרון גדול, לא מתאים לכל תוכנית.

4. **Event Driven Model:** מודל מונחה אירועים.

מאפשר מקביליות ע"י:
- תהליך אחד המכיל thread בודד, שישמש כ"מוקדן אירועים", ויזמן handler מתאים לכל בקשה
- מסתמך על I/O מרובים. כשנוצר מצב ש: פעולת I/O < (context switch * 2)
יתרונות: מרחב הכתובות משותף, דרישת הזיכרון קטנה יחסית, אין context switch, אין סנכרון.
חסרון: בלוק של request / handler חוסמים את התהליך.
פתרון לחסרון: שימוש ב- I/O אסינכרוני, וב-helpers למי שלא יכול להיות א-סינכרוני.

מקבול וסנכרון

תהליכים ותהליכונים זקוקים לשיתוף פעולה ביניהם- בגישה לנתונים, קוד ומידע.

הם יכולים לתקשר ביניהם ב-channel (ערוץ) או ב-shared memory (מרחב זיכרון משותף), אך אז ייווצרו בעיות של סנכרון.

דוגמת הבנק: משיכת כסף מהחשבון, בשני כספומטים שונים. תהליכי המשיכה- מתבצעים בנפרד, ולכל אחד שמורים נתונים בנפרד. בחטיפת המעבד- המידע לא מתעדכן ונוצרת שגיאה חמורה- יהיה אפשרות למשוך יותר ממה שיש באמת בחשבון...

פונקציית המשיכה מחשבון הבנק נראית כך:

```
float CashWithdrawal (char* account_number, float sum){  
    float balance = GetBalance(account_number);  
    balance = balance - sum;  
    UpdateBalance(account_number, balance);  
    return balance;  
}
```

דוגמת הרשימה: כשנרצה להוסיף לרשימה מקושרת איברים נוספים, בשני תהליכונים שונים. בחטיפת המעבד- בין 2 השורות האחרונות (הקוד המלא בספר)- יתווסף רק איבר אחד לרשימה ולא שניים...

בהמשך נציג בעיות סנכרון נפוצות נוספות.

מושגים:

race condition – מרוץ על תנאי: זהו מצב בו נכונות החישוב נפגמת כתוצאה מתזמון אקראי של תהליכונים.

data race – מרוץ על מידע: זהו מצב בו התהליכונים משתמשים באותם משתנים ומעדכנים אותם, אך ללא סנכרון.

Critical Section – קטע קריטי: זהו קטע קוד שמתעסק בדבר מה המוגדר קריטי, כלומר: אם יהיו שני תהליכים או יותר שיבצעו את הקטע הזה במקביל- נוכל להגיע למצבי data race או race condition. בזמן בו תהליך מבצע קטע קריטי – הוא 'בלבדי' בביצוע קטע קריטי וחייב להתבצע באופן אטומי.

שאר התהליכים עסוקים בקטע קוד שיורי שאינו קריטי או ממתנים להיכנס לקטע הקריטי.

הפתרון:

שימוש במנגנוני סנכרון ונעילה, על קוד או על משאב או על אובייקט. תמיד נעדיף לנעול אובייקט/ נתון/ משאב שהוא המשמעותי ביותר בנעילה, ולא סתם לנעול קוד.

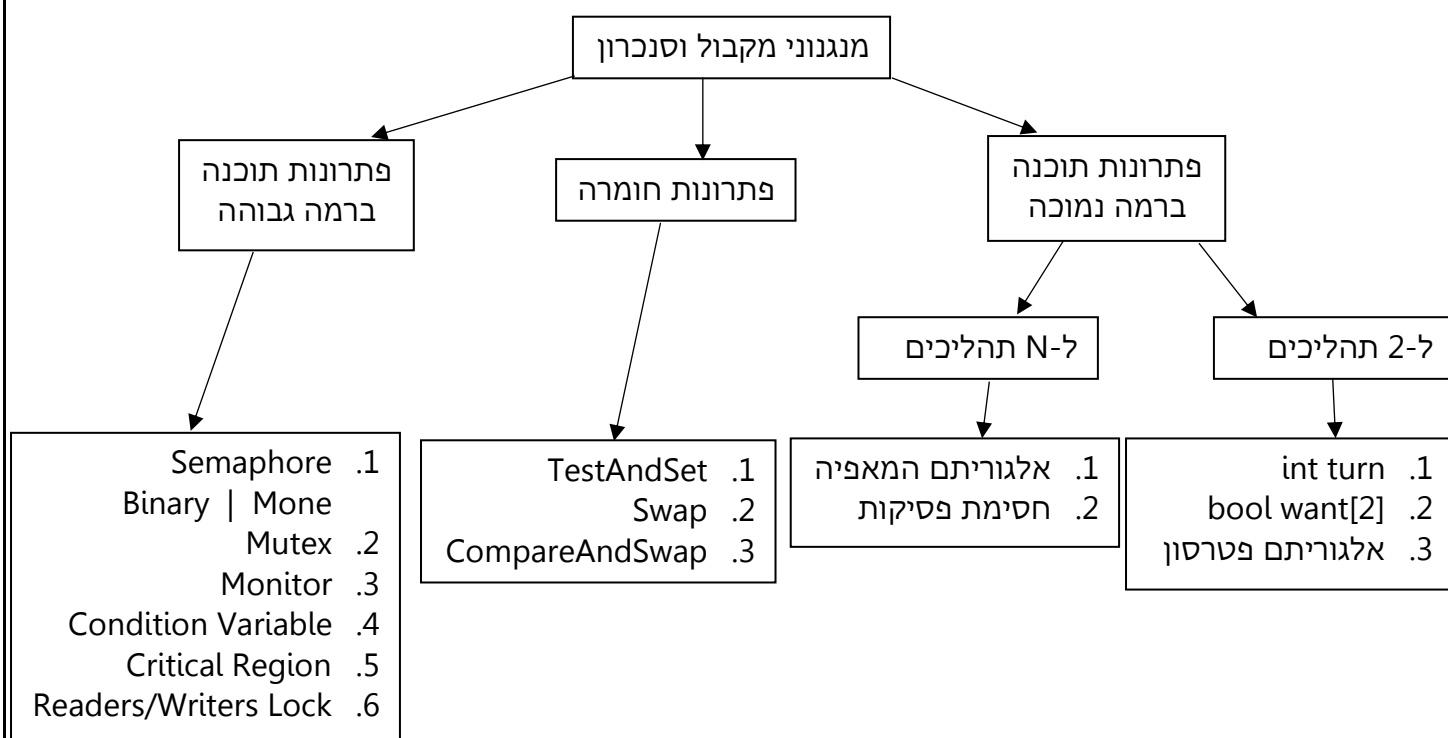
דרישות ממנגנוני סנכרון:

1. Mutually Exclusiveness – מניעה הדדית. חובה. רק תהליך אחד מבצע את הקטע הקריטי שלו בכל נקודת זמן.
2. Progress – התקדמות = אין קיפאון. חובה. תמיד תהליך כלשהו ייכנס לקטע הקריטי שלו, מתוך m תהליכים שממתינים.
3. Fairness – הגיונות, המתנה חסומה = אי הרעבה. לא חובה. נדאג שכל אחד מהתהליכים יזכה להיכנס לקטע הקריטי שלו, ולא ייזנח בין הממתינים. ויש חסם על מס' הפעמים

כל פרוטוקול שנמשש בו פתרון לקטע קריטי יכול:

- א. קטע כניסה – פעולות אותן חייב לבצע תהליך המעוניין להיכנס לקטע הקריטי שלו.
- ב. קטע יציאה – פעולות אותן חייב לבצע תהליך שסיים את הקטע הקריטי שלו.

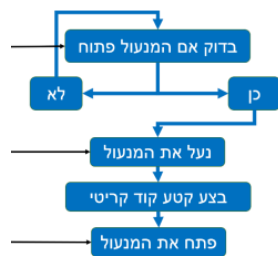
המנגנונים הם: [שרטט]



פתרונות תוכנה ברמה נמוכה

פתרונות אלו יממשו פרוטוקולים להשגת בלבדיות, ומבוססים על לולאות busy wait.

הפתרון הבסיסי והראשוני הוא משתנה lock- מנעול. משתנה בוליאני. 0 – המנעול סגור, 1 – המנעול פתוח. נגדיר פונקציה לבדיקת מצב ה-lock, ופונקציה לשינוי מצב ה-lock.



```
while(lock == 0);  
lock(lock);  
Critical_Section();  
unlock(lock);
```

המנעול יעבוד על לולאות המתנה – busy wait.

אך נראה כי המשתנה lock יצטרך סנכרון נוסף גם עבורו. לכן נתקדם לפתרונות נוספים, ונחלק אותם לשני סוגים: פתרונות לסנכרון בין 2 תהליכים, ופתרונות לסנכרון תהליכים מרובים.

הנחה בעייתית: פעולת מכונה (כולל השמה) מתבצעת באטומיות. [במחשבים מודרניים השמה יכולה להימשך מספר מחזורי זיכרון].

פתרונות עבור 2 תהליכים בלבד:

א. נגדיר את המשתנה הבוליאני turn, שיגדיר את תור הכניסה לקטע הקריטי.

```
int turn = 0;
```

קטע הקוד ייראה כך:

```
while(turn != process_id);  
Critical_Section();  
turn = 1 - process_id;  
// -- code --
```

כשבחנן את קטע הקוד בהרצת שני תהליכים, ובחנן האם המנגנון עומד בדרישות, נשים לב שהדרישה למניעה הדדית מושגת, וכן הדרישה להגינות.

אך ההתקדמות בעייתית מכיוון שהקוד מחייב את כניסת התהליכים לקטע הקריטי בתורנות בלבד. אם יקרה מצב שתהליך 1 ירצה להיכנס פעמיים ברציפות, בעוד התהליך המקביל אליו כלל לא רוצה להיכנס – תהליך 1 יהיה חייב להמתין עד שהשני ייכנס, אחרת- ההתקדמות תיעצר לגמרי.

ב. נגדיר את המערך הבוליאני want, בו נסמן true עבור תהליך המבקש כניסה לקטע הקריטי.

```
bool want[2] = {false, false};
```

קטע הקוד יראה כך:

```
want[process_id] = true;  
while(want[1 - process_id] == true);  
Critical_Section();  
want[process_id] = false;  
// -- code --
```

כשנבחן את קטע הקוד בהרצת שני תהליכים, ונבחן האם המנגנון עומד בדרישות, נשים לב שהדרישה למניעה הדדית מושגת, וכן הדרישה להגינות. אך ההתקדמות בעייתית מכיוון שיכול להיות מצב בו תהליך 1 סימן את עצמו כ-true במערך, ומיד המעבד נחטף עבור התהליך השני. הוא גם כן יסמן את עצמו כ-true, אך לולאת ההמתנה תייצר מצב של קיפאון – deadlock. כלומר: כל אחד מהתהליכים ממתיק שהתהליך השני יסמן את עצמו כ-true. אך זה לא יוכל לקרות בלי שהתהליך יבצע את הקטע הקריטי. ומצד שני – אף אחד מהם לא יצליח להיכנס לקטע הקריטי בגלל התנאי שבלולאת ההמתנה.

ג. האלגוריתם של פטרסון: נגדיר את המערך want ואת המשתנה turn, לולאת ההמתנה תתחשב בשניהם.

```
bool want[2] = {false, false};
```

```
int turn = 0;
```

קטע הקוד יראה כך:

```
want[process_id] = true;
turn = 1 - process_id;
while(want[ 1 - process_id ] == true && turn == (1 - process_id) );
Critical_Section();
want[process_id] = false;
// -- code --
```

כשנבחן את קטע הקוד בהרצת שני תהליכים, ונבחן האם המנגנון עומד בדרישות, נשים לב שהדרישה למניעה הדדית מושגת, הדרישה להתקדמות מושגת וכן הדרישה להגינות.

פתרונות עבור תהליכים מרובים:

א. **אלגוריתם המאפייה** – כשם שבמאפיה או בבית המרקחת ניקח מספר שיציין את מיקומנו בתור, ננסה לממש אלגוריתם שיחלק מספרים לתהליכים העומדים בתור להיכנס לקטע הקריטי. באלגוריתם המאפייה, לכל תהליך מוקצה מספר (כרטיס) לפי סדר לקסיקוגרפי. לפני הכניסה לקטע הקריטי, תהליך מקבל מספר כרטיס, והתהליך עם מספר הכרטיס הקטן ביותר נכנס לקטע הקריטי. אם שני תהליכים מקבלים את אותו מספר כרטיס, התהליך עם מזהה התהליך הקטן [הכוונה לאינדקס שלו במערך] מקבל עדיפות. נגדיר מערך number שישמור לכל תהליך את המספר שקיבל בתור. [אינדקס המערך מסמן את ה-id של התהליך]. המערך מאותחל ב-0.

```
int number[N];
```

קטע הקוד יראה כך:

```
number[process_id] = MAX(number) + 1;
for( int i = 0; i < N && i != process_id; i++){
    while( WAIT(number[i] == 0) || WAIT(number[i] > number[process_id]) );
}
Critical_Section();
number[process_id] = 0;
// -- code --
```

הערה: יש כאן 2 פונקציות אבסטרקטיות:

- $MAX(arr)$ – פונקציה המקבלת מערך מספרים ומחזירה את המספר הגבוה ביותר שיש בו.
- $WAIT(condition)$ – פונקציה שמתפקדת כלולאת המתנה לתנאי מסוים.

הלולאה החיצונית סורקת את מערך המספרים, והלולאה הפנימית ממתינה לאחד מהתנאים הבאים: או שהתהליך האחר לא ממתין להיכנס לתור הקטע הקריטי, או שנמתין שהוא יקבל מספר משלו.

אך נשים לב שנצטרך סנכרון על המערך `number` בעצמו, מכיוון שיש כמה תהליכים שקוראים אותו בו זמנית, וכן משנים בו ערכים! לכן יכול להיות שתנאי שהיה תקף בזמן הסבב הקודם לא תקף בזמן הסבב הנוכחי, וממילא יהיו 2 תהליכים או יותר שיקבלו את אותו המספר בתור $\leftarrow$ קיפאון `deadlock`.

ננסה לתקן את הקוד – כך שיוודא שכל התהליכים לפני קיבלו מספר:

```
number[process_id] = MAX(number) + 1;
for( int i = 0; i < N && i != process_id; i++){
    while( WAIT(number[i] == 0) ||
           ( WAIT(number[i] > number[process_id]) && i > process_id) );
}
Critical_Section();
number[process_id] = 0;
// -- code --
```

אך גם לאחר תיקון הקוד נראה כי הוא אינו עומד בדרישות, ותיווצר בעיה במניעה ההדדית. כאשר תהליך קורא את תוכן המערך – תהליך אחר יכול לגשת ולשנות את הערכים ששאר התהליכים מתבססים עליהם...

שוב ננסה לתקן את הקוד כך שלא יהיו תקלות.

נגדיר מערך נוסף בשם `choosing` מסמן האם התהליך כרגע בוחר מספר בתור. המערך מאותחל ב-`false`.

```
bool choosing[N];
```

קטע הקוד יראה כך:

```
choosing[process_id] = true;
number[process_id] = MAX(number) + 1;
choosing[process_id] = false;
for( int i = 0; i < N && i != process_id; i++){
    while( WAIT(choosing[i] == true) );
    while( WAIT(number[i] != 0) ||
           ( WAIT(number[i] < number[process_id]) && i < process_id) );
}
Critical_Section();
number[process_id] = 0;
// -- code --
```

האלגוריתם המתוקן אכן משיג את כל הדרישות הנצרכות.

החסרונות באלגוריתם זה: נצטרך פעולות ומשתנים רבים, ולולאות ההמתנה מבזבזות את ביצועי המעבד.

ב. **חסימת פסיקות** – החסימה מונעת החלפת תהליכים – נחסום את פסיקת ה-context switch. הפונקציה לחסימת הפסיקות:

```
Lock_Aquire(lock) {  
    DisableInterrupts();  
    while( lock != FREE) {  
        EnableInterrupts();  
        DisableInterrupts();  
    }  
    lock = BUSY;  
    EnableInterrupts();  
}
```

הפונקציה לביטול חסימת הפסיקות:

```
Lock_Realise(lock) {  
    lock = FREE;  
}
```

-פירוט נוסף בספר בעמ' 442,443 -

- במעבד יחיד:
התוכנית תתרוסק בגלל החסימה, נאבד פסיקות חשובות, ועיכוב ה-I/O שמעכב ביצועים.
- במעבדים מרובים: החסימה לא פותרת כלום, כי כל תהליך רץ במעבד אחר...

החסרונות בפתרונות התוכנה ברמה הנמוכה:

מורכבים, קשים להבנה, מטעים בפתרון, משתמשים בלולאות המתנה המכונות גם spin lock, מבזבזות CPU, וכן היפוך עדיפות עלול ליצור deadlock. לא מתאימים לארכיטקטורות מודרניות.

פתרונות חומרה

פתרונות החומרה ישתמשו בפקודות חומרה אטומיות [כלומר: מתבצעות בשלמות ללא context switch תוך כדי ביצוע הפעולה], אך אינן מבטיחות התקדמות והוגנות.

תיאום פעולות אטומיות בין מס' מעבדים ממומש ע"י Cache Coherence Protocol.

לעיתים פעולות אטומיות מהוות תחליף יעיל יותר לשימוש במנעולים.

א. **פקודת Test And Set** – הפקודה הזו תמיד תנעל את המשאב, אך תחזיר את הערך שהיה עד כה במנעול. (יש כאלו המכנים פקודה זו בשם TSL)
נגדיר את המשתנה lock ונאתחל אותו:

```
bool lock = false;
```

הפונקציה נראית כך:

```
bool TestAndSet( bool* x) {  
    bool old_x = x;  
    x = true;  
    return old_x;  
}
```

הקוד המנהל את הכניסה לקטע הקריטי ייראה כך:

```
while(true) {  
    while(TestAndSet(lock) == true);  
    Critical_Section();  
    lock = false;  
    // -- code -  
}
```

היתרון: פשוט ומהיר, נתמך בחומרה. נשיג את דרישת המניעה ההדדית, וההתקדמות, אך ללא הגינות.

החסרונות: לולאת spin lock, וגישה לזיכרון תוך כדי ע"מ לעדכן את ערך המנעול. (נפתור זאת- ע"י שרק נבדוק את ערך המנעול, ונשנה רק לאחר שהמנעול ישתחרר)

ב. **Swap** – נמתין בלולאת spin lock עד שנקבל את המנעול true בעזרת הפקודה swap.
נגדיר את המשתנים:

```
bool key = true, lock = false;
```

הפונקציה נראית כך:

```
Swap( bool* v1, bool* v2) {  
    bool temp = v1;  
    v1 = v2;  
    v2 = temp;  
}
```

הקוד המנהל את הכניסה לקטע הקריטי ייראה כך:

```
while(true) {  
    key = true;  
    while(key == true) {  
        Swap(key, lock);  
        Critical_Section();  
    }
```



```
lock = false;
}
```

ג. **Compare And Swap** – השוואה בין התהליכים, החלפה ביניהם, והחזרת ערך בוליאני. הפונקציה נראית כך:

```
CompareAndSwap(bool* mem, bool* v1, bool* v2) {
    if(mem == v1) {
        mem = v2;
        return false;
    }
    return true;
}
```

היתרונות בפתרונות החומרה: פשוטים, פותרים 'out of order', תומכים בתהליכים מרובים, תומכים בריבוי גרעינים למעבד יחיד.

החסרונות בפתרונות החומרה: מבוססים על spin lock, יוצרים הרעבה/ פתרון קשה, אינם ניידיים, לא תומכים בריבוי מעבדים.

הפתרון לכל הנ"ל: ניצור אבסטרקציה גבוהה אך קלה למימוש, תממש ממשק אחיד (פותר את הניידות), תתמוך בצורכי הסנכרון, תמנע מ-busy wait ובעיות הנגרמות משינוי העדיפות.

פתרונות תוכנה ברמה גבוהה

פתרונות אלו נתמכים במערכת ההפעלה, הקומפיילר, והחומרה.

semaphore – הגדרה:

זהו מבנה המכיל משתנה מסוג int שאינו שלילי ותור לתהליכים ממתינים, ומשותף למספר תהליכים. הוא משיג מניעה הדדית ומניעת race condition ויישום סנכרון בין תהליכים.

המנגנון בו עובד על איתותים (פניה לתהליך שביקש את המשאב "הנה סיימתי, קח בבקשה" - שיטת תקשורת המבוססת על שימוש ב-Flags ע"מ לאותת לתהליכים – למי מותר להיכנס כעת לביצוע הקטע הקריטי).

הסמאפור משתמש במנגנון איתות, וב-2 פעולות אטומיות: (הפעולות ממומשות לפי סוג הסמאפור).

הסמאפור עבר מס' גרסאות עד לגרסה זו- הטובה ביותר. [שאר הגרסאות בסוף החוברת]

1. wait() – המתנה לאישור תפיסת הסמאפור.

2. signal() – שחרור הסמאפור.

נגדיר את מבנה הסמאפור:

```
struct semaphore {  
    int value;  
    Queue<Process> waiting_processes;  
};
```

קוד הכניסה לקטע הקריטי בכל תהליך יראה כך:

```
semaphore s;  
while(true){  
    wait(s);  
    Critical_Section();  
    signal(s);  
    // --code--  
}
```

ההמתנה שערכו של הסמאפור יהיה חיובי – עוצרת את ביצוע התהליך ע"י לולאת ההמתנה שבפעולה wait(s).

ובעת שנסיים את ביצוע הקטע הקריטי – נגדיל את ערכו של הסמאפור ב-1, ע"י הפעולה signal(s), כלומר: נשחרר את האחיזה של התהליך בסמאפור.

כך השגנו מניעה הדדית והתקדמות, ללא הוגנות.

היתרונות ב-semaphore:

+ threads רבים יכולים לגשת לקטע הקריטי בו זמנית – כשה-threads הללו שייכים לאותו תהליך [הרי סמאפורים מסנכרנים בין תהליכים]

- + סמאפורים אינם תלויים במכונה עליה פועלים
- + ניהול המשאבים גמיש

החסרונות ב-semaphore:

- משתמש בהיפוך עדיפויות לפי הצורך
- נצטרך לממש את הפעולות האטומיות באופן יעיל ע"מ להימנע מ-deadlock
- מהחיסרון הקודם ← לא נוכל להשתמש ב-semaphore במערכות בקנה מידה גדול [חוסר מודולריות]
- ה-semaphore נוטה לשגיאות תכנות: נגיע ל-deadlock או לאי-מניעה הדדית
- מעה"פ חייבת לעקוב על ביצוע הפעולות האטומיות הנ"ל

סוגי סמאפורים:

1. **Semaphore Mone** – מימוש זה מאפשר גישה למשאב בעל n עותקים. (כמו תחנת דלק עם 4 משאבות), נשתמש בו כשיש דרישה למקביליות, וכן כשיש דרישה לחוסר מודולריות, הגמישות היא בשחרור הסמאפור. כך שמי שמשחרר את הסמאפור הוא לא בהכרח מי שנעל.

ביצע נגן על הכניסה לקטע הקריטי?

`wait()` – פעולה זו מקטינה את ערך ה-`value` ב-1, ממתינים עד שהערך אינו שלילי. כאשר הסמאפור שווה ל-0 התהליך שניסה לתפוס מועבר למצב חסום.
(מכונה גם `down`, `P`, `proben`)

```
wait(semaphore s){
    s.value - -;
    if(s.value < 0) {
        s.waiting_processes.Enqueue(Current_Process);
        block();
    }
    return;
}
```

`signal()` – מגדילה את ערך ה-`value` ב-1, ואם יש צורך – משחררת את אחד הממתינים. התהליך מועבר לתור ה-`ready`.
(מכונה גם `up`, `post`, `VP`, `vehogen`, `V`)

```
signal(semaphore s){
    s.value + +;
    if(s.value <= 0) {
        wakeup( s.waiting_processes.Pop());
        return;
    }
}
```

2. **Binary Semaphore** – זהו סמאפור כמו שהכרנו עד כה, אך בשינוי משמעותי: הוא יכול לקבל את הערכים '1' או '0' בלבד, ולא כמו סמאפור מונה – שיכול לקבל כל מספר שלם. הפעולות האטומיות שבו ייראו כך:

פעולת ה-wait תוגדר כך:

```
wait(semaphore s){
    s.value - -;
    if( s.value == 1 )
        s.value = 0;
    else {
        s.waiting_processes.Enqueue(Current_Process);
        sleep();
    }
}
```

וכמובן גם פעולת ה-signal בהתאמה:

```
signal(semaphore s){
    if( s.waiting_processes.IsEmpty() != true)
        s.value = 1;
    else
        wakeup(s.waiting_processes.Pop());
}
```

כמו כן נצטרך לעיתים להתמודד במצב בו יש לרשותנו רק סמאפורים בינאריים.

נוכל לייצר סמאפור ע"י ה-system call הבאה: **semget()** כיצד?

1. נייצר key שישמש את התהליכים שירצו להשתמש בסמאפור
2. נקצה סמאפור: `int sem_id = semget(key, num_sem, IPC_CREATE | 0600);`
3. כשבצטרף לסמאפור קיים – נשלח 0 בפרמטר השני.
4. נקצה את הערך התחילי של הסמאפור:
5. נאתחל את הסמאפור: `int status = semctl(sem_id, 0, SETVAL, sem_value);`
6. קוד הבניסה יהיה בהתאם...

נוכל למחוק סמאפור דרך ה-shell באמצעות הפקודה: **ipcrm -s <msg_id>**

הפונקציות ב-semaphore מהספרייה <semaphore.h>:

1. `sem_open(const char *, int, ...)` – פתיחת סמאפור.
2. `sem_init(sem_t * s, int, unsigned int)` – אתחול סמאפור.
3. `sem_trywait(sem_t * s)` – ניסיון לתפיסת סמאפור.
4. `sem_wait(sem_t * s)` – תפיסת סמאפור.
5. `sem_post(sem_t * s)` – שחרור סמאפור.
6. `sem_getvalue(sem_t *, int * s)` – קבלת ערך הסמאפור.
7. `sem_close(sem_t * s)` – סגירת סמאפור.
8. `sem_destroy(sem_t * s)` – מחיקת סמאפור.
9. `sem_unlink(const char *)` – ניסיון למחיקת הסמאפור (אם הוא כעת שייך לתהליך אין השפעה ע"כ).

☑ **תרגול:** כתבי תוכנית המייצרת תהליך בן. וכן הקצי משתנה `number` שיהיה משותף לשניהם! התוכנית תסנכרן ביניהם באמצעות סמאפור בינארי, כך שכל תהליך מוסיף 20 לערך המשתנה המשותף `number`. הדפיסי את הערך הסופי.

פירוט גרסאות הסמאפור באופן תחילי:

(1) **Mone Semaphore** – סמאפור מונה הוא משתנה שלם [int] שנאתחל אותו לערך מסוים. וכן מוגדרות עליו הפעולות האטומיות הבאות:

א. **wait** – פעולה זו מקטינה את ערך ה-value ב-1, ממתינים עד שהערך אינו שלילי. (מכונה גם down, P, proben) התהליך מועבר למצב חסום.

```
wait(semaphore){  
    while(semaphore <= 0);  
    semaphore - -;  
}
```

ב. **signal** – מגדילה את ערך ה-value ב-1, ומשחררת את אחד הממתינים. (מכונה גם up, post, VP, vehogen, V) התהליך מועבר לתור ה-ready.

```
signal(semaphore){  
    semaphore + +;  
}
```

ביצע נגן על הכניסה לקטע הקריטי?

נגדיר משתנה שיוכר לכל התהליכים, מסוג Semaphore [int] בשם mutex, ונאתחל אותו בערך 1.

קוד הכניסה לקטע הקריטי בכל תהליך ייראה כך:

```
while(true){  
    wait(mutex);  
    Critical_Section();  
    signal(mutex);  
    // --code--  
}
```

ההמתנה שערכו של ה-mutex יהיה חיובי – עוצרת את ביצוע התהליך ע"י לולאת ההמתנה שבפעולה wait(mutex).

ובעת שנסיים את ביצוע הקטע הקריטי – נגדיל את ערכו של הסמאפור mutex ב-1, ע"י הפעולה signal(mutex), כלומר: נשחרר את האחיזה של התהליך בסמאפור. כך השגנו מניעה הדדית והתקדמות, ללא הוגנות.

(2) כעת נגדיר **סמאפור 'ישנוני'**: שילך לישון במקום להיות ב-busy wait. נגדיר מבנה המכיל value ורשימה של ממתינים. כך:

```
struct sleepy_semaphore{  
    int value;  
    struct process* waiting_to_semaphore;  
}
```

פעולת ה-wait תוגדר כך:

```
void wait(sleepy_semaphore){  
    sleepy_semaphore.value - -;  
    if(sleepy_semaphore.value < 0){  
        // add yourself to: sleepy_semaphore->  
        waiting_to_semaphore
```

```

    sleep(5);
}

```

וכמובן גם פעולת ה-signal בהתאמה:

```

void signal(sleepy_semaphore){
    sleepy_semaphore.value + +;
    if(sleepy_semaphore.value <= 0){
        // remove yourself from: sleepy_semaphore->
        waiting_to_semaphore
        // wake-up
    }
}

```

נדגיש שפעולות אלו חייבות להתבצע באופן אטומי – הרי הן משנות ערכים ו/או בודקות את ערכם. ואם גם יעמוד לרשותנו spinlock – נשתמש בו לשמירת האטומיות.

ואם כך – מה עשינו בזה? מה שיפרנו???

הרי אם כבר משתמשים ב-spinlock למה שנשתמש דווקא ב-sleepy_semaphore??

נשים לב:

א. פתרונות סמאפור אלו – משתמשים בלולאות busy wait ← לא יעיל, בזבז זמן המעבד...

ב. במערכת מרובת מעבדים – לעיתים נעדיף לבצע לולאת busy wait במקום לבצע פעמיים context switch. ואילו במערכת עם מעבד יחיד (או כאשר הקטע הקריטי ארוך) נעדיף 'להרדים' את התהליך עד שיגיע זמנו להיכנס לקטע הקריטי, ואז – נעיר אותו.

שימוש בסמאפור מונה באמצעות סמאפורים בינאריים:

כמו כן נצטרך לעיתים להתמודד במצב בו יש לרשותנו רק סמאפורים בינאריים.

כעת, ננסה לממש סמאפור מונה באמצעות משתנה שלם – שייצג את מס' העותקים מהמשאב, ו-2 סמאפורים בינאריים. נגדיר את המשתנים כך:

```

binary_sem sem_for_counter = 1;
binary_sem sem_for_cs = 0;
int counter = N;

```

פעולות ה-wait (המתנה) וה-signal (שחרור הסמאפור) ייראו כך:

```

wait(){
    wait(sem_for_counter);
    counter - -;
    if(counter < 0) {
        // זהו מצב בו לא נוכל להיכנס לקטע הקריטי
        signal(sem_for_counter);
        wait(sem_for_cs);
    }
}

```

```

    }
    signal(sem_for_counter);
}

```

אם הקטע הקריטי לא היה פנוי עבורנו (וביצענו המתנה) נבצע signal ו-wait לסמאפורים.
למה??

פעולת השחרור תראה כך:

```

signal(){
    wait(sem_for_counter);
    counter + +;
    if(counter <= 0) {
        signal(sem_for_cs);
    }
    else{
        signal(sem_for_counter);
    }
}

```


mutex – הגדרה:

זהו סוג ספציפי של סמאפור בינארי המספק לנו מנגנון לנעילת משאב עבור תהליכון מסוים. ובו התהליכון שנעל גם ישחרר את ה-mutex ויבצע את הקטע הקריטי בבלעדיות.

שמו מעיד על ייעודו – קיצור של Mutually Exclusion (מניעה הדדית).

ה-mutex מיועד לסנכרון בין תהליכונים השייכים לאותו תהליך. ברגע שה-mutex ברשותו של תהליכון A אף אחד מהתהליכונים השייכים לתהליך זה יוכל להשיג את ה-mutex. אם ה-mutex מוגדר כרקורסיבי אז במצב בו אותו תהליכון ינסה "לנעול" שוב את ה-mutex – הוא יצליח לעשות זאת, ויצטרך גם לשחרר כמספר הפעמים שנעל.

אם תהליכון מתהליך X מנסה להשיג את ה-mutex באמצעות הפונקציה `pthread_mutex_lock()` כאשר ה-mutex ברשותו של תהליך Y – כל התהליכונים של תהליך X ייחסמו. אך אם תהליך X ישתמש בפונקציה `pthread_mutex_trylock()` לא תבוצע חסימה לתהליכון כלל.

אם נשתמש בו לסנכרון בין תהליכים הוא יתנהג כמו סמאפור בינארי

ה-mutex משתמש במנגנון עדיפות כזה: לתהליכים בעדיפות גבוהה יינתן מינימום זמן המתנה ככל האפשר.

הדגשה: mutex וסמאפור בינארי שונים במטרתיהם

הפונקציות ב-mutex מהספרייה pthread:

1. `int pthread_mutex_init( params )` – פונקציה לאתחול ה-mutex.

פרמטרים:

- `pthread_mutex_t* restrict mutex` – מצביע ל-mutex
- `const pthread_mutexattr_t* restrict attr` – מצביע למבנה המגדיר את תכונות ה-mutex. אם נשלח NULL נקבל את ברירת המחדל, mutex שאינו רקורסיבי.

ערך מוחזר:

בהצלחת האתחול יוחזר 0, בכישלון יוחזר -1.

2. `int pthread_mutex_lock( param )` – פונקציה לנעילת mutex

אם ה-mutex כבר נעול ע"י תהליכון אחר – אנו נמתין עד שה-mutex ישתחרר. [תזכורת: אם ה-mutex רקורסיבי ותהליכון 1 נעל אותו מספר פעמים – הוא חייב גם לשחרר אותו כמספר הפעמים שנעל!]

3. `int pthread_mutex_unlock( param )` – פונקציה לשחרור mutex

הפעולה תשחרר את המנעול ותעיר את אחד הממתינים, באין ממתינים – אין כרגע בעלים ל-mutex. אם ה-mutex מוגדר כרקורסיבי נעדכן את מספר השחרורים ורק בשחרור האחרון נוכל להעיר ממתיין.

4. `int pthread_mutex_destroy( param )` – פונקציה למחיקת mutex

בפונקציות 2,3,4 : **הפרמטר הוא:** mutex pthread_mutex_t* – מצביע ל-mutex

והערך המוחזר יהיה: בהצלחת הפעולה יוחזר 0, בכישלון יוחזר 1-.

היתרונות ב-mutex:

- + לא ייווצרו מצבי race condition מכיוון שרק תהליך אחד מבצע את הקטע הקריטי בכל פעם.
- + זהו מנגנון פשוט שתהליך יכול לנסות לתפוס לפני כניסתו ביצוע קטע קריטי
- + הנתונים תמיד עקביים = שלמות נתונים

החסרונות ב-mutex:

- במצב בו התהליכון נכנס לקטע הקריטי, ומועבר למצב sleep או שנחטף ממנו המעבד – אין שום תהליך שיוכל להיכנס לקטע הקריטי, ואנו עלולים להגיע להרעבה.
- כשתהליכון מסיים את הקטע הקריטי הוא יאפשר לתהליכונים אחרים לנסות לתפוס את ה-mutex. אין מנגנון אחר לנעילת או שחרור ה-mutex.
- לעיתים מימוש mutex עלול להוביל אותנו לבזבז וללא לניצול ה-CPU

mutex VS semaphore

mutex	semaphore	
אובייקט	משתנה int	טיפוס:
מנגנון נעילה	מנגנון איתות	מנגנון:
lock() unlock()	wait() signal()	פעולות להפעלה עליו:
צורת מימוש יחידה	• סמאפור בינארי • סמאפור מונה	צורות מימוש:
ניתן לשנות mutex רק ע"י התהליך שמבקש / משחרר את המשאב	אין אכיפה בין התהליך שנעל – שדווקא הוא ישחרר	אכיפת נעילה:
כשה-mutex נעול נלך להמתין בתור עד שהוא ישתחרר	כשה-semaphore תפוס התהליך יבצע המתנה עד שהוא ישתחרר	מצבי המתנה:

☑ **תרגול:** כתבי תוכנית המייצרת 3 תהליכונים, וכן הקצי משתנה number שיהיה משותף לכולם. התוכנית תסנכרן ביניהם באמצעות mutex, כך שכל תהליכון מוסיף 20 לערך המשתנה המשותף number. הדפיסי את הערך הסופי.

Condition Variable – הגדרה:

משתני תנאי הם אמצעי נוסף לסנכרון בין תהליכים בתקן POSIX. נשתמש ב-CV בתוספת לסמאפורים כדי לסנכרן בין תהליכים.

זהו סוג של משתנה הבודק אם תנאי מסוים התקיים או לא, [הוא כמו תור (Queue) שלא עובד בשיטת FIFO]: תהליכון בודק אם התנאי שב-CV התקיים, ואם התנאי לא התקיים – הוא ייכנס לתור ההמתנה. ברגע שהתנאי נבדק שוב ע"י תהליכון אחר ואכן מתאמת – הוא יישלח `signal()` לתהליכון שבתור כדי שיתעורר.

היתרון: הרדמת תהליכים / תהליכים בלי לבזבז CPU.

הפעולות המוגדרות על CV:

1. `wait()` – המתנה שתנאי X יתקיים, תוך שינה.
2. `signal()` – הערת אחד התהליכים מהתור, לאות שהתנאי התקיים. [שליחת סיגנל ממש] הערה חשובה: אם אין תהליכים הממתינים בתור – פעולת ה-`signal()` מתבזזת. כאשר תהליכון שוב ימתין – הוא יוער רק ע"י ה-`signal()` הבא.
3. `broadcast()` – הערת כל התהליכים שבתור.

הערות:

- CV לא מבטיח בלבדיות!! לכן תמיד נשלב אותו עם semaphore. כך ה-CV יבצע המתנה [מתוך שינה ולא בלולאת המתנה] על הסמאפור, וכשיעירו את התהליכון הוא גם ינעל את הסמאפור.
- שליחת ה-`signal()` נעשית באופן **אקראי** לאחד מהתהליכים הממתינים.
- בעיה:
נניח וקיימים במערכת N תהליכים, הממתינים שיהיה ערך ב-DS כלשהו, ואז יוכלו נניח להדפיס אותו.
כעת נכנסו ל-DS N איברים. נשלחו N פונקציות `signal()`, ו-N תהליכים התעוררו, והלכו לטפל באיברים.
תוך כדי הטיפול נכנס ל-DS האיבר ה-(N+1). גם הוא ישלח `signal()`, אך לא יהיה תהליכון שיקבל אותו!!
כל התהליכים שסיימו – לא מודעים לאיבר החדש ששלח מקודם `signal()`
מה נעשה???
נוכל להוסיף משתנה mone שיספור כמה בקשות יש במערכת, ולפני שתהליכון יחזור לישון – יבדוק שערכו של ה-mone שווה ל-0.
כמובן – הטיפול ב-mone יתבצע בקטע קריטי!

Condition Variable VS Semaphore

Condition Variable	semaphore
התהליכון ימתין לתנאי שיתקיים.	הממתינים לסמאפור משנים את ערכו לקטן או שווה ל-0
אם אין תהליכון ממתין – ה- <code>signal()</code> אובד	<code>sem_post()</code> לא אובד גם אם אין ממתינים, רק ערך הסמאפור גדל
קיים <code>broadcast()</code>	אין <code>broadcast()</code>
<code>wait()</code> תמיד חוסם את התהליכון	<code>wait()</code> לא תמיד חוסם את התהליכון
מיועד לסימון שינויי תנאים / מצבים	מיועד לפתרון critical section

דוגמה לניהול קטע קריטי באמצעות CV וסמאפור:

נגדיר את המשתנה, ונאתחל אותו בפונקציית האתחול:

```
pthread_cond_t my_cv = pthread_cond_init();
```

נגדיר גם סמאפור, וכמובן גם נאתחל:

```
pthread_mutex_t mutex = pthread_mutex_init();
```

ננעל את הסמאפור:

```
status = pthread_mutex_lock(&mutex);
```

```
if(status)
```

נשלח את הסמאפור לשחרור, ונלך לישון עד שיעירו אותנו. [ההערה גם נועלת שוב את הסמאפור!]

```
status = pthread_cond_wait(&my_cv, &mutex);
```

```
Critical_Section();
```

הערת ממתין אחד – נבצע ע"י `pthread_cond_signal(&cv)`

הערת כל הממתינים – נבצע ע"י `pthread_cond_broadcast(&cv)`

שחרור ה-CV יתבצע ע"י `pthread_cond_destroy(&cv)`

חשוב!

לוק או מיוטקס + CV + סיגנל = דרישה לתנאי מסוים ולנעילה

לוק או מיוטקס + CV + ברודקסט = דרישה לתנאי מסוים ולנעילה ולהעיר יותר מאחד

Critical Region – תחום קריטי – הגדרה:

נניח שמספר תהליכים או תהליכונים חולקים משתנה משותף K – שחובה לטפל באופן מסונכרן, כך שכל אחד ישתמש בו באופן בלבדי. כמובן שלכל תהליך יש משתנים משל עצמו.

כדי לאפשר למישהו לגשת באופן בלבדי ל-K נשתמש בפקודה:

region K when(<condition>) <statement>;

פקודה זו מבטיחה לנו שתהליך יבצע את הפקודה שב-<statement> רק כאשר:

א. התנאי שב-<condition> התקיים

ב. אף אחד אחר אינו מטפל ב-K

כאשר 2 תהליכים יבצעו בו זמנית:

region K when(true) s1;

region K when(true) s2;

נקבל הבטחה ש-s1 ו-s2 לא יבוצעו במקביל. אחד מהם יקבל את K לטיפול, ובזמן הזה – השני ימתין. כשהראשון יסיים – השני יקבל את K.

[בעמוד 477 יש מימוש אבסטרקטי לבעיית היצרן-צרכן באמצעות critical region]

אובייקט זה קיים לדוגמה ב-JAVA ומספק פתרון תוכנה ברמה גבוהה. המוניטור מאפשר בלבדיות בטיפול במשתנים או משאבים משותפים. הוא מקושר לאובייקט, ולכן מתאים לשפות OOP, והמשתמש לא צריך לזכור לנעול בכל גישה. מבנה המוניטור יכול:

1. משתנה או מבנה נתונים [כמו data member רגיל]
2. קטע אתחול למבנה הנתונים הנ"ל [כמו פונקציית בנאי]
3. פונקציות שיתבצעו באופן בלבדי = תהליך 1 מבצע בכל נק' זמן
4. להוספת כוח – גם משתני condition variable [כמות- לפי הצורך] משתנה זה מאפשר לתהליך ללכת לישון עד שיוכל לפעול.

מצב בעייתי:

P1 הלך לישון בגלל CV, ו-P2 מעיר אותו $\leftarrow$ 2 תהליכים נמצאים בקטע הקריטי.

פתרונות:

1. מי שבא להעיר את השני – יושבה, עד שהשני יסיים את פעולתו
2. השני ימתין עד שהראשון יסיים את פעולתו.

כאשר נתעסק בבעיות מסוג בעיית הקוראים-כותבים, כלומר: יש 2 סוגי תהליכים: תהליכי קריאה ותהליכי כתיבה – נצטרך כלי אחר לפתרון וסנכרון.

הפתרון: מנעול. המנעול יוכל להינעל באופן **יחידני** ע"י כותב יחיד, או באופן **משותף** ע"י קוראים רבים.

שחרור המנעול ייעשה ע"י הפונ' `unlock()`

כשתהליך / תהליכון מעוניין לנעול את המנעול כדי להשתמש בו – והמנעול כבר תפוס לכתיבה של תהליך אחר – הוא **ייחסם**.

-?- מה יקרה כאשר: יש X קוראים על המנעול, מגיע כותב והוא נחסם מיד. מגיע קורא נוסף – הוא יצליח להיכנס???

-!- כדי למנוע הרעבה מהכותבים – נחסום גם את תהליכי הקריאה הללו.

כמובן שגם בסוג מנעול זה יש פונקציה שרק **מנסה** לנעול את המנעול, ע"מ להימנע מחסימה שלא לצורך, ואם ניסיון הנעילה נכשל- יוחזר כישלון.

דוגמה לניהול קטע קריטי באמצעות מנעולי R/W:

נגדיר את המשתנה:

```
pthread_rwlock_t lock;
```

נאתחל את המשתנה עם הגדרות ב"מ, ובבדוק אם חזרה שגיאה:

```
if(pthread_rwlock_init(&lock, NULL) == -1)
```

פונקציית הנעילה לכותבים, ופונקציית ניסיון נעילה לכותבים:

```
pthread_rwlock_wrlock(&lock)
```

```
pthread_rwlock_trywrlock(&lock)
```

פונקציית הנעילה לקוראים, ופונקציית ניסיון נעילה לכותבים: [כמובן שיש הגבלה על מס' התהליכים שאוחזים במנעול במקביל]

```
pthread_rwlock_rdlock(&lock)
```

```
pthread_rwlock_tryrdlock(&lock)
```

פונקציה להריסת המנעול:

```
pthread_rwlock_destroy(&lock)
```

בעיות סנכרון קלאסיות

כעת נביר מספר בעיות קלאסיות של בקרה מקבילית, עליהן נהוג לבחון מנגנוני ו/או פרוטוקולי סנכרון.

בעיית היצרן – צרכן = בעיית החוצץ המוגבל

בהינתן מאגר של N תאים [בכל תא ניתן להכניס פריט בודד] – נצטרך לדאוג לסנכרון פעולות המילוי והריקון במאגר זה.

פתרון אפשרי – 1:

נשתמש ב-3 סמאפורים, ונאתחל אותם בערכים הבאים:

```
int mutex = 1;
```

```
int empty = N;
```

```
int full = 0;
```

שימו לב שה-mutex הוא סמאפור בינארי, המשמש לנו כמנעול.

full ו-empty הם סמאפורים מסוג סמאפור מונה, ואינם משמשים עבורנו כמנעולים, לכן אינם mutex.

קוד היצרן ייראה כך:

```
while(true){  
    next = Create_Product();  
    wait(empty); // המתנה – כאשר אין מקום פנוי להוסיף אליו איבר  
    wait(mutex); // המתנה – נרצה לקבל רשות לגשת למאגר עצמו, ע"מ להוסיף אליו מוצר  
    Add_Product(next);  
    signal(mutex);  
    signal(full); // נאותת לשאר הצרכנים שנוסף איבר חדש למאגר  
}
```

קד הצרכן ייראה כך:

```
while(true){  
    wait(full); // המתנה – כאשר אין מוצרים שניתן להסיר מהמאגר  
    wait(mutex); // המתנה – נרצה לקבל רשות לגשת למאגר עצמו, ע"מ להסיר ממנו מוצר  
    Remove_Product();
```



```

signal(mutex);
signal(empty); // נאותת לשאר היצרנים שהתפנה מקום במאגר
}

```

מימוש בקוד – מצורף בקובץ הקוד וב-PDF

הערות:

* נשים לב ששטח הזיכרון המוקצה – גדול ב-2 משטח המאגר כולו. כך ש-2 התאים האחרונים ישמשו אותנו לשמירת **מספר** התא אליו נכניס מוצר בפעם הבאה, ולשמירת **מספר** התא ממנו נשלף מוצר.

* הפעולה `sem_open()` מקבלת כפרמטרים את:

1. שם הסמאפור [יכול להתחיל ב'\']
2. האם הסמאפור קיים או שיש ליצור אותו
3. הרשאות על הסמאפור
4. ערך תחילי לסמאפור [הערך '1' יציין סמאפור בינארי]

* קוד זה יכול להריץ סנכרון בין תהליכים או בין תהליכונים.

בעיית הקוראים – כותבים

נניח שקיים אובייקט, שפונים אליו שני סוגי תהליכים:

א. תהליכים הקוראים מידע מהקובץ

ב. תהליכים הכותבים לתוך הקובץ – מעדכנים את תוכנו

ייתכן כי קוראים רבים מעוניינים לפנות לאובייקט זה במקביל. מצד שני – נצטרך לאכוף מצב בו כל כותב – מבצע כתיבה לקובץ באופן בלבדי [למניעת דריסת נתונים או שליחת מידע שגוי לתהליכי קריאה].

בבעיה זו קיימים שני ווריאנטים בקדימויות שונות לפי התהליכים:

1. בעיית קריאה-כתיבה ראשונה: הרעבת הכותבים. כלומר – קורא יאלץ להמתין רק אם כבר קיים כותב שניגש לאובייקט. אם קיימים כבר קוראים – הקורא החדש יצטרף אליהם.
2. בעיית קריאה-כתיבה שניה: הרעבת הקוראים. כלומר – כותב יוכל לכתוב מהר ככל האפשר, כרצונו מבלי שיהיו קוראים באותו הזמן.

פתרון לווריאנט הראשון:

המשתנים הגלובליים:

```
int readers_cnt = 0;           // מונה לספירת הקוראים
semaphore rc_mutex = 1;       // סמאפור לקוראים, מטופל באחריותם בלבד
semaphore writers = 1;        // לקוראים ולכותבים
```

שגרת הכותבים:

```
writer() {
    wait(writers);
    Write_Data();
    signal(writers);
}
```

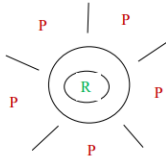
שגרת הקוראים:

```
read() {
    wait(rc_mutex);
    readers_cnt ++;
    if(readers_cnt == 1)
        wait(writers);
    signal(rc_mutex);
```

```
Read_Data();  
wait(rc_mutex);  
readers_cnt - -;  
if(readers_cnt == 0)  
    signal(writers);  
signal(rc_mutex);  
}
```

בעיית הפילוסופים הסועדים

סביב שולחן עגול יושבים 5 פילוסופים, והם עשויים לחשוב או לאכול. השולחן מסודר באופן הבא: בין פילוסוף לפילוסוף מונח צ'ופסטיק בודד.



בעת שהפילוסוף חושב ושקוע בהגיגיו [=קטע קוד שאינו קטע קריטי] – אין לו אינטראקציה עם עמיתיו. ובאשר הפילוסוף נתקף ברעב הוא מעוניין לאכול מסיר האורז שעל השולחן [=זקוק לביצוע קוד קטע קריטי]. לשם כך – הוא זקוק ל-2 צ'ופסטיקים (מקלות אכילה).

פעולה אטומית יחידה שהפילוסוף יכול לבצע היא- הרמת צ'ופסטיק אחד בלבד.

רק כאשר יש ברשות הפילוסוף 2 צ'ופסטיקים הוא רשאי לאכול, ובסיום האכילה- יניח את הצ'ופסטיקים וישקע בשרעפיו.

מטרתנו היא לספק את צרכי הפילוסופים מבלי לגרום להרעבה או לחסימה הדדית [קיפאון]

כיצד נפתור את הבעיה?

נחזיק סמאפור לכל צ'ופסטיק, שיאפשר להמתין כל עוד הסמאפור תפוס:

```
semaphore chopsticks[5] = {1, 1, 1, 1, 1};
```

כל תהליך פילוסוף יבצע:

```
while(true) {  
    wait(chopsticks[i]);  
    wait(chopsticks[ (i+1) % 5 ]);  
    Eat(); // CS  
    signal(chopsticks[i]);  
    signal(chopsticks[ (i+1) % 5 ]);  
}
```

אך אם כל הפילוסופים ייתקפו ברעב בו זמנית – תיגרם חסימה הדדית [קיפאון]

פתרונות:

1. נוסף צ'ופסטיק לשולחן, אחד הפילוסופים יאכל, ישחרר את מה שברשותו, ואז לפי התור- כולם יישבעו.
2. תפיסת הצ'ופסטיק תתבצע בתוך קטע קריטי, ובמצב בו תפסנו רק צ'ופסטיק אחד ואין עוד אחד פנוי – נשחרר גם את מה שתפסנו.
3. נמספר את הפילוסופים. פילוסוף אי-זוגי ייקח מקל משמאל, ופילוסוף זוגי ייקח מקל מימין. [נגיע להרעבה]

Deadlock

הגדרה: מצב בו N תהליכים, ממתינים לאירוע [או משאב] שרק תהליך אחר **מהקבוצה** יכול לשחרר. המשאבים יכולים להיות מסוגים או טיפוסים שונים, וכן מכל משאב קיימים מס' עותקים זהים זה לזה.

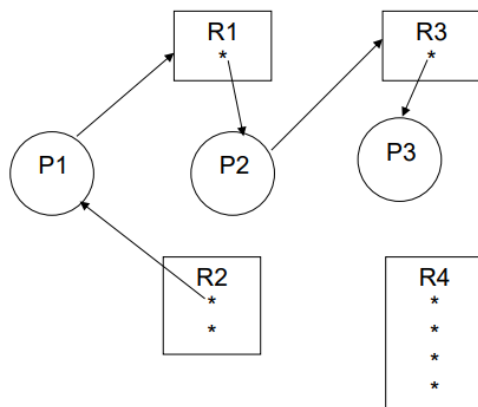
המערכת צריכה לעקוב אחר המשאבים: רשימה של הפנויים, רשימת התפוסים ומי אוחז בהם, וקבוצת תהליכים ממתינים לכל משאב.

תנאים הכרחיים ל-deadlock:

1. **mutual exclusion** – מניעה הדדית: אין אפשרות להקצות משאב במקביל למס' תהליכים.
2. **hold & wait** ['החזק והמתן'] – התהליך מחזיק במשאב X, וממתין שמשאב Y ישוחרר משימוש בתהליך אחר.
3. **ללא חטיפות** – לא ניתן להפקיע משאבים מתהליכים אחרים. [אחרת לא יתקיים תנאי ההמתנה המעגלית]
4. **המתנה מעגלית** – כל תהליך ממתין לתהליך הסמוך אליו מימין או משמאל.

נזהה את ה-deadlock ע"י 'גרף הקצאת המשאבים':

זהו גרף מכוון. ה**צמתים** בגרף יכולים לייצג את התהליכים (בצורת עיגול) ואת המשאבים (בצורת מלבן, ובתוכו נק' כמספר העותקים מהמשאב), וה**קשתות** מייצגות את בקשות ההקצאה ("קשת בקשה"), ואת ההקצאות בפועל ("קשת הקצאה").

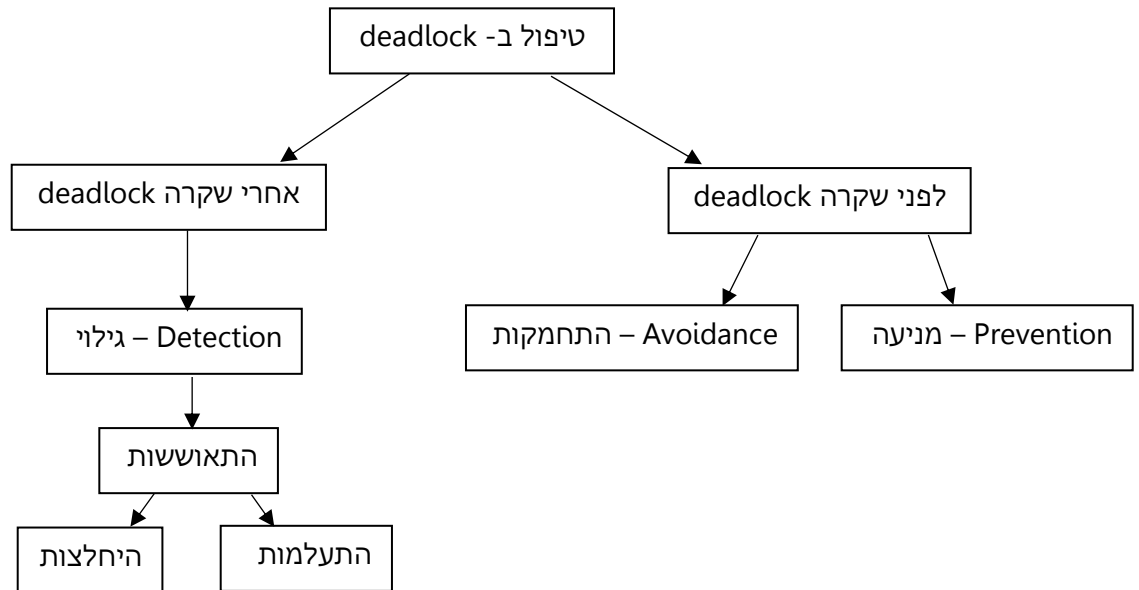


דוגמה לגרף הקצאת משאבים ←

1. האם המערכת כעת ב-deadlock?
2. נניח ו-P3 יבקש עותק של R2 זה יגרום ל-deadlock?
3. אם היה לנו עותק יחיד מ-R2, וגם P3 יבקש עותק של R2 זה יגרום ל-deadlock?

מסקנות:

- א. כדי שיהיה deadlock חייב להיות מעגל בגרף
- ב. אם יהיה לנו עותק 1 מכל משאב, וגם קיים מעגל – בהכרח שהמערכת ב-deadlock בעקבות התהליכים שבמעגל.



לפני שקרה deadlock:

1. Prevention – מניעה:

ננסה לגרום שאחד מ-4 תנאי הקיפאון לא יתקיים.

- את תנאי מס' 1 – לא נוכל למנוע
- את תנאי מס' 2 – נוכל למנוע ע"י שיטת "הכול או כלום". כלומר: תהליך יידרש לבקש את כל המשאבים שהוא זקוק להם בו-זמנית, ורק כאשר ניתן לספק לו את **כולם** נקצה אותם עבורו.
נוכל 'להחליש' מעט את הדרישה, ולתת משאבים לתהליך שמבקש, רק אם אין ברשותו משאבים אחרים ברגע זה. למעה "פ קל מאוד לבדוק האם יש לתהליך משאבים שכרגע ברשותו.
- חסרונות:**
 - תהליך יכול לבקש משאב שיזדקק לו בעתיד הרחוק, ואז יש ירידה במקביליות.
 - קיים חשש להרעבת תהליך הזקוק למשאב 'מבוקש'.
- את תנאי מס' 3 – ניתן תמיד עדיפות לתהליך שרץ. נוכל למנוע ב-2 דרכים:
 - "הענשה" – תהליך המחזיק במשאבים, ומבקש נוספים – ינושל לחלוטין מכל משאביו, ועובר להשהיה / חסימה.
 - כאשר תהליך מבקש משאב תפוס, נבדוק אם מי שמחזיק בו כרגע נמצא במצב waiting: אם כן- נחטוף ממנו את המשאב.
 אך אם המשאב תפוס ע"י תהליך אחר שרץ במקביל – התהליך המבקש יועבר להמתנה.
- את תנאי מס' 4 – נוכל למנוע ע"י הטלת סדר על המשאבים, כלומר: נהיה חייבים לבקש את A לפני B.
נוכל לקבוע את הסדר בין המשאבים באופן שרירותי או הגיוני.

2. Avoidance – התחמקות:

כאן נדרוש מכל תהליך להצהיר בתחילת הריצה – לרשימת המשאבים בהם הוא עלול לעשות שימוש. ברגע שהמערכת תזהה כי אם נקצה משאב כלשהו לתהליך ניכנס לקיפאון – הבקשה לא תאושר.

"מצב בטוח" – יש ברשות המערכת מספיק משאבים כך שהיא תאפשר לכל תהליך לקבל את כל המשאבים הדרושים לו, מבלי להיכנס לקיפאון. [לא בו-זמנית, אלא בזה אחר זה].
כלומר: יש סדרה בטוחה של תהליכים בה לכל תהליך מתקיים שכמות המשאבים שהוא רשאי לדרוש קטנה או שווה ל [מס' המשאבים הפנויים + מס' המשאבים המוקצים לתהליכים שלפניו בסדרה].

הנוסחה היא:

$need[i, j] \leq available[j] + Sum_Resources(0, i, j);$
כאשר: i מסמן את מס' התהליך בסדרה, j את מס' המשאב המבוקש, והפונקציה $Sum_Resources$ מקבלת כפרמטרים את טווח התהליכים שלפניו ואת המשאב הרצוי, ומחזירה את כמות המשאבים שיתפנו בעקבותיהם.

"מצב לא בטוח" – בטוח יהיה קיפאון / אולי יהיה קיפאון. [אין סדרה המקיימת את הנ"ל]
לא חייב שמערכת הנמצאת במצב שאינו בטוח תהיה ב-deadlock אך הסיכון שלה גבוה.

ביצד מעה"פ תזהה את המצב הנוכחי?

1. כאשר יש עותק יחיד מכל משאב – נשתמש באלגוריתם פשוט.
2. כאשר יש יותר מעותק אחד לכל משאב במערכת נצטרך אלגוריתם מורכב יותר.

נפרט:

1. נשתמש בגרף הקצאת המשאבים המשמש אותנו לזיהוי deadlock, ונוסיף עוד סוג של קשת – "קשת היתכנות" [יכול להיות שהתהליך יבקש את המשאב, לא וודאי].
 - כשתהליך אכן מבקש משאב נחליף את קשת ההיתכנות בקשת בקשה
 - כשתהליך אכן מקבל את המשאב קשת הבקשה מוחלפת בקשת הקצאה
 - כשתהליך משחרר את המשאב נחליף את קשת ההקצאה בקשת היתכנותבכל פעם שתהליך מבקש הקצאה – נבדוק אם הפיכת קשת הבקשה לקשת הקצאה לא יוצרת מעגל. אם עלול להיווצר מעגל – הבקשה לא תאושר והתהליך ייחסם. אלגוריתם זה מתאים רק כאשר קיים עותק יחיד מכל משאב במערכת.

2. **אלגוריתם הבנקאי –** אלגוריתם זה מכונה כך ע"ש הלוואות הבנקאים: הבנק בודק אם הוא יכול להקצות אשראי נוסף ללקוח ועדיין להישאר במצב בטוח, ולפרוע את כל חובותיו.

נסמן ב- N את מס' התהליכים

נסמן ב- M את מס' סוגי המשאבים

האלגוריתם ישתמש בנוסף במבני הנתונים הבאים:

- מערך $available$ – השומר את מס' היחידות הפנויות מכל משאב
- מטריצת max – השומרת לכמה יחידות מקסימום ממשאב יזדקק כל תהליך

- מטריצת allocation – השומרת כמה יחידות משאב אכן הוקצו לכל תהליך
- מטריצת need – שומרת כמה יחידות מהמשאב חסרות עבור כל תהליך
- די מיותר, כמו: [max-allocation]
- מערך finish – האם התהליך הוקצה במלואו באופן בטוח

כלומר: לפני אישור הקצאת המשאב- נשנה את מצב המערכת, כאילו כבר הקצינו. ואז נבדוק האם המערכת בטוחה (safety algorithm)
האלגוריתם פועל באופן האינטואיטיבי הבא:

- חפש את התהליך שבקשותיו $\geq$ כמות המשאבים הקיימת.
- אם בסיום הלולאה – כל התהליכים מסומנים כמוקצים = מצב בטוח.

```
int temp[M] = available
bool finish[N] = {false};
while(true) {
    // find p doing: finish[p] == false && need[p] <= temp
    else
        break;
    temp += allocation[p];
    finish[p] = true;
}
```

נעת שוב נעבור בלולאה על המערך finish ונבדוק שכולם עד לאינדקס i במצב true. כך נדע שהמצב בטוח.

האלגוריתם הראשי מקבל בכל פעם את ווקטור בקשת המשאבים לתהליך ובודק את הבקשה. ווקטור הבקשות הוא: int request[M].
האלגוריתם מבצע:

- בדיקה אם $request > need[p]$ או ש: $request + allocation[p] > max[p]$
- אם כן – מחזירה שגיאה.
- בדיקה אם $request > available$ – התהליך יצטרך להמתין לשחרור משאבים
- הקצאת המשאבים תגרום למערכת לעבור למצב:

```
available -= request
allocation[p] += request
need[p] -= request
```


Detection – גילוי: המערכת תרוץ בלי הפרעות או בדיקה מראש, ורק מידי פעם ננטר אותה. במציאת קיפאון (מעגל בגרף הקצאת המשאבים) - נתאושש: נעיף את אחד התהליכים הבעייתיים שגרמו לקיפאון.

-? - מתי נחפש את הקיפאון?

* תלות בשכיחות החסימות ההדדיות, עד כמה חשוב לנו למנוע deadlock, עלות אלגוריתם הגילוי.

* במקרה קיצון: בכל הקצאת משאב לתהליך.

* אופציה נוספת: כאשר יורדת ניצולת המעבד מתחת לסף כלשהו (40%) - יופיע חשד ל-deadlock.

מערכת עם פריט יחיד מכל סוג משאב:

נשתמש ב'גרף המתנה' – הצמתים הם תהליכים, והקשתות מסמנות בקשות.

אם קיימים מעגלים בגרף אז יהיה deadlock

לכן נצטרך: לתחזק את הגרף, וגם מידי פעם לנטר ולבדוק אם נוצר מעגל.

מערכת עם מספר פריטים מכל סוג משאב:

דומה לאלגוריתם הבנקאי, אך השוני מתבטא בכך שהבדיקה- אם ייווצר קיפאון- מתבצעת לאחר ההקצאה כולה.

האלגוריתם ישתמש בנוסף במבני הנתונים הבאים:

- מערך available – השומר את מס' היחידות הפנויות מכל משאב
- מטריצת allocation – השומרת כמה יחידות משאב אכן הוקצו לכל תהליך
- מטריצת request – שומרת לכל תהליך לכמה תהליכים מסוג J הוא ממתין

האלגוריתם יפעל כך:

```
int temp[M] = available
bool finish[N] = {false};
while(true) {
    // find p doing: finish[p] == false && request[p] <= temp
    else
        break;
    finish[p] = true;
    temp += allocation[p];
}
```

}

המערכת אכן ב-deadlock אם יש תהליך ש: $finish[p] == false$

-?- כיצד נתאושש?

!- 2 אופציות בפנינו:

1. **התעלמות:** מעה"פ מדווחת למפעיל על deadlock והתהליכים המעורבים בו. מועברת לאחריות המפעיל.

2. **Recovery – היחלצות:** נבטל את כל התהליכים שקפאו בבת אחת /
נבטל תהליך אחר תהליך עד להיחלצות /
נפקיע [נחטוף] את המשאבים מהתהליכים

מציאת קבוצה אופטימלית (של תהליכים / משאבים) לביטול / הפקעה- קשה מאד!

בחירת הקורבן: לפי קדימות התהליכים, זמן חישוב שהושקע בתהליך וכמה יידרש בעתיד, כמות ומספר המשאבים שבשימוש כעת, לכמה יזדקק בעתיד, כמה תהליכים נותרו (באופן כללי), תהליך אינטראקטיבי / אצווה, פעולות בלתי הפיכות שכבר נעשו, וכיצד נישמר מהרעבה.

מבוא לניהול הזיכרון

תזכורת: תוכנית שוכנת בדיסק, בזמן ההרצה – הקוד נטען לזיכרון ב-PCB של התהליך. הוא מחולק למקטעים, ביניהם:

1. Code / Text – מכיל את פקודות התוכנית.
2. Data & bss – מכילים נתונים סטטיים וגלובליים.
3. Stack & Heap – מכילים פרמטרים, משתנים אוטומטיים והקצאות דינאמיות.
4. מקטעים נוספים כמו shared memory.

בעת הריצה: המעבד קורא פקודה מהזיכרון לאוגר ה-IR [במידת הצורך קורא גם את האופרנדים של הפקודה לאוגרים נוספים. כך המעבד מבצע את הפקודה, ולעיתים גם שומר את התוצאה בזיכרון.

ה-CPU רשאי לפנות לכל ה-address space של התהליך שמורץ בו כרגע.

בשנגדיר בקובץ המקור של התוכנית שורה כזו (לדוגמה):
`int number;`
מדובר בכתובת סמלית (symbolic address) – שאין לנו דרך לדעת היכן תאוחסן בזיכרון הפיזי.

בזמן הקמפול – המהדר יקבע: כמה בתים יוקצו לכתובת זו, וכן קביעות שונות על כתובת תא הזיכרון הפיזי [בו יאוחסן המשתנה בזמן הריצה] וכו'.

-?- מה קובע המהדר?

!- המהדר קובע מס' חלקים. ביניהם:

א. Absolute Code – קוד מוחלט:

כאשר ה-CPU יודע באילו כתובות זיכרון תימצא התוכנית בזמן הריצה, הוא יוכל לקבוע את הכתובת הפיזית המוחלטת והסופית של המשתנה [לדוגמה: 1266028]
+ בזמן טעינת התוכנית אין צורך לחשב את הכתובות, וכמובן שלא בזמן הריצה
- המהדר חייב לדעת את כתובת התוכנית.
[בקבצי .com במערכת ההפעלה MS-DOS השתמשו בגישה זו]
יש אפשרות לשמור כמה גרסאות של התוכנית – למס' אופציות של כתובות.

ב. Relocated Code – קוד הניתן להזזה:

בגישה זו – המהדר לא יודע / לא קובע את כתובת התוכנית בזיכרון, אך ידוע שלאחר טעינת התוכנית לזיכרון – היא לא תזוז מכתובות אלו עד סיום הריצה.
במצב זה – תיקבע 'כתובת יחסית למשתנה' – כתובת הניתנת להזזה. בעת טעינת התוכנית לזיכרון [ע"י ה-loader], הוא ימיר את הכתובות היחסיות בכתובות מוחלטות.
כלומר: כעת מוטלת יותר עבודה על ה-loader, אך מנגד נרוויח גמישות יצר במיקום הפיזי של התוכנית בזיכרון.

ג. PIC (Position Independent Code) – קוד תלוי מיקום:

הזכרנו כי מעה"פ יכולה לשחלף תהליך מזיכרון ה-RAM לדיסק הקשיח, (ע"י ה-swapper).
כמובן לכתובות השונות מהכתובות בהן התוכנית שוכנת בקביעות.
במצב זה הכתובות היחסיות לא יומרו לכתובות מוחלטות בשום שלב. אלא בכל גישה לכתובת השוכנת במרחב זיכרון של תהליך מסוים – תתבצע המרה של כתובת יחסית בכתובת מוחלטת. כיצד? נוסיף את הכתובת היחסית לערכות של האוגר BASE.

+ נוכל להגדיל את גמישות הזיכרון מאוד, תוך שימוש בשחלוף בעת הצורך.
- מחיר לתשלום: עבור כל כתובת **לוגית** (יחסית) תתבצע המרה לכתובת **פיזית** (מוחלטת)

ד. שיטה נוספת – זיכרון מדומה:

במערכות מבוססות virtual memory (זיכרון מדומה)
עד כה חשבנו שבעת טעינת התוכנית לזיכרון – נטען את התוכנית בשלמותה. מעתה נדע
שיש אפשרות להריץ תוכניות במחשב מבלי לטעון כל הקוד לזיכרון. [אפילו תוכניות שגדולות
מגודל זיכרון ה-RAM]
מה הרעיון?

- כל פעם נטען לזיכרון חלק מהתוכנית.
- כאשר התוכנית ניגשת לכתובת חוקית שלא הבאנו לזיכרון ה-RAM, יתבצע ניהול
page fault (חריגת דף) – ואז נטען לזיכרון את הכתובת הדרושה.

לפי שיטה זו – נטען באופן דינאמי כל פעם חלק אחר לזיכרון, וגם כל חלק נטען למקום
אחר בזיכרון (החלקים מפוזרים ע"פ מקטעים ב-RAM).

לניהול שיטה זו נצטרך אוגר BASE לכל חלק שננטען ל-RAM.

היתרון הוא שכריכת ספריות מתבצעת באופן דינאמי.

הסבר: כשנרצה לכתוב פונ' כלשהי לתוכנית, נוכל לכתוב באחד מהאופנים הבאים:

1. **כריכה סטאטית**: קוד הפונ' מתווסף לקובץ הבינארי של התוכנית [הגדלת גודל
הקובץ] כלומר: כעת הפונ' היא חלק ממשי מקוד התוכנית.
2. **כריכה דינאמית**: במקום להוסיף לקובץ הבינארי את קוד הפונ' – נוסיף לו stub
(=בדל) שיאפשר למעלה"פ לאתר את קוד הפונ' תוך כדי ריצה.
כך נרוויח קובץ שכמעט ולא גדל בגלל זימן הפונ', וכן את האפשרות לזמן קוד באופן
דינאמי תוך כדי ריצה.

-?- מדוע שיטה זו מכונה 'זיכרון מדומה'?

!- מכיוון שחלק מהתוכנית נמצא בדיסק, והרי הדיסק הוא חלק מהזיכרון, וזו ההדמיה...

ניהול הזיכרון

שיטות לניהול הזיכרון:

1. הקצאה רציפה

2. paging

סוגי טבלאות דפים – PT:

- טבלת דפים פשוטה – לא ישים בהקצאה רציפה
- טבלת דפים כעץ
- טבלת דפים מגובבת- מוחזקת ע"י טבלת גיבוב
 - גרסת טבלה מגובבת גלובלית
- טבלת דפים הפוכה / מהופכת

3. Segmentation

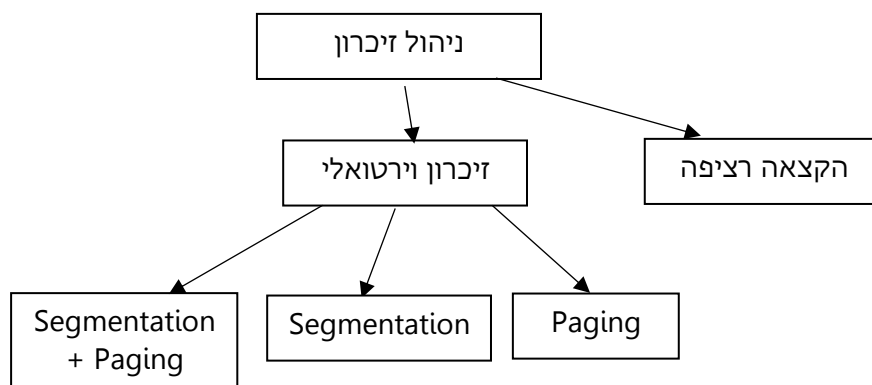
- גרסה משולבת: Segmentation + Paging

מטרות ניהול הזיכרון:

1. חיסכון במקום ב-RAM
2. הרצת מספר תהליכים במקביל, תוך הגנה על נפרדות הזיכרון ביניהם
3. יכולת שימוש בדיסק הקשיח כהרחבה זולה (ואיטית) של הזיכרון
4. אפשרות לשינוי מיקום מבני הנתונים של התוכנית תוך כדי הרצה (מבלי להשפיע עליה)

תזכורת – היררכיית מבנה הזיכרון:

1. רגיסטרים
2. זיכרון מטמון cache (רמות)
3. זיכרון ראשי- RAM
4. זיכרון משני- דיסק קשיח



1. הקצאה רציפה

בשיטה זו – כל התוכנית נטענת **בשלמות וברציפות** לזיכרון, כולל האפשרות לשחלוף. כך תהליך יוכל לרוץ רק אם יש עבורו מקום פנוי בזיכרון.
כאשר התוכנית מתוזמנת לריצה – בעת ה-context switch – נטען את ערכו של האוגר BASE (יש כאלו המכנים אותו reallocation), וכל כתובת לוגית [שיוצר המעבד] מתווספת לערכו של האוגר – וכך מגיעים לכתובת הפיזית.
(מיועד בעיקר למערכות משובצות מחשב-Embedded ומערכות זמן אמת-RT-real time)

-?- כיצד מתבצעת ההקצאה לתוכנית?

!- הפתרון הפשוט והבסיסי יהיה לחלק את הזיכרון ל-N חלקים בגודל קבוע אך לא בהכרח שוויו גודל. כך כל חלק יכיל תהליך – שכמעט ימלא את כל החלק, אך לא בהכרח.
הפתרון המתוחכם: מערכת ההפעלה תחזיק טבלה של המקומות הפנויים והתפוסים.
'שטח פנוי' – זהו חור, והוא מאופיין ע"י כתובת ההתחלה והגודל הכולל.
בתחילה – כל הזיכרון פנוי ← יש חור יחיד.

בכל פעם שמגיע תהליך חדש – מערכת ההפעלה תחפש עבורו חור מתאים, ורק אז התהליך ייטען לזיכרון ויוכל לרוץ. כנ"ל לגבי תהליך ששוחלף לדיסק.

אופציות לשיפור:

- בשחרור זיכרון מתהליך- נבדוק אם קיים חור סמוך ונאחד אותם יחד.
- לעיתים נבצע 'איחוי' = איחוד חורים תוך הזזת תהליכים בזיכרון.

-?- כיצד נבחר כל פעם 'חור' מתאים?

!- ישנם 4 סוגי פתרונות:

גישה:	הסבר:	יעילות:
First Fit	מחפש כל פעם מחדש ברשימת החורים את החור הראשון שמתאים	זמן חיפוש יעיל ניצול זיכרון יעיל
Next Fit	מחפש כל פעם מהמקום האחרון שמצא , ברשימת החורים את החור הראשון שמתאים	פחות מ-First Fit
Best Fit	מחפש כל פעם מחדש ברשימת החורים את החור הראשון הכי קטן שמתאים	זמן חיפוש יעיל ניצול זיכרון יעיל
Worst Fit	מחפש כל פעם מחדש ברשימת החורים את החור הגדול ביותר כרגע	פחות מ-First Fit ומ-Best Fit

מסקנות בעקבות סימולציות:

- שיטת worst fit אכן גרועה ביותר
- שאר השיטות שקולות זו לזו מבחינת ניצולת הזיכרון, והן סובלות מפרגמנטציה חיצונית.

יתרונות בהקצאה רציפה:

1. הגנה מגישה לאזור לא חוקי בזיכרון
2. נIOD כתובות של תוכניות
3. שינוי דינאמי של זיכרון מעה"פ
4. תמיכת חומרה פשוטה

חסרונות בהקצאה רציפה:

1. אם אין מקום בזיכרון לתהליכים- נבצע swapping לזיכרון המשני = פעולה יקרה
2. פרגמנטציה פנימית- חורים בשטח התהליך (כאשר המחיצות קבועות!)
3. פרגמנטציה חיצונית- חורים בשטח הזיכרון (כאשר המחיצות משתנות!)
4. קושי בתמיכה להגדלת זיכרון לתהליך
5. חיוב שכל הקוד יהיה ב-RAM
6. קושי תמיכה לתהליכים גדולים
7. מוגבל בכמות תהליכים להרצה

פרגמנטציה חיצונית: עם הזמן נוצרים חורים קטנים ובלתי שימושיים – מחוץ לשטחי הזיכרון המוקצים, ולכן זוהי פרגמנטציה **חיצונית**. [לדוגמה – במערכת בעלת גבולות נעים].

☐ כלל ה-50% : שטחם של החורים = שליש מנפח הזיכרון = חצי משטחי הזיכרון שבשימוש.

פתרון: ביצוע 'איחוי' או דחיסה – במחיר עבודה לא מבוטלת.

פרגמנטציה פנימית: בד"כ הקצאת הזיכרון מתבצעת ב'גושים' של בתים, ולא בבתיים בודדים. לכן כמעט תמיד תהליך יקבל מקום בזיכרון – יותר ממה שהוא צריך באמת. כאן החורים פנימיים-בתוך שטח הזיכרון שהוקצה לתהליך, ולכן זו פרגמנטציה **פנימית**. [לדוגמה – במערכת בעלת גבולות קבועים].

2. Virtual Memory – זיכרון וירטואלי

בשיטת הזיכרון הווירטואלי – התוכנית נמצאת בזיכרון, אך לא ברציפות ולא בשלמות. [בהמשך נראה הרחבה על זיכרון וירטואלי]

אפשרות א – Paging

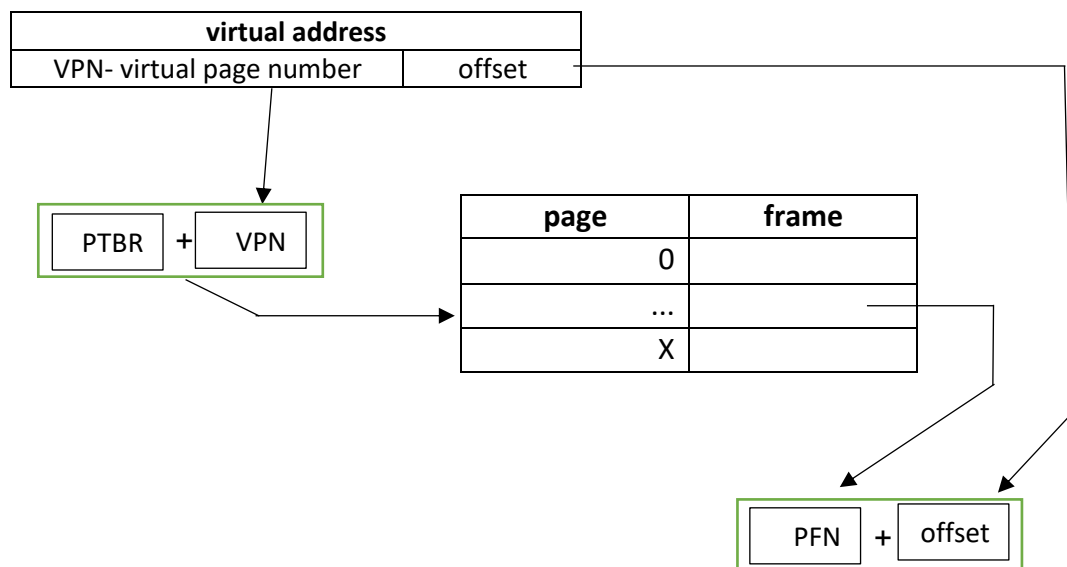
נשתמש בתוכנית כמו קריאת **עמודים** בספר. כך שרק נראה שכל התוכנית בזיכרון, אבל היא שמורה בחלקה בדיסק.
את הזיכרון **הפיזי** נחלק ל-**frames** [מסגרות]
ואילו את הזיכרון **הלוגי** נחלק ל-**pages**
כך נשמור כל page לוגי – ב-frame פיזי.

בעת טעינת תהליך – נטען page'ים ל-frame'ים הפנויים, ולא בהכרח שהם פנויים באופן רציף. נשתמש ב- 'page table' – ע"מ לשמור את מיפוי דף i של התהליך – למסגרת j.

כאשר ה-CPU מייצר כתובת לוגית חדשה – אנו נתייחס לכתובת ככזו המורכבת מ-2 רכיבים: מספר הדף הלוגי [page number] + מספר היסט בתוך הדף [page offset].
והרי מספר הדף הוא האינדקס בטבלת הדפים – אז נוכל לגשת ל-frame הנכון, נוסיף את ההיסט הנכון, ואז ניגש לכתובת הנכונה בזיכרון.
מבחינה רעיונית – טבלת הדפים מקבילה לאוגרי ה-BASE וה-OFFSET, רק שבעת נסיט כל דף בנפרד ולא את כל התוכנית.

החלק העליון בזיכרון הפיזי מכיל את הטבלאות עבור המרחבים הווירטואליים של כל התהליכים (טבלה לכל תהליך), ואילו החלק התחתון מכיל את ה-frame'ים.

תהליך תרגום הכתובת הווירטואלית לכתובת פיזית יראה כך:



לסיכום, הנוסחה היא: $PTE_ADDRESS = [PTBR] + (VPN * PTE_SIZE)$

הרכבת הכתובת הלוגית:

נרכיב את הכתובת מ:

1. מספר הדף (number page)
2. ההיסט בדף (offset number)

שלבי מציאת הכתובת הפיזית:

1. נקבל כתובת ווירטואלית בבסיס עשרוני
2. נמיר את הכתובת למספר בינארי
- כיצד נדע כמה סיביות מיועדות ל-offset וכמה ל-VPN?
- א. בהנחה שידוע לנו גודל ה-page – נמצא את ה-offset:
 1. נחשב את מספר ה-bytes שיש בכל ה-page
 2. סה"כ מספר ה-bytes הוא כמות הכתובות האפשריות
 3. כמות סיביות ה-offset היא: $\log_2$ [כמות הכתובות]
- ב. בהנחה שידוע לנו מספר ה-pages הקיימים – נמצא את ה-VPN:
 1. כמות הסיביות הנדרשות לייצוג ה-VPN הוא: $\log_2$ [כמות ה-pages]

תהליך חישוב הכתובת:

1. ניגש לכתובת הבסיסית של טבלת התרגום. (הרגיסטר PTBR)
 2. נשרשר אליו את ה-VPN
 3. ניגש לכתובת המלאה שנוצרה – במיקום בטבלת הדפים.
 - כך קיבלנו את הכתובת הפיזית - **PFN**
 4. נשרשר ל-PFN את ה-offset
 5. ניגש לזיכרון בכתובת שנוצרה מהשרשור
- אם הדף לא נמצא בזיכרון- ייווצר exception page fault והדף הרצוי ייטען לזיכרון במקום פנוי.

לדוגמה:

- נתונה כתובת פיזית בת 5 סיביות. לכן, גודל הזיכרון הוא: $2^5 = 32$
 - נניח שהזיכרון מחולק ל-frame'ים בגודל 4 בתים. ($2^2 = 4$)
- מסקנות: **א.** יהיו סה"כ 8 מסגרות, כי $32/4 = 8$
- ב.** מס' סיביות ה-offset הוא 2, ו-3 סיביות למס' ה-page.

- נניח שתהליך זקוק ל-5 page'ים
- נתונה טבלת הדפים של התהליך:

page number	frame number
0	6
1	3
2	1
3	2
4	5

- נביח שה-CPU הנפיק את הכתובת הלוגית: 01011 (= 11 בבסיס עשרוני)
- הרי 3 הסיביות השמאליות ביותר מסמנות את מספר ה-page הרצוי: 010, כלומר page מס' 2 בבסיס עשרוני.
- לפי טבלת הדפים – 2 page נמצא ב-frame-1. נמיר את ה-frame לבינארי: **001**.
- נשארנו עם היסט של 11
- לכן הכתובת הפיזית הסופית תהיה **00111** = בהמרה לעשרוני: 7.

נשים לב שקביעת Page'ים בגודל קטן גורמים ל:

- פרגמנטציה פנימית קטנה
 - טבלת דפים גדולה
 - השיעור היחסי של תקורה בעת טעינת דף [או הסרתו] עולה
 - טעינת הלוקליות של התהליך מדויקת יותר
 - יותר page faults ע"מ לטעון חלק מהתוכנית לזיכרון.
- במעלה"פ Linux גודל דף בד"כ 2^{12} בתים.
- אם נייצר את ה-page'ים בגודל גדול יותר, נגדיל את ה-TLB reach $\leftarrow$ זמן הגישה האפקטיבי יורד.

הבעיה: נשים לב לבעייתיות שבגישה לזיכרון: נצטרך לגשת 2 פעמים לזיכרון = זמן הגישה מוכפל ב-2!

פעם אחת- ניגש לטבלת התרגום, ופעם שניה ניגש לכתובת מהטבלה- בזיכרון הפיזי.

הרי מהירות הגישה לזיכרון איטית ממהירות המעבד עד פי 1,000. כך שהביצועים נפגעים משביתת המעבד במספר גדול של מחזורי שעון.

הפתרון:

TLB – (Translation Look-aside Buffer): זיכרון מטמון מהיר המכיל חלק מטבלת הדפים.

כך נחפש קודם את ה-frame של ה-page הרצוי ב-TLB. ורק אם לא נמצא שם- ניגש לטבלת הדפים. TLB פותר את הגישה ה**ראשונה** לזיכרון.

TLB miss – חיפשו ערך ב-TLB ולא מצאנו.

TLB ratio – אחוז הפעמים שחיפשו ערך ב-TLB ואכן מצאנו.

TLB reach – כמות הכתובות שניתן להגיע אליהן באמצעות ה-TLB.

- נביח נתון address space בגודל 32 סיביות
- גודלו של כל page הוא 2^{12} בתים

מסקנה: מספר הכניסות (שורות) בטבלת הדפים יהיה כמעט מיליון $[2^{32}/2^{12} = 2^{20}]$

-?- אם כך- מה היתרון של TLB?

1- עיקרון הלוקליות טוען ש: בנקודות זמן סמוכות, ניגש לכתובות סמוכות. [זוהי לוקליות בזמן ולוקליות במקום].

לכן, ככל שהדפים גדולים יותר ה-TLB reach גדל.

האחריות לטיפול ב-TLB מוטלת על החומרה (הוספת כניסה או פינוי כניסה). אך כאשר מעה"פ פולטת דף לדיסק או מזיזה דף בזיכרון – היא תעדכן את ה-TLB.

בעת ביצוע context switch נצטרך לנקות את ה-TLB. במידה וה-TLB מחזיק גם את מס' התהליך – נוכל להשאיר כתובות של מס' תהליכים במקביל באותו TLB.

דוגמה:

- נתון TLB ratio = 80%
- זמן חיפוש ב-TLB הוא 20ns (10^{-9} ns)
- זמן גישה לזיכרון הוא 100ns
- [זמן גישה לאוגר הוא 0.5ns | זמן גישה לדיסק הוא 25ms (10^6 ms)]

לכן: זמן הגישה בפועל לזיכרון יהיה:

1. ללא טבלת דפים כלל = 100ns
2. טבלת דפים בלבד [ללא TLB] = 200ns (שימו לב שזה כפול מזמן רגיל!)
3. טבלת דפים + TLB = 140ns

TLB והוגנות:

טבלת הדפים מסייעת לשמירה והגנה על הזיכרון.

בכל כניסה לטבלה [Page Table Entry – PTE], בנוסף לשמירת מס' ה-page ומס' ה-frame – נשמור מס' סיביות נוספות להגנה. כל PTE מכיל את הסיביות הבאות: [אין צורך לזכור את שמות הסיביות]

v	r	m	p	s	PFN
-valid	-reference	-modified / dirty	-protection	-user/ supervisor	physical frame number
מציינת אם ה-page קיים בזיכרון הפיזי. אם לא- 1. או שקיים בדיסק (ואז swap ניקח מ- area ו-slot) 2. או שמדובר בכתובת לא חוקית	מציינת האם ניגשו לאחרונה ל- page זה (מסייע לשיקולי פינוי pages)	מציינת האם התרחש שינוי בדף לאחרונה (כתיבה ל- page) אם היה- נעדכן בדיסק את ה-page לכשנפנה את ה-page מהזיכרון	מציינת הרשאות read, write, execute כדי להגן על הזיכרון	מציינת האם ניתן לפנות ל- page במצב מיוחס. $\text{kernel} \leftarrow 0$ $\leftarrow 1$ כל התהליך	מספר ה-frame בה נמצא ה-page בזיכרון הפיזי

מעבר לכך, נוכל לשמור גם אם הדף משותף למס' תהליכים, האם נכלל ב-address space של התהליך, האם הוא כעת בדיסק או בזיכרון [RAM] ועוד.

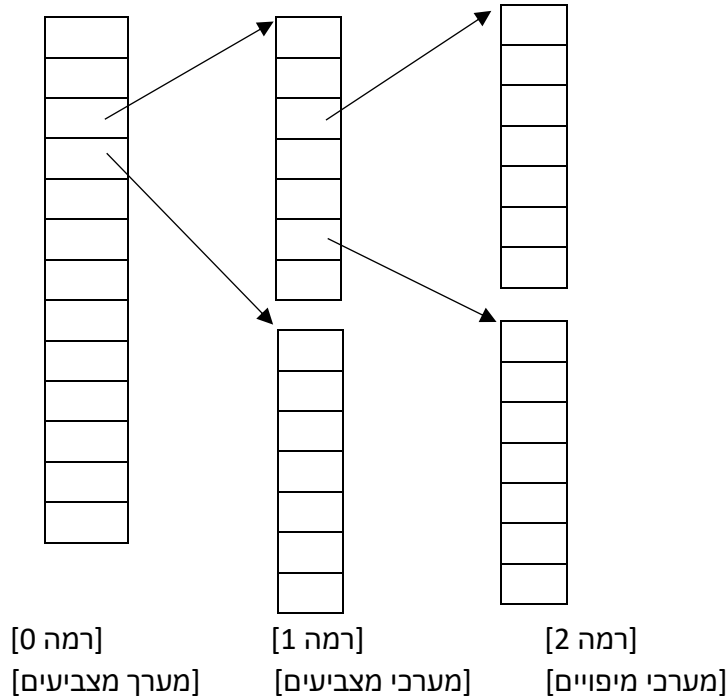
??- כיצד בנויה טבלת הדפים?

!!- כיום כתובת מכילה 32 או אפילו 64 סיביות. לכן גודלו של מרחב הכתובות הוא 2^{32} בתים. ובהנחה שגודל דף הוא 2^{12} בתים – מספר ה-page'ים יהיה 2^{20} בתים $\leftarrow$ בערך מיליון דפים! וכן אם כל PTE בטבלת הדפים מחייב 4 בתים – נפח הטבלה יהיה: $4 * 2^{20} = 4\text{MB}$ (אפשר גם לחשב את זה כך: $2^{12} = 2^{22} / 2^{10} = 2^2 + 2^{20}$)
הקצאת 4MB ברציפות בזיכרון – גורמת לאילוף קשה. בפרט שיכולות להיות כניסות רבות ריקות כי הקוד מצוי בקצה התחתון של ה-PCB וה-stack בקצה העליון, ויכול להיות חור ביניהם.

לכן, נצטרך למצוא דרך לשמור את טבלת הדפים בדרכים נוספות:

טבלת דפים היררכית:

שיטה זו מקובלת ב-Linux – נהפוך את הטבלה לעץ בעל 2 רמות לפחות, ולעיתים יותר. המיפויים יישמרו בעץ חיפוש, מס' ה-page מפורק לקבוצות של סיביות, כך כל קבוצה גבוהה מצביעה על תתי הקבוצות שלה.



כך: אם גודל כתובת הוא 32 סיביות, וגודל דף הוא 2^{12} בתים – אז 12 הסיביות הימניות יציין את ההיסט, ו-20 סיביות שמאליות – יציין את מס' הדף. כלומר: נחלק אותן ל-10 ו-10. שמאליות ביותר יגדירו את מספר ה-PTE בטבלה החיצונית (ברמה הראשונה), ו-10 סיביות נוספות יפנו אותנו ל-PTE בטבלה ברמה השנייה. ואז נסיף את ההיסט ונגיע לכתובת הנכונה.

יכול להיות ערך בטבלה שיכיל NULL ← אין טבלה פנימית מתאימה ← חיסכון בזיכרון!
לדוגמה:

• נתונה הכתובת **000000001100000010001000000000111**

לכן:

1. 10 הסיביות השמאליות ביותר יפנו אותנו ל-PTE בטבלה ברמה הראשונה לכניסה מס' 3 – הרי **0000000011** = 3 בהמרה לעשרוני.
2. 10 הסיביות הבאות מצד שמאל יפנו אותנו ל-PTE (בטבלה שבכניסה 3) נצטרך לגשת לכניסה 17 – הרי **00000010001** = 17 בהמרה לעשרוני.
3. ניקח את מס' ה-frame שבכניסה 17 – נניח כי הוא מכיל את הערך 8. **[1000 בבינארי]**
4. **נשרשר** אליו את ההיסט שנותר מהכתובת למעלה – **0000000000111**
5. נגיע לכתובת הבאה: **10000000000000111** (משמאל יהיו רק 0'ים)

כאשר מדובר בכתובת בגודל 64 סיביות – מס' הרמות יהיה ממש לא מעשי. [למרבה הנס- גם מעבדי 64 סיביות לא מנצלים את 64 הסיביות].

מעבדי x86 ב-Linux:

נקבע כי מרחב הכתובות של תהליך הוא $2^{30} * 3$ בתים, וכן כי גודל דף הוא 2^{12} בתים. לכן, מרחב הכתובות יכול $2^{20} * 0.75$ דפים.

כמו כן, כל כניסה בטבלה מכילה 4 בתים $\leftarrow$ כל דף של טבלת הדפים יכול 2^{10} כניסות $(2^{12}/4) \leftarrow$ טבלת הדפים תתפוס $2^{10} * 0.75 = (0.75 * 2^{20}) / 2^{10}$ זה סה"כ 768 דפים!!!

הקצאת כזו כמות דפים לכל תהליך = בקשה לא מעשית!!

בנוסף – רוב בתהליכים לא משתמשים בכל מרחב הכתובות שלהם, אך גם משתמשים בכתובות המפוזרות ע"פ כל מרחב הכתובות.

הפתרון: נחזיק את מרחב הכתובות בעץ. תת עץ שאינו בשימוש – 'יגזם'. כך נרוויח: טבלה שאינה רציפה, וגם לא נצרך 768 דפים!

שורש העץ ייקרא **PGD – Page Global Directory**: יכול מצביע ל-4 כניסות כאשר 1 לא בשימוש.

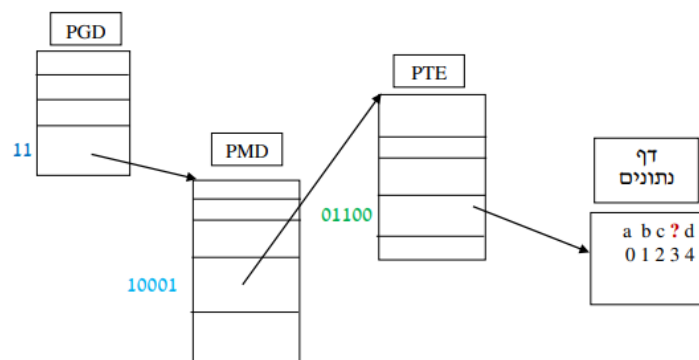
כל כניסה תצביע לטבלה מרמה אמצעית המכונה **PMD – Page Middle Directory**. כל טבלה כזו תכיל 512 כניסות. כל כניסה תכיל מצביע לטבלת דפים פשוטה.

הספרות באינדקס 21 עד 29 (כולל) מהכתובת – מכילות את מס' השורה אליה יש לפנות בטבלת ה-PMD. ואילו הספרות באינדקס 12 עד 20 (כולל) – מכילות את מס' השורה אליה יש לפנות בטבלה הפשוטה (הפנימית)

לדוגמה:

אינדקס:	11-0	20-12	29-21	31-30
כתובת:	00000000011	000001100	000010001	11
מפנה ל:	Offset	PTE	PMD	PGD

נגיע לתו '?' בתוך ה-Page.



כתובת אליה יש לפנות:

11 $\leftarrow$ PGD (30-31)
 000010001 $\leftarrow$ PMD (21-29)
 000001100 $\leftarrow$ PTE (12-20)
 000000000011 $\leftarrow$ Offset in frame (0-11)

הערות:

יש ארכיטקטורות בהן טבלת הדפים מכילה 2 רמות בלבד, ולא 3 כמו שראינו למעלה. כך שורש הטבלה (PGD) מכיל 2^{10} כניסות [נקבל את מס' הכניסה הרצוי ע"י 10 הסיביות המשמעותיות ביותר], ובתוכו כל כניסה מפנה ל-PTE רגיל – מכיל ג"כ 2^{10} כניסות [נקבל את מס' הכניסה הרצוי ע"י 10 הסיביות הבאות], ואילו 12 הסיביות שנותרו מימין יציינו את ה-Offset.

ואילו כאשר נצטרך להקצות זיכרון נוסף לתהליך – נשתדל להוסיף לדפים הקיימים, אחרת-נחפש כניסה פנויה בטבלת הדפים, ובה נוסיף מצביעים וצמתים בהתאם.

יתרונות בטבלאות דפים היררכיות:

+ חיסכון עצום בזיכרון, כאשר מקצים רק חלק ממרחב הזיכרון.

חסרונות בטבלאות דפים היררכיות:

- חיפוש מורכב, יותר מחיפוש בטבלה שטוחה
- יותר גישות לזיכרון בזמן החיפוש [כמס' הרמות בעץ]

כדי למנוע גישות מיותרות לזיכרון, נשתמש בטבלאות דפים הפוכות:

טבלת דפים מהופכת:

במקום להחזיק טבלת דפים לכל תהליך [מס' תהליכים גדול ← נפח טבלאות גדול] נחזיק טבלה יחידה שתקרא 'טבלת דפים מהופכת' – או אפילו בשם מתאים יותר – 'טבלת מסגרות מס' הכניסות יהיה כמס' ה-frame'ים הקיימים, ובכל כניסה נשמור את מס' התהליך ומס' ה-page. כדי למצוא page של תהליך מסוים – נסרוק סדרתית את הטבלה ע"מ למצוא כניסה המכילה את מס' התהליך הנוכחי, ומס' ה-page הרצוי.

כל כניסה תראה כך:

מס' תהליך , מס' page	מס' frame
----------------------	-----------

נוכל גם כאן לייעל את התהליך ע"י טבלת גיבוב, שפונקציית ה-hash שלה תקבל את מס' הדף וה-PID של התהליך.

נשים לב כי טבלת דפים מהופכת אינה מכילה מידע על ה-page'ים שלא קיימים בזיכרון. הפתרון: נחזיק טבלת דפים חיצונית [נוספת] לכל תהליך, בה נשמור את נתוני הדפים ששוחלפו, ואת כתובתם במרחב השחלוף.

מכיוון שנפנה לטבלה זו רק בעת page fault – נשתמש בה מעט, ולכן ניטה לשחלף אותה לדיסק. כך, בעת page fault נטען לזיכרון את טבלת הדפים החיצונית, ניקח ממנה את מיקום הדף במרחב השחלוף, ואז נטען את הדף הרצוי. בכך נשלם ב-2 page faults!

-?- מה לגבי עבודה עם דפים המשותפים למס' תהליכים?

!- כשנשתמש בדפים משותפים (לדוג' בכריכה דינאמית לתוכנית) ע"מ לשכן אותם פעם יחידה בכל הזיכרון.

כדי להצליח להריץ פונ' במקביל ממס' תהליכים – הקוד חייב להיות pure code (קוד טהור) או reentrant code (קוד כניסה חוזרת), ומקיים את הדרישות הבאות:

1. אין שינוי בקוד תוך כדי הריצה.
2. אין גישה למשתנים סטטיים או גלובליים מתוך הקוד.
3. אין השפעה ושינוי על המערכת הכוללת (כמו ע"י סמאפור)
4. אין זימון פנימי לקוד שאינו טהור.

בטבלת הדפים של כל התהליכים תהיה הצבעה ל-frame'ים בהן שוכן הקוד המשותף, ואילו כאשר הקוד מעדכן משתנים – השינוי יתבצע על ה-stack של התהליך הנוכחי.

הערה: אם נרצה להשתמש בטבלת דפים מהופכת – יהיה קושי לשמור ערכים בתוך הטבלה: איזה תהליך יישמר כערך המחזיק ב-page הרצוי? הרי יש כמה תהליכים...

טבלת דפים מגובבת:

בשיטה זו נזין את מס' הדף לפונ' $hash()$ והיא תחזיר לנו את ערך הכניסה לטבלת הגיבוב. כל כניסה בטבלת הגיבוב תכיל רשימה מקושרת, בה כל איבר מכיל את: מס' ה-page הלוגי, מס' ה-frame הפיזי, ומצביע לאיבר הבא ברשימה.

אנו נסרוק את הרשימה המקושרת באופן סדרתי ע"מ למצוא את הדף הרצוי.

מבחינת סיבוכיות המקום: נניח והטבלה תיכנס באוגרי ה-CPU, אך הרשימות המקושרות (סך כל האיברים = מס' ה-page'ים) נמצאות בזיכרון.

כדי לגשת לכתובת הרצויה נצטרך את פונ' ה- $hash()$ + סריקת הרשימה המתאימה באופן לינארי + גישה ל-frame.

כמובן שבטבלת גיבוב אנו מקווים שאורך הרשימות יהיה שווה למס' ההתנגשויות, וכמה שיותר קצר...

טבלת דפים מגובבת גלובלית:

הרעיון המרכזי דומה לטבלת דפים מגובבת כנ"ל, אך בעת תהיה טבלה אחת משותפת לכל התהליכים, וכן פונ' ה- $hash()$ תקבל 2 פרמטרים: מס' התהליך, ומס' הדף – ואז תוחזר לנו מס' הכניסה הרצויה. וכן איברי הרשימה המקושרת יכילו: מס' תהליך, מס' page, מס' frame, ומצביע לאיבר הבא.

יתרונות ב-paging:

1. אין פרגמנטציה חיצונית
2. גמישות בהקצאת זיכרון והזזת דפים (ללא relocation)
3. תמיכה בריבוי תהליכים
4. הגנה על זיכרונות התהליכים
5. שיתוף זיכרון בקלות

חסרונות ב-paging:

1. יש פרגמנטציה פנימית בשיעור של 0.5 דף בממוצע לתהליך.
2. בטעינת תהליך ששוחלף לדיסק בחזרה לזיכרון – זמן ה-context switch עולה, מכיוון שנצטרך לטעון גם את טבלת הדפים באוגר ה-PTBR (Page Table Base Register). אוגר זה שומר מצביע לטבלת התרגום של התהליך שרץ עכשיו.

אפשרות ב – Segmentation

גם בשיטה זו נחלק את התוכנית לחלקים הקרויים page'ים. אך גודלם של הדפים אינו תואם דווקא לתוכנית אלא מפורק באופן שרירותי.

כאן המטרה היא לחלק את התוכנית לפי חלקים **לוגיים** – סגמנטים – ולכל אחד יש מטרה הגיונית, ובעקבות כך גודלם שונה זה מזה.

נחלק את מרחב הכתובות הלוגי ל: שם (או מס') סגמנט + Offset בסגמנט זה.

מי מחלק את התוכנית לסגמנטים? המהדר בלבד!

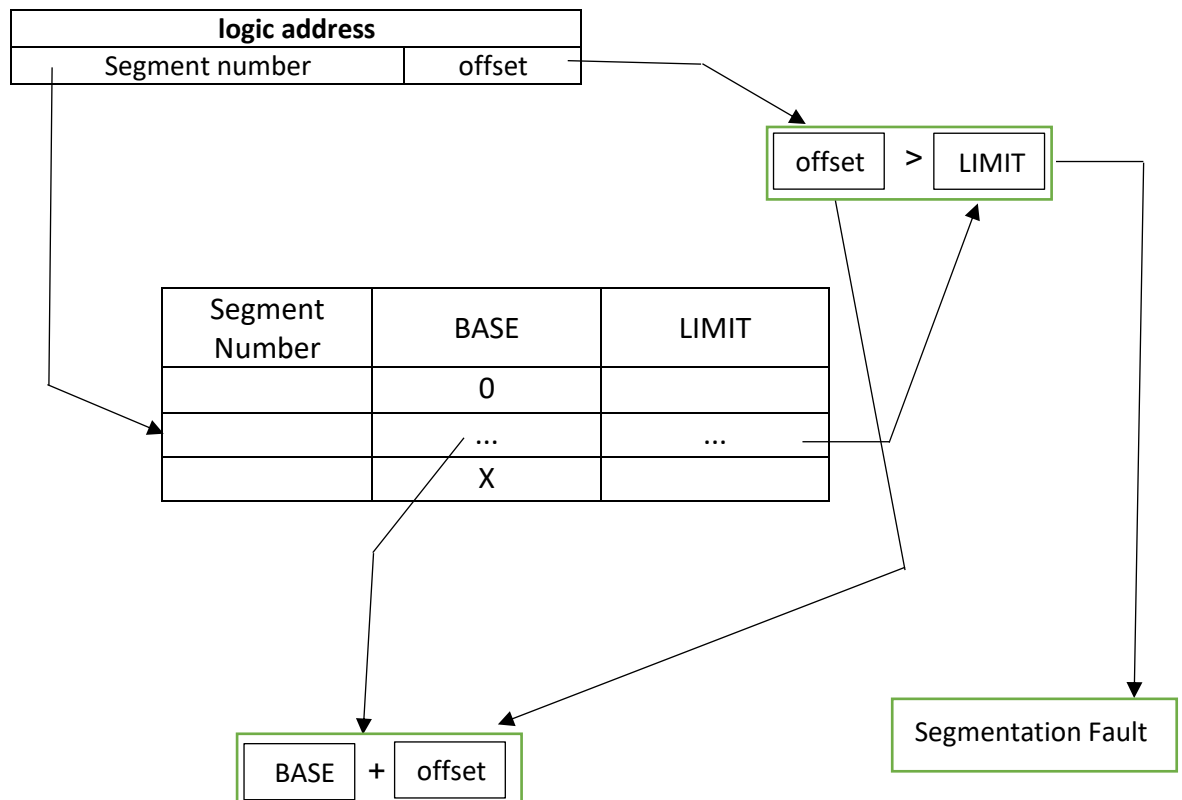
ה-CPU ינפיק כתובת המורכבת משם הסגמנט + היסט בסגמנט. נשווה את ההיסט לאורך הסגמנט – ואם היא תקינה נקבל את הכתובת הפיזית של הסגמנט.

שיטה זו מקלה עלינו למצוא חריגות (חריגה מחוץ לגבולות המערך = חריגה מהסגמנט).

מכיוון שהסגמנט הוא יחידה לוגית – נוכל להגן עליו באופן כוללני: הרשאות גישה, מצבי הרצה ועוד. שיתוף סגמנט פיזי יתבצע ע"י הכללתו בטבלת הסגמנטים של התהליכים המשותפים.

תהליך תרגום הכתובת הלוגית לכתובת פיזית יראה כך:

1. ניגש לטבלה במיקום של מס' הסגמנט
 2. נשווה את גודל ה-LIMIT וה-offset:
- א. אם ה-offset גדול $\leftarrow$ Segmentation Fault
- ב. אם הוא קטן: ניקח את ערך ה-BASE + ה-offset ונגיע למיקום הרצוי.



-?- מה הבעיה?

!- הסגמנטים תוארו כשטחי זיכרון רצופים – דומה לשיטת ההקצאה הרציפה, ונהיה חייבים לטפל בפרגמנטציה חיצונית וכדו'. אולי חומרת הבעיה תוקל מעט, כי הסגמנטים קטנים מגודלה של תוכנית, אך עדיין הבעיה עומדת בעינה.

מבחינת הזזת סגמנטים – ניתן לבצע ע"י העתקה ועדכון טבלת הסגמנטים.

הפתרון: בואו ניקח מכל שיטה את היתרון שבה!!

Segmentation VS Paging

Segmentation	Paging	
כל סגמנט באורך משתנה	כל הדפים בגודל קבוע	גודל:
אין תלות במיקום הסגמנט בזיכרון, ניתן לשנות דינאמית את גודל הסגמנט	כל הקטעים נמצאים ברציפות בטבלת הדפים	מיקום בזיכרון:
הקומפילר [המתכנת מודע]	החומרה	מי מחלק?
קל לקבוע הרשאה לכל סגמנט	קשה לקבוע הרשאות לכל דף	הרשאות:
ע"י שיתוף הסגמנט בטבלת הסגמנטים.	ע"י הצבעה בטבלת הדפים לכל frame.	שיתוף ספריות:

אפשרות ג – Segmentation + Paging

בשיטה זו נחלק את התוכנית לסגמנטים, וכדי לא להיתקל בבעיית הפרגמנטציה החיצונית – נחלק כל סגמנט ל-page'ים.

(הפירוט הבא מיושם במחשב IBM OS/2 במעבד x86)

תהליך מורכב ממקסימום 2^{14} סגמנטים, כאשר גודלו של כל סגמנט מכיל מקסימום 2^{32} בתים. [הזיכרון הלוגי בגודל 2^{46} גדול מהזיכרון הפיזי]

נחלק את הסגמנטים ל-2 קבוצות:

1. חצי מהסגמנטים יהיו מקומיים, ויכללו בתהליך הנוכחי בלבד. המידע עליהם נשמר בטבלה **LDT** – Local Descriptor Table.
2. חצי מהסגמנטים יהיו גלובליים, ויכללו להיכלל במס' תהליכים במקביל. המידע עליהם נשמר בטבלה **GDT** – Global Descriptor Table.

כל כניסה בטבלאות ה-LDT או ה-GDT תכיל את: כתובת הבסיס של הסגמנט, גודלו, ומידע נוסף על הסגמנט.

כל כתובת לוגית המונפקת מה-CPU תורכב מ-2 חלקים:

1. 16 סיביות המציינות את מס' הסגמנט:
הסיבית הראשונה מציינת האם הסגמנט גלובלי או מקומי.
13 סיביות נוספות יציינו את מס' הכניסה בטבלה המתאימה.
2 סיביות אחרונות יציינו האם הסגמנט שייך ל-user או ל-kernel.
2. 32 סיביות יציינו את ה-Offset בסגמנט עצמו.

כשנקבל כתובת נתרגם אותה פעמיים:

- ניגש לטבלה המתאימה לפי 2 הסיביות הראשונות, אח"כ לשורה המתאימה.
- נבדוק את גודל ההיסט לעומת גודל הסגמנט – במצב תקין ניקח את כתובת הבסיס של הסגמנט.
- נוסיף את ההיסט – ונקבל כתובת לינארית בת 32 סיביות
- נפנה עם כתובת זו לטבלת הדפים ההיררכית כמו שלמדנו: 10 סיביות לשורש, 10 סיביות לרמה הבאה, ו-12 סיביות להיסט.

Virtual Memory – הרחבה:

היתרונות בזיכרון מדומה:

1. אפשרות להריץ תוכנית שגדולה מהזיכרון הפיזי [קל וחומר מהגודל המוקצב לתוכנית]
2. הקצאת זיכרון קטן יותר לתוכנית מאשר בהקצאה רציפה ← העלאת רמת ה-multiprogramming ← הגדלת ניצולת הציוד.
3. פחות עבודה בשחלוף תהליכים.

זיכרון מדומה = הדמיה לתוכנית ששטח הזיכרון במחשב גדול ממה שהוא באמת.

זיכרון מדומה ממומש באחת מ-2 שיטות: **demand paging** או **prepaging**.

שיטת **Demand Paging** – דפדוף (טעינת דף) לפי דרישה.

טעינת תהליך לזיכרון מתבצעת:

- א. ע"י lazy swapper (משחלף עצל) – טוען רק את הדפים שחייב להשתמש בהם.
- ב. או ע"י pager (משדף) – טוען את הדפים רק לאחר page-fault.

בטבלת הדפים קיימות מס' אפשרויות:

1. הדף לא תקף – לא נמצא במרחב הכתובות
 2. הדף נמצא במרחב הכתובות אך עדיין לא הייתה לו דרישה (כעת עדיין בדיסק)
 3. הדף נמצא במרחב הכתובות וגם מצוי בזיכרון
 4. הדף נמצא במרחב הכתובות, הובא בעבר לזיכרון אך הוסר לשטח שחלוף מיוחד
- כדי לא לדרדר את מצב הביצועים נצטרך דיסק מהיר שנשחלף אליו וממנו דפים. המהירות מתאפשרת בזכות ניהול המאפשר למעלה פ לעבוד ביעילות.
- דיסק זה מכונה ב-Linux 'Swap Area' (ב-Windows זהו קובץ ייעודי).

חלופות למרחב השחלוף:

1. בתחילת הרצת התוכנית נטען את כולה למרחב השחלוף, וטעינת דפים לזיכרון תהיה מהירה יותר. המחר הוא: מרחב שחלוף גדול + העתקה בדיסק יקרה
2. לא נעתיק את התוכנית למרחב השחלוף, רק בעת פליטת דפים – נפלוט למרחב השחלוף.
3. כמו מס' 2, בהבדל ש-page'ים של code לא נפליטים, ובמידת הצורך – נדרוס את הדפים הללו בדפים חדשים. למרחב השחלוף נכתוב רק דפי ש'לולכלכו' – דפי `data \ heap \ stack \ bss \`.

עלות השימוש בשיטת Demand Paging

מהו זמן הגישה לכתובת שנמצאת בדיסק?

- מכיוון שהדף לא נמצא בזיכרון נשלחת פסיקה מה-CPU למעלה"פ.
- מעלה"פ משהה את התהליך, פונה לטפל בפסיקה, ומגלה שמדובר ב-page fault
- מאתרת את כתובת הדף בדיסק [במערכת הקבצים או במרחב השחלוף], ושולחת הוראה לבקר הדיסק לטעון את הדף לזיכרון
- מעלה"פ משהה את התהליך, מבצעת context switch.
- פעולת הקריאה ממתינה בתור לבקר הדיסק עד שיגיע זמנה לקבל שירות.
- נשקיע סה"כ 24ms לקריאת הדף:
 - זמן חיפוש – 15ms
 - זמן חביון – 8ms
 - זמן העברה – 1ms
- מסתמא בזמן הזה ה-CPU יעבור להריץ תהליך אחר.
- כשהבקר מסיים את קריאת הדף לזיכרון – הוא ישלח פסיקה
- מעלה"פ משהה את התהליך ועוברת לטפל בפסיקה, מגלה שזו פסיקת קריאה של הבקר ובעת מעדכנת את טבלת הדפים של התהליך, וגם מעבירה אותו לתור ה-ready. לכשישוב התהליך לרוץ – התהליך ישלים את הפקודה שרצה לבצע.

נוסיף לחישוב זמן הקריאה עוד 1ms לפחות – לכל יתר הפעולות המבוצעות בזיכרון [בלי להתחשב בזמן ההמתנה לצידוד!].

לכן זמן הגישה האפקטיבי לזיכרון יחושב בנוסחה: (כאשר p מציין את ההסתברות ל-page fault)

$$[(1 - p) * 100] + p * 25 * 10^6$$

Copy On Write – COW [העתקה בעקבות כתיבה]

בעת ביצוע `fork()` תהליך הבן מעתיק את ה-PCB של האב. אם הוא יבצע מיד ממש גם `exec()` – כל ההעתקה הייתה די לחינם, וגם אם לא – ניתן לומר שההעתקה לא הכרחית.

בשיטה זו – כשנוצר תהליך הבן נקצה לו PCB וטבלת דפים משלו, ובתחילה טבלאות הדפים (בעותקים נפרדים) של האב והבן יצביעו לאותם מספרי frame'ים. דפים העשויים להשתנות יסומנו בסימון מיוחד: אם וכאשר ישתנה הדף – שכפל אותו, ושנה את ההצבעה של התהליך שביצע זאת לעותק החדש. הסימון ישתנה למצב בו לא נשכפל את הדף שוב.

[משתמשים בשיטה זו ב-Linux וב-Windows]

החלפת דפים – Page Replacement

עד כה למדנו כי לכל page יכול להתבצע page fault פעם אחת.

אך למעשה, כשנריץ מס' תהליכים במקביל, וכ"א זקוק למס' דפים, כאשר ה-frame'ים מתחלקים ביניהם – מצב כזה נקרא over allocation (הקצאת יתר) – משום שלא כל התהליך נטען לזיכרון.

מה יקרה אם כל תהליך יצטרך יותר frame'ים כך שלא נוכל להריץ את כל 6 התהליכים במקביל?

- אופציה א': נעיף את אחד התהליכים. אך מדוע להפלות אחד מהם?
- אופציה ב': נשהה את אחד התהליכים, ונשחלף אותו לדיסק.
- אופציה ג': החלפת דפים

הרעיון בהחלפת דפים הוא: כאשר מעה"פ צריכה לטעון page חדש על frame, ואין אחד פנוי – המעה"פ גוזלת frame מהתהליך, ומעדכנת את טבלת הדפים בהתאם. המסגרת שפונתה מכונה victim frame – מסגרת קורבנית.

התוצאה: page fault עלול לגרום ל-2 פעולות $O(1)$ ← זמן הגישה האפקטיבי גבוה מאוד!!!

נוכל להקל על התהליך ב-2 אופנים:

1. מעה"פ תחזיק מלאי frame'ים פנויים – ובעת page fault תיקח משם frame פנוי.
 2. נשמור לכל frame סיבית dirty \ modify – שתאופס בטעינת ה-page ל-frame. בעדכון ה-page ← הסיבית תידלק.
- בפינוי ה-page נבדוק: אם הסיבית כבויה והדף כבר מצוי במרחב השחלוף – לא נפלוט את הדף לדיסק.

מסקנה: נצטרך אלגוריתם למימוש החלפת הדפים לבחירת frame קורבני. בהמשך גם נראה שצריך אלגוריתם להקצאת frame'ים לתהליך.

האלגוריתם יוכל לפעול בשיטת demand paging או בשיטת pre-paging.

כלל: בעת השוואת אלגוריתמי החלפת דפים והקצאת מסגרות – נפעל באופן הבא:

- א. נחליט על כמות מסגרות המוקצית לתהליך / לקבוצת תהליכים
- ב. נבחר references string (מחרוזת פניות) – סדרת פניות ל-page'ים.
- ג. נבדוק את כמות ה-page fault שנוצרה מכל אלגוריתם – ונעדיף את הקטן יותר.

אלגוריתמים להחלפת דפים

1. First In First Out – FIFO

בשיטה זו נשמור לכל דף את הזמן בו הוא נטען לזיכרון. כשנרצה לפנות דף – נבחר את הדף הישן ביותר.
ובצורה קלה יותר – בכל טעינת דף נוסיף אותו לתור, ובעת הסרה – נסיר מראש התור.
בעיה: יכול להיות דף שהגיע הראשון, אך הוא בשימוש גבוה, כך שלאחר הסרתו – נצטרך לטעון אותו שוב.
ליקוי: אנומליות בלאדי – אנו מצפים שעבור אלגוריתם X, ומחרוזת פניות כלשהי: אם נגדיל את מס' ה-frame'ים לתהליך ← מס' ה-page fault הכולל יקטן.
אך הציפיה לא תמיד מתממשת.
[תרגיל לדוגמה: 1,2,3,4,1,2,5,1,2,3,4,5 מס' מסגרות 3 ומס' מסגרות 4 ...]

2. Optimal Page Replacement – OPT – החלפת דפים מיטבית:

האלגוריתם מפנה את הדף בו יהיה שימוש בעתיד הרחוק ביותר.
אלגוריתם זה ממזער את כמות ה-page fault למינימום האפשרי, אך דורש יכולת לבנא את העתיד 😊. לא כ"כ נוכל להשתמש באלגוריתם זה, והוא משמש בעיקר להשוואה עם אלגוריתמים אחרים...

3. Last Recently Used – LRU

אלגוריתם זה מתבסס על עיקרון הלוקליות: דף שפינו אליו לאחרונה – נצטרך אותו בעתיד הקרוב, ואילו דף שלא פנו אליו לאחרונה – כנראה כבר לא נחוץ.
אלגוריתם זה מפנה את הדף שהכי הרבה זמן לא פנינו אליו.
ביצד האלגוריתם ממומש?

א. נצמיד למעבד mone שערכו יגדל ב-1 – בכל פעם שמתבצעת פניה לדף.
וכן נצמיד לכל דף את ערך ה-mone שלו, שיתעדכן בכל פניה.

השפעות:

- לבחירת הקורבן – נצטרך לסרוק את כל טבלת הדפים של התהליך, או אפילו את הטבלאות של כל התהליכים...
 - כל גישה לדף בזיכרון מחייבת גישה נוספת לזיכרון – לעדכון המונה!
 - נצטרך לדאוג לגלישה במונה
- ב.** נשתמש בתור – בכל פניה לדף נוסיף אותו לסוף התור, ואם הוא כבר היה בתור – יצא ממקומו הקודם. נפנה תמיד את הדף שבראש התור.
- ג.** ניצור מטריצת ביטים – נסמן לכל דף '1' בכל פעם שנפנה אליו, ונפנה את הדף בו יש הכי מעט '1'ים.

חסרונות: באופציות המימוש – האלגוריתמים יקרים, ומחייבים עבודה בכל פניה לזיכרון. וכן התמיכה בחומרה נמוכה.

לכן, מערכות הפעלה רבות ממשות אלגוריתמי קירוב ל-LRU, פחות יקרים.

אלגוריתמים אלו עושים שימוש בסיבית reference – המוצמדת לכל frame בזיכרון. בעת טעינת page ל-frame הסיבית **בבויה**. בעת פניה ל-page הסיבית **נדלקת** ע"י החומרה. מידי פעם מעה"פ

בודקת את ערך הסיבית ומכבה אותה. באופן זה מעה"פ יודעת האם הייתה פניה ל-page מהפעם האחרונה שהסיבית כובתה. לפני הכיבוי, מעה"פ מעתיקה את ערך הסיבית למבנה נתונים משלה, ולפי תקרב את LRU.

נזכור כי בשנגלה 2 סיביות דולקות – לא נוכל לדעת למי פנו קודם מבניהן, או כמה פעמים פנו לכל אחת.

4. קרוב ל-LRU :

Not Frequently Used – NFU – אלגוריתם מבוסס ספירה

ממומש ב-Linux: מעה"פ מחזיקה לכל page בית המשמש כ-mone. בכל סריקה של מעה"פ על סיביות ה-reference היא תגדיל את ערך ה-mone: אם הייתה פניה לדף – נגדיל, ואם לא – נקטין.

כשנצטרך לבחור דף לפינוי – נבחר את זה שערך ה-mone שלו הנמוך ביותר. בעיה: דף שהשתמשנו בו הרבה בהתחלה – התקבע עם ערך גבוה, ולא בהכרח שעדיין זקוקים לו...

[גם כאן – Linux מבצעת דרך מתוחכמת יותר]

5. קרוב ל-LRU :

שיפור NFU – Aging – אלגוריתם סיביות הפניה נוספות

שיטה זו דומה לשיטה הקודמת, אך בשינוי הבא: בכל סריקה של מעה"פ על סיביות ה-reference היא תסיט את תכולת הבית צעד ימינה, ועל הספרה המשמעותית ביותר – תועתק סיבית ה-reference הנוכחית.

לדוגמה: ערך הבית יהיה: 0101. ונתון שסיבית ה-reference דולקת.

כעת ערך הבית יהיה 1010.

אלגוריתם זה בוחר לפנות את הדף שערך הבית שלו – כמס' בינארי – הנמוך ביותר.

6. שיפור FIFO – אלגוריתם השעון / ההזדמנות הנוספת

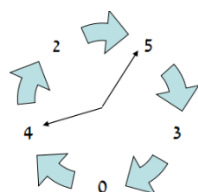
אלגוריתם זה מפנה דפים בשיטת FIFO, אך אם סיבית ה-reference שלו דולקת נבצע:

- העברה של הדף לסוף התור
- כיבוי סיבית ה-reference
- חיפוש קורבן חדש

כך דף שבא לידי שימוש בתדירות גבוהה – לא יפונה לעולם.

אלגוריתם זה מכונה אלגוריתם השעון מכיוון שניתן להתייחס לדפים כאילו הם מסודרים במעגל. כאן הכוונה שלעון בעל מחוג אחד.

שיפור 1: נשנה את האלגוריתם כך שיפעל עם שני מחוגים. המחוג הקדמי – יכבה את סיבית ה-reference, והמחוג השני יבדוק אם הייתה פניה לדף מאז שהמחוג הראשון כיבה את הסיבית שלו.



בכל שהמרחק בין המחוגים קטן יותר – יש לדף פחות סיכוי להינצל...

שיפור 2: האלגוריתם יתעניין מעתה גם בסיבית ה-dirty של הדף.

בסריקה הראשונה [של 2 המחוגים!] נחפש דף שגם ה-reference שלו כבוי, וגם ה-dirty שלו כבוי. אם לא נמצא דף כזה – נבצע סריקה שניה, ע"מ למצוא דף שה-reference שלו כבוי. אם לא נמצא גם דף כזה – נבצע סריקה שלישית ע"מ למצוא דף כלשהו שה-dirty שלו כבוי.

[אלגוריתם זה ממומש במחשבי mac]

ייעול האלגוריתם:

- נשמור מאגר של frame'ים פנויים.
- בעת טעינת דף חדש – נטעון ל-frame פנוי, ואח"כ נחפש קורבן, ונוסיף את ה-frame שלו למאגר הפנויים.
- נשמור רשימה של הדפים ש'התלכלכו'.
- כשנוסיף frame פנוי למאגר ה-frame'ים הפנויים – נזכור מאיפה הוא בא. כך שאם תהיה דרישה לדף שב-frame שפונה, **וטרם** טענו ל-frame זה דף אחר – נוכל לספק את הדף ללא ביצוע קריאה.

7. Distance String – מחרוזת המרחק

נדמה את המערכת למחסנית, ובאשר מגיע דף חדש – נכניס אותו לראש המחסנית. כשנגמר המקום במחסנית (הגענו למקסימום frame'ים ששייכים לתהליך) נצטרך לפנות דף. כיצד? נעביר אותו לתחתית המחסנית, מתחת לקו המסמן את מקסימום ה-frame'ים של התהליך.

$X = \text{מס' ה-frame'ים בתהליך.}$

נדע ש-X האיברים העליונים במחסנית קיימים ב-frame'ים של התהליך. מה שלא- גורם ל-page fault.

נייצר את ה-distance string: נשמור לכל דף ערך המסמן את מרחק הדף מראש המחסנית – בעמודה הקודמת. אם הוא לא קיים – נסמן את הערך אינסוף.

לפי המחרוזת שתיווצר – נדע כמה page faults ייווצרו [מס' הפניות שערכן במחרוזת גדול ממס' ה-frame'ים המקסימלי]. לדוגמה: נתון $X=4$

reference string	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	7	2	3
	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	7	2	3
		0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	7	2
			0	2	1	3	5	4	6	3	3	4	4	7	7	7	5	3	1	7
				0	2	1	3	5	4	6	6	6	6	4	4	4	7	5	3	1
					0	2	1	1	5	5	5	5	5	6	6	6	4	4	5	5
						0	2	2	1	1	1	1	1	1	1	1	6	6	4	4
							0	0	2	2	2	2	2	2	2	2	1	1	6	6
								0	0	0	0	0	0	0	0	0	2	2	1	1
																	0	0	2	2
																			0	0
distance string	P	P	P	P	P	P	P		P					P			P		P	
	∞	∞	∞	∞	∞	∞	∞	4	∞	4	2	3	1	5	1	2	6	4	8	4

SUM = distance_string[i] > X ? SUM++: SUM; :page faults-ה-למס' הנוסחה

דרוג האלגוריתמים להחלפת דפים

1. אלגוריתם אופטימלי – OPT
2. שיפור – Aging – NFU – אלגוריתם סיביות הפניה נוספות
3. LRU
4. Aging
5. Clock
6. Second Chance
7. NRU
8. Not Frequently Used – NFU – אלגוריתם מבוסס ספירה
9. LFU
10. MRU
11. FIFO

הקצאת frame'ים

נצטרך להקצות frame'ים לתהליך, תוך שמירה על מצב מאוזן.
אם נקצה מסגרות רבות מידי לתהליך – נצטרך זיכרון רב, ורמת ה-Multiprogramming תרד ←
ירידה בניצולת המערכת.
אם נקצה מעט מסגרות לכל תהליך – התהליך יבצע page faults רבים ויושהה פעמים רבות. ניצולת
הציוד תהיה גבוהה, אך ניצולת ה-CPU תהיה נמוכה.
לכן, נצטרך למצוא כמות מינימלית של מסגרות, המאפשרת לכל תהליך לרוץ ברווחה. כיצד?

הקצאה גלובלית VS הקצאה לוקלית

הקצאה לוקלית	הקצאה גלובלית	
נקצה frame מתוך מלאי ה-frame'ים של התהליך	נקצה ממאגר ה-frame'ים של כל התהליכים.	כיצד?
כמות ה-page faults תלויה בתהליך בלבד	כל תהליך מקבל frame'ים כפי צורכו. (נהנה ולא חסר)	יתרונות

רוב המעה"פ משתמשות בהקצאה הגלובלית.

ישנן 2 שיטות הקצאה עיקריות:

1. Fixed Allocation – הקצאה שווה: כל תהליך יקבל מס' קבוע של frame'ים לפי פרופורציה – ביחס מתאים לגודלו.
2. Priority Allocation – הקצאה לפי עדיפות: כל תהליך יקבל מס' frame'ים לפי עדיפותו.

אלגוריתמים להקצאת frame'ים

1. Page Frame Reclaiming Algorithm – PFRA – האלגוריתם של Linux:

מעה"פ מקצה זיכרון לתהליך בעת יצירת התהליך [תוך שמירה על COW], במיפוי קובץ לזיכרון [mmap()], בעת גדילת המחסנית, בעת הקצאת זיכרון דינאמית [מוסיף memory region], ובעת הקצאת זיכרון משותף [shmget()]. חשוב לדעת שבעת הקצאה דינאמית תזומן הפונ' sbrk() על גבי malloc(). לכל דף מעה"פ שומרת page descriptor המכיל: mone גישות לדף, הרשאת כתיבה, dirty, האם הדף נעול.

האלגוריתם מופעל מידי פעם, או כאשר מאגר המסגרות קטן. זהו קירוב של LRU. נחזיק 2 רשימות מקושרות דו כיווניות, המנוהלות כתור (FIFO), ומכילות דפי מרחבי המשתמשים, ודפי זיכרונות המטמון, אך לא דפי גרעין. רשימה א – active – מכילה את כל הדפים הפעילים, ובעצם מכילה את קבוצת העבודה של התהליכים.

רשימה ב – inactive – מכילה את הדפים המועמדים לשחלוף. כשנרצה לפנות דפים – נפנה את ראש רשימת ה-inactive בכל פניה לדף החומרה מדליקה את סיבית ה-reference ומעה"פ מחזיקה גם בית המציין את מידת השימוש בדף. האלגוריתם בודק את הסיבית ומידי פעם מכבה אותה. אם הסיבית דולקת – ה-mone גדל ב-1, ואם הסיבית כבויה – נחלק את ערך ה-mone ב-2. עבור דף פעיל – כאשר הסיבית דלוקה נעביר את הדף לסוף התור active כשערך ה-mone מתאפס – נעביר את הדף לסוף התור inactive. כדי לא להעביר סתם דפים – האלגוריתם יסרוק רק חלק מתור ה-active. דף המצוי בתור הדפים הלא פעילים, ויש אליו פניה: סיבית ה-reference שלו מודלקת, גילו הופך להיות 1, אך הוא אינו מועבר לתור הדפים הפעילים. רק אם תהיה אליו פניה שנייה, טרם שהסיבית כובתה גילו יהפוך להיות 2, הוא יעבור לתור ה-active, וגילו יאופס. השיטה נקראת LRU/2.

דפים מלוכלכים בתור ה-inactive 'מנוקים' כך שנקל יהיה לפנותם, ומועברים לזנב רשימת הלא פעילים.

כשנרצה לפנות דפים, גם נשקול את מקורם של הדפים: האם מתהליכי משתמשים או מזיכרונות מטמון.

באופן בסיסי ההעדפה היא לפנות דפי מטמון. רק אם ערכו של משתנה כלשהו גדול מסף המחושב לפי מצב המערכת יפוננו גם דפי תהליכי משתמש.

גם תהליך הפינוי (כמו התהליך המעביר דפים מתור הפעילים לתור הלא פעילים) סורק רק חלק מתור ה-inactive. גודל החלק שנסרק נקבע על-פי מצב מלאי המסגרות הפנויות.

תהליך שאיבד אחוז ניכר דיו מדפיו ישוחלף בשלמותו לדיסק.

בכל נקודת זמן, קיים תהליך יחיד במערכת האחוז באסימון השחלוף (token swap). תהליך זה, הרוכש דפים במרץ, ואוחז באסימון, מובטח שדפים לא יחטפו ממנו. באסימון ניתן לאחוז רק זמן מוגבל (אך לא מבוטל), רק אם כמות ה-page faults של התהליך גדולה, ושל מי שאחז בו קודם מועטה.

מטרת האסימון למנוע דישה - trashing.

כשנרצה לטעון דף לזיכרון – נטען גם את הסביבה שלו, מתוך תקווה שהדבר יחסוך page faults. כלומר: מעה"פ מבצעת prepaging.

הטענה היא שלינוקס עושה זאת באופן יחסית פרימיטיבי (ולא שאחרות עושות זאת מתוחכם יותר...). עדיף היה ללמוד את דפוס הגישה של התהליך לדפים ולפיו לבצע את הטעינה המקדימה.

כשמאגר המסגרות הפנויות מתדלדל בצורה חמורה מעה"פ הורגת תהליכים כדי לשקם את מלאי המסגרות.

תהליך יבחר בקורבן אם:

א. הוא וצאצאיו אוחזים במסגרות רבות.

ב. הוא אינו תהליך גרעין.

ג. הוא אינו פונה ישירות לציוד (ולכן עלול להותיר את הציוד במצב לא עקבי)

ד. הוא רץ יחסית מעט זמן.

ה. קדימותו נמוכה.

[ב-BSD: אם קצב ה-page faults עולה, או יש תהליך שממתין יותר מעשרים שניות, אזי התהליך הוותיק ביותר במערכת משוחלף כולו לדיסק, ומסגרותיו ממוחזרות.]

גזלת המסגרות הינה גלובלית (ולא פר תהליך).

דפי טקסט אינם משוחלפים למרחב השחלוף (רק דפי נתונים, או דפים משותפים משוחלפים). דפי קובץ שמופה לזיכרון 'מסונכרנים' עם הקובץ, (ולא משוחלפים). כלומר: הקובץ בדיסק מעודכן, כך שהוא יכול את הנתונים החדשים ששוננו בזיכרון.

מודול פינוי הדפים צובר מלאי של דפים שיש לפנות, ורק אז הדפים מפונים. זאת כדי לצמצם את כמות זמן החיפוש (זמן הזזת הראשים), וכדי לאפשר כתיבה של דפים סמוכים ממרחב הכתובות של תהליך על גושים סמוכים במרחב השחלוף.

2. אלג' מערכת הרעים – Buddy System Algorithm

לכאורה, כשמשתמשים בטבלת דפים – אין סיבה להקצות דפים ברציפות.
למעשה יש ויש:

a. בקרי DMA אינם מכירים את טבלאות הדפים, ומעבירים נתונים תוך שימוש בכתובות פיזיות.

b. הקצאה רציפה תאפשר טיפול בגושים גדולים יותר של נתונים

מסקנה: מעה"פ משתדלת לשמור דפים רציפים במסגרות רציפות בזיכרון.

השיטה: שימוש באלגוריתם מערכת הרעים הבא:

i מסגרות הזיכרון מקובצות לגושים של דפים, בחזקות 2. בכל גוש 2^i מסגרות רציפות (עבור $i=0...9$).

כניסה #i במערך (בגודל 10) מכילה רשימה מקושרת, שכל איבר מהרשימה מתייחס לגוש רציף בן 2^i מסגרות.

כל גוש בגודל 2^i יתחיל בהכרח במסגרת שמספרה הוא כפולה של 2^i .

על דרך השלילה: לא ייתכן גוש של 16 מסגרות, שכולל את המסגרות 40 עד 55. 16 מסגרות אלה תוחזקנה כשני גושים בני 8 מסגרות.

כשנצטרך להקצות גוש רציף בן 32 מסגרות נפעל באופן הבא:

א. נבדוק האם קיים איבר כלשהו ברשימה של הגושים בגודל 32 מסגרות.
אם יש: נבחר בו. אם אין:

ב. נבדוק האם קיים איבר כלשהו ברשימה של הגושים בגודל 64 מסגרות.
אם יש: נקצה 32 מסגרות מהאיבר, ואת ה-32 הנותרות נוסיף לרשימה הקודמת. אם אין:

ג. נבדוק האם קיים איבר כלשהו ברשימה של הגושים בגודל 128 מסגרות.
אם יש: נקצה 32 מסגרות מהאיבר, ואת ה-64 הנותרות נוסיף לרשימות המתאימות. וכך הלאה...

כשמשחרר גוש בן b מסגרות, נבדוק האם ניתן למזגו עם גוש משוחרר סמוך לו בן b מסגרות (כך שיתקבל גוש בן $2b$ מסגרות שמתחיל בכתובת המתאימה), ונוסיף את הגוש לרשימה המתאימה.

החיסרון: פרגמנטציה פנימית כאשר דפים מכילים מבני נתונים של הגרעין הקטנים ממס' דפים בחזקת 2...

הפתרון: Stub Algorithm

3. Stub Algorithm

לכל סוג אובייקט שמוקצה בתדירות רבה עבור הגרעין, מעה"פ מחזיקה זיכרון מטמון (ובו מס' frame'ים), בו נמצאים אובייקטים מסוג זה. כשנרצה להקצות אובייקט – נקצה מתוך המטמון, וכשנשחרר אובייקט – נוכל להשתמש שוב באותו אובייקט בהקצאה הבאה. ישנם גם זיכרונות מטמון לאובייקטים מטיפוסים פחות שכיחים, ובהם האלגוריתם יהיה דומה ל-Buddy System Algorithm כך נבזבז עד חצי מגודל הזיכרון.

4. תחומי זיכרון – Regions Memory

בלינוקס, כשמעה"פ מקצה מקטע של כתובות לוגיות (לינאריות) לתהליך היא אינה מקצה אותן ביחידות בודדות, אלא מקצה מקטעי זיכרון (regions memory). בד"כ לתהליך יהיו אך מספר מצומצם של מקטעי זיכרון, אולם קיימים תהליכים (מסד נתונים) אשר עשויים להיות מורכבים מאלפי מקטעי זיכרון. מקטעי הזיכרון של התהליך מקושרים ברשימה מקושרת ממוינת (לפי כתובת ההתחלה) דו-כיוונית, אל אבריה ניתן לפנות גם דרך עץ אדום-שחור. הרשימה והעץ הינם מרכיבים במתאר הזיכרון של התהליך (הכולל, בין היתר, מצביע לשורש טבלת הדפים שלו), שהינו חלק מה-Process Descriptor.

Trashing – דשדוש

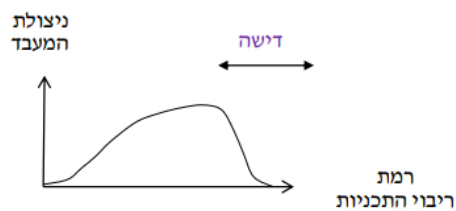
נניח כי תהליך מסוים זקוק ל- N מסגרות, והוקצו לו M מסגרות בפועל. $M < N$. $\leftarrow$ ייווצרו הרבה page faults.

ההגדרה ל-Trashing היא: כאשר תהליך מבלה יותר זמן ב-paging (ייבוא דפים בעקבות page fault) מאשר בריצה.

לדוגמה, תסריט ל-Trashing – כמפורט:

- המערכת מנטרת את ניצולת המעבד, ומבחינה שהניצולת נמוכה
- המתזמן לטווח ארוך מגדיל את רמת ה-Multi Programing
- התוכניות שרצות זקוקות למסגרות רבות, חוטפות זו מזו מסגרות וריצתם איטית
- המערכת מבחינה שניצולת המעבד נמוכה, והכל מהתחלה...

מסקנה: עלייה ברמת ה-Multi Programing עד רמה מסוימת – מגדילה את ניצולת המעבד, ומעבר לה – מקטינה אותה דרסטית.



נשים לב: באלגוריתם החלפה לוקלי – תהליך שנמצא ב-Trashing לא יכניס תהליכים אחרים למצב זה, אך הם נפגעים מכך שהוא מעמיס על ציוד ה-I/O, וזמן הגישה האפקטיבי לזיכרון גדל.

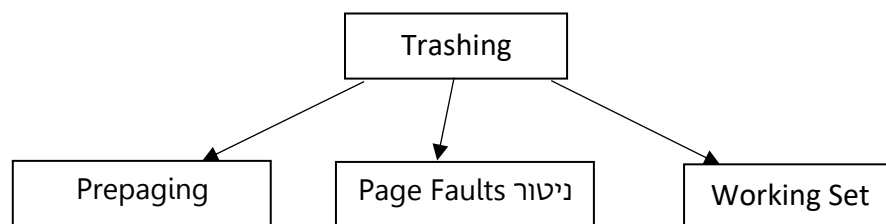
כדי למנוע Trashing נצטרך להקצות לכל תהליך כמות מספקת של מסגרות.

-? כיצד נדע מהי הכמות?

!- עיקרון הלוקליות. נגדיר את ה'לוקליות' של התוכנית' בתור **קבוצת דפים** שהתוכנית משתמשת בהם **בו זמנית**.

לכן אם נקצה לתהליך די מסגרות, כך שהלוקליות שלו תשכון בזיכרון בשלמות, נוכל למנוע Trashing.

פתרונות ל-Trashing :



-? כיצד נגלה את הלוקליות של התהליך?

!- באמצעות Working Set – מודל קבוצת העבודה או שיטת נוספות (שלא נלמד).

Working Set – מודל קבוצת העבודה

זוהי שיטה בה נגלה את הלוקליות של התהליך: נגדיר פרמטר הקרוי 'חלון קבוצת העבודה', ומסומן כ: Δ .

מידי פעם נבחן את Δ הפניות האחרונות לדפים שביצע התהליך, ונחשיב אותם לקבוצת העבודה. גודלה של קבוצת העבודה ייקבע להיות מס' הדפים אליהם הייתה פניה ב- Δ הפניות האחרונות. שימו לב: זוהי שיטה דיכוטומית (הכול או כלום) – רק אם התהליך לא יקבל את כל קבוצת הדפים שלו – הוא יושעה. כך כולם יסתיימו מהר יותר. כאשר נתונה קבוצת עבודה בגודל S . המערכת תחליט בהמשך אילו דפים יימצאו בזיכרון, תוך שימוש באלגוריתם החלפה כלשהו (אחד מהנ"ל).

לדוגמה, נתונה המחזורת הבאה: 2, 6, 1, 5, 7, 6, 7, 5, 8, 7, 8, 8, 8, 3, 4, 3, 4, 3, 4. נניח שלאחר כל 10 פניות נבחנת קבוצת העבודה, וכן ש: $\Delta = 6$.

- 10 הפניות הראשונות הן: 2, 6, 1, 5, 7, 6, 7, 5, 8, 7. ובסופן קבוצת העבודה תכיל את 7, 6, 5, 8
- עד שתיקבע קבוצת עבודה חדשה – לתהליך יהיו 4 מסגרות.
- 10 הפניות הבאות הן: 8, 8, 8, 8, 3, 4, 3, 4, 3, 4. ובסופן קבוצת העבודה תכיל את 3, 4 ולתהליך יהיו 2 מסגרות.

-?- כיצד נקבע את Δ ?

!- אם Δ קטן מידי – יש חשש שקבוצת העבודה לא תכיל את כל הלוקליות.

אם Δ גדול מידי – נקצה לתהליך מסגרות יותר מהנדרש.

נוכל לדעת שהמערכת מועדת ל-Trashing אם סך הגדלים של קבוצות העבודה (של התהליכים המורצים) גדול מכמות המסגרות שניתן להקצות להם. [אם כך – נשהה את אחד התהליכים, ונפלוט את דפיו לדיסק]

אלגוריתם לאיתור ה-Working Set:

ניצור תור ריק, ובכל פניה לזיכרון – נוסיף את הדף שפנינו אליו לסוף התור. התור מכיל רק Δ ערכים. מעבר לכך – נסיר מראש התור את הדף שיש בו. מידי פעם – כשנבחן את תכולת התור, נדע כי הערכים שבו מכילים את קבוצת העבודה.

חסרון: בכל פניה לזיכרון נצטרך לטפל בזיכרון $\leftarrow$ יקר!!

מסקנה: נצטרך למצוא רק קירוב של קבוצת העבודה.

אלגוריתם למציאת קירוב לקבוצת העבודה – שילוב עם Aging:

נניח כי לכל דף ישנה סיבית הפניה, וכן נחזיק מערך בוליאני לכל הדפים, המכונה: `working_set`. נאפס את כל סיביות ההפניה של הדפים וכן את המערך `working_set`. בכל פניה לדף – החומרה תדליק את סיבית ההפניה של הדף. כל X פניות לזיכרון – נבצע עבור כל דף:

א. הסטה של המערך תא אחד ימינה (`shift right`)

ב. נעתיק את סיבית ההפניה של הדף לתא המשמעותי ביותר (MSB)

אם באחד מ- N התאים המשמעותיים ביותר במערך של דף כלשהו קיים הערך `true` – אזי הדף נמצא בקבוצת העבודה שגודלה $(X * N)$. במילים אחרות: ב- $(X * N)$ הפניות האחרונות ניגשו לדף זה.

Page Faults ביטור

נעקוב אחר תכיפות ביצועי page faults.
נקבע סף עליון וסף תחתון ל-page faults.
אם מתבצעים page faults מעל הסף העליון שקבענו – נדע כי התהליך ב-Trashing ונקצה לו מסגרות נוספות.
אך אם כמות ה-page faults נמוכה מהסף התחתון שקבענו – נוכל לגזול ממנו מסגרות.
אם כמות ה-page faults של כל התהליכים גבוהה מהסף העליון – נצטרך להשהות תהליכים.

Preaging

ננסה לחזות אילו דפים יידרשו, ולהכניס אותם לזיכרון עוד בטרם ה-CPU פונה אליהם. לדוגמה: נוכל לשמור לכל תהליך את רשימת הדפים בה הוא משתמש בד"כ...
נוכל גם לעקוב אחר קבוצת העבודה לכל תהליך, לפי הרצות קודמות, ולטעון אותה לזיכרון בכל הרצה מחדש.

הקשר בין מבנה התוכנית ל-Page Faults

הנחות:

- התוכנית מגדירה מטריצה: $\text{int mat}[N][N];$
 - המערך נשמר בזיכרון לפי שורות.
 - כל שורה זקוקה לדף שלם $N \leftarrow$ דפים בזיכרון המכילים את תוכן המטריצה.
 - מטרת התוכנית כעת היא לאפס את המערך.
 - מוקצה frame אחד לתוכנית.
- אם נאפס את המערך לפי שורות – יתבצע page fault פעם אחת לכל שורה $\leftarrow$ סה"כ N פעמים page faults.
- אך אם נאפס את המערך לפי עמודות – אזי בכל פניה לתא, יתבצע page fault $\leftarrow$ סה"כ N^2 פעמים page faults !!!
- מסקנה: בכל שהתוכנית תשמור על הלוקליות שלה – נגרום לפחות page faults.
לדוגמה: מחסנית תגרום לפחות page faults מאשר טבלת גיבוב (שמטרתה לפזר את הערכים...)
נעדיף לכתוב פונקציות המזמנות זו את זו באותו הדף, ולהפריד את דפי הנתונים מדפי הקוד.

נעיצת חוצצים

נניח שתהליך P1 מבקש לבצע I/O לחוצץ שלו. הבקשה מועברת לבקר DMA וכמובן גם את כתובת החוצץ (= מספר ה-frame ומספר ה-page), ומס' הבתים שיש להעביר. עד שהבקר יסיים לבצע את עבודתו – התהליך P1 נחסם, וה-CPU עבור לביצוע תהליך אחר. כשהבקר יסיים את פעולתו – הוא ישלח פסיקה, שתגרום להעברת P1 ממצב blocked למצב ready. ייקח זמן עד שבקר ה-DMA יעבור לבצע את הבקשה הנ"ל, יש לו תור ארוך של משימות דומות לביצוע. נניח כי תהליך P2 זקוק ל-frame נוסף. האלגוריתם להחלפת דפים מוצא כי page מסוים של P1 לא היה בשימוש זמן רב, ומחליט לפנות אותו ולהקצות את המסגרת הזו לתהליך P2. לכשיתפנה בקר ה-DMA לבצע את הבקשה הנ"ל – הוא לא מודע שה-frame עבר לרשותו של P2, ובעצם ידרוס את הנתונים ש-P2 הביא לתוכו ← **מצב גרוע!**

הפתרונות:

1. נבצע I/O רק לחוצצים של מעה"פ ורק כשהתוכנית רצה בזיכרון – נעתיק את הנתונים למרחב הכתובות של התהליך.
- תקורה בהעתקה בתוך הזיכרון.
2. נעילת דף שמתבצע אליו או ממנו I/O.

בעיה דומה אך שונה, שנוכל גם בה להשתמש בפתרון מס' 2:

נניח כי תהליך X בעל עדיפות נמוכה ביצע Page Fault ולכן הוא blocked עד שתסתיים טעינת הדף המבוקש אל הזיכרון.

נניח כי בקר ה-DMA טען את הדף ל-frame והתהליך X חזר לתור ה-ready. אך מכיוון שעדיפותו נמוכה – הוא ימתין זמן רב בתור...

בינתיים תהליך Y רץ על ה-CPU וזקוק ל-frame נוסף. לכאורה, הדף P נראה מתאים לתנאי הפינוי (נקי, ולא פנו אליו זמן רב).

אולי אכן כדאי לפנות אותו? בכל מקרה תהליך X ימתין זמן רב בתור ה-ready.... אך לעומת זאת, לא חבל לפנות את הדף – עד שהוא כבר נטען?

לכן, לאחר טעינת הדף ועד שייעשה בו שימוש ראשוני – ננעל את הדף.

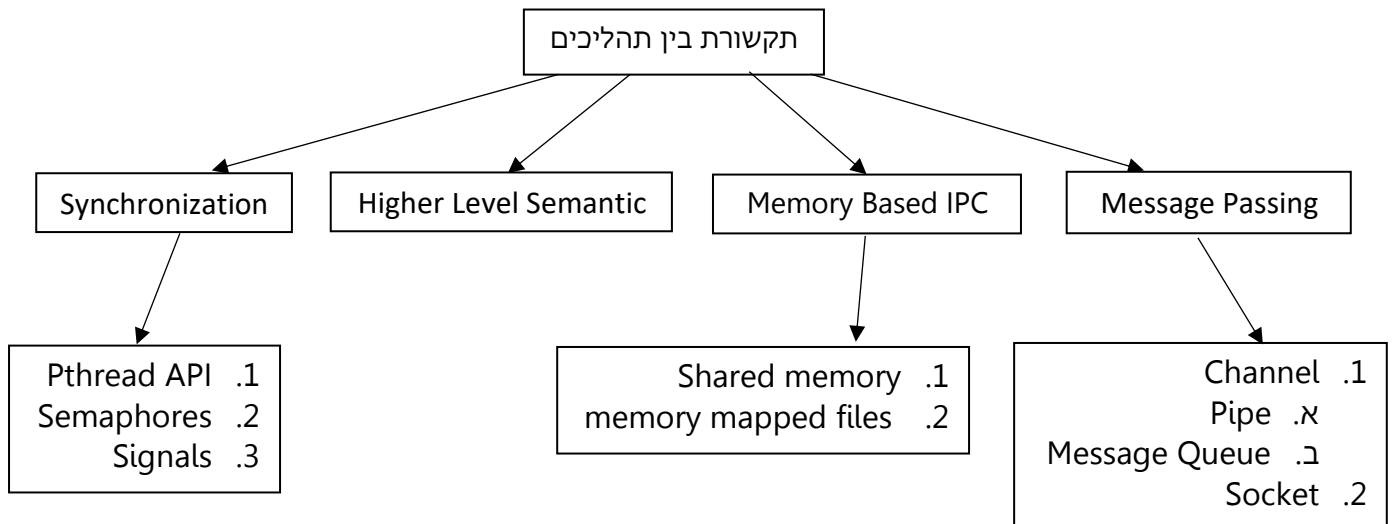
עלולה להיווצר תקלה בעקבות באג במעה"פ – שהדף לא ישתחרר לעולם.

אך נוכל לפתור זאת באופן הבא: הנעילה תהווה 'המלצה' למעה"פ (כך שמעה"פ תוכל לבחור להתעלם מההמלצה, אם: מלאי ה-frame'ים הפנויים מדולדל / התהליך נעל frame'ים רבים מידי / הדף נעול זמן רב).

IPC – תקשורת בין תהליכים

Inter process Communication – IPC

מכיוון שלכל תהליך יש מרחב כתובות שונה – אין לתהליך X רשות לגשת למידע או נתונים שמשותפים לו, כאשר הם שמורים במרחב הכתובות של תהליך Y. לכן מעה"פ מספקת מנגנונים לתקשורת, תיאום וסנכרון בין תהליכים.



Message Passing – העברת הודעות דרך ערוץ משותף.

1. Channel:

מעה"פ מגדירה את ה-channel (הערוץ) – יכול להיות buffer (חוצץ) / קובץ / Fifo Queue או מבנה נתונים אחר. גם לאחר ההקמה מעה"פ אחראית לתחזוקת הערוץ, ומספקת ממשק גישה, דרכו יוכלו התהליכים לגשת לערוץ. לכל תהליך מוגדר מס' port – אליו ישלח הודות / יקבל הודעות. כך כל שליחת הודעה מפעילה system call. יוצר תקורה גבוהה – העתקות מרובות: התהליך השולח יוצר system call שמעתיק את ההודעה ממרחב הכתובות של התהליך אל הערוץ, וכן התהליך המקבל יקרא את ההודעה ע"י system call – נעתיק את ההודעה מהערוץ למרחב הכתובות של התהליך.

מנגנוני מימוש ה-Channel:

א. Pipes:

צינור מאפשר לתהליך הורה לתקשר עם תהליך צאצא, ולעיתים מאפשר לצאצאים לתקשר ביניהם. זהו אפיק תקשורת חד כיווני המתנהל כמו תור, כך שכל תהליך יכול רק לקרוא או לכתוב בצינור, ופועל לפי עיקרון FIFO כמובן. ברגע שהתהליכים השותפים בתור מסתיימים – הצינור נעלם. כל צינור מורכב מ-2 מתארי קבצים (descriptor) שהם 2 כניסות במערך המתארים של התהליך: 1 לקריאה ו-1 לכתובה.

כדי לייצר צינור, נגדיר את המשתנים הבאים:

```
int pipe_descs[2];  
    נפעיל את הצינור ע"י הפונ' pipe() ונוודא שהצינור נוצר בהצלחה:  
if (pipe(pipe_descs) == -1) {  
    perror(...);  
    exit(...);  
}  
char buffer[N];  
int n_bytes;  
    כדי לקרוא תוכן מהצינור – נצטרך קודם לסגור אותו לכתיבה, ואז לקרוא:  
close(pipe_descs[1]);  
n_bytes = read(pipe_descs[0], buffer, N);  
close(pipe_descs[0]);
```

כדי לכתוב תוכן בצינור – נצטרך קודם לסגור אותו לקריאה, ואז לכתוב:

```
char letter[] = "hello every one";  
close(pipe_descs[0]);  
write(pipe_descs[1], letter, strlen(letter) + 1);  
close(pipe_descs[1]);
```

הפונ':	מבצעת:
pipe()	יצירת צינור – הקצאת descriptors והשתמשם במערך.
dup()	משכפלת את ערכו של התא הנשלח אליה בפרמטר, במערך המתארים.
read()	קוראת הודעה מהצינור (דרך pipe_descs[0]) • אם קיימת כמות הבתים הרצויה – מביא את כולה, ואם לא – אז את מה שיש. • אם הצינור ריק אך יש כותבים פעילים – התהליך נחסם עד שאחד מהם יסיים לכתוב. אם אין – יוחזר 0. • בכישלון יוחזר -1.
write()	כותבת הודעה לצינור (דרך pipe_descs[1]) • בכישלון יוחזר -1. • הכתיבה לצינור – סימטרית: אם אין מספיק מקום פנוי לכתיבה – התהליך נחסם. • הכתיבה חייבת להיות אטומית
close()	סוגרת את המתאר הרצוי.

הקשר בין Pipe לבין הפקודה 'b-UNIX:

כאשר התוכנית יוצרת פלט, היא מעבירה את הנתונים ל-stdout.

מצורף קטע קוד המדגים את ביצוע התהליך.

- הערה לגבי הקוד:
נוכל להחליף את 2 הפקודות – `close(STDIN_FILENO)` ואת הפקודה
`dup(pipe_desc[0])` – בפקודה אחת יחידה, המבצעת אותן יחד.
`dup2(pipe_desc[0], STDIN_FILENO);`

הערה:

- כדי ששני תהליכים יוכלו לשלוח מידע זה לזה יש להשתמש בשני צינורות: כל תהליך יכתוב לצינור אחד, ויקרא ממשנהו.
סידור כזה עלול להכניס אותנו ל-**deadlock** !! כיצד?
מיד בתחילת ריצתם שני התהליכים מנסים לקרוא, כל אחד מצינור הקריאה שלו.
התוצאה: שניהם נחסמים עד ש-'מישהו' יכתוב נתונים לצינור. רק ש-'מישהו' כזה אינו בנמצא.
באופן סימטרי: אם שניהם מנסים לכתוב, כל אחד על צינור הכתיבה שלו, יותר נתונים מכפי שהצינור יכול להכיל, אזי שניהם ייחסמו עד ש-'מישהו' יקרא נתונים מהצינור.

ב. תור FIFO / צינור בעל שם ('משויים') Named Pipe:

בניגוד ל-pipe רגיל שהוא אנונימי ומשמש לתקשורת רק בין תהליכים בעלי קשר של אב וצאצא – Named Pipe מאפשר תקשורת בין תהליכים שאינם כאלה.
תור הוא קובץ במערכת הקבצים, מסוג p.
תהליכים הרצים במקביל יכולים להעביר ביניהם נתונים, ע"י קריאה או כתיבה בקובץ (שהוא התור).

הפקודה ליצירת קובץ התור: **mkfifo selected_name**

כמו כן, קיימת פקודה בשפת C ליצירת קובץ תור:

```
mkfifo("selected_name", S_FIFO | 0644);
```

הפונקציה תחזיר 0 בהצלחה, ו-1 - בכישלון. [סיבה לכישלון: התור כבר קיים]

מצורף קוד המדגים העברת נתונים באמצעות תור כזה.

הערות:

- כל תהליך יכול לפתוח את התור רק למטרה אחת – קריאה או כתיבה.
- אם רק קורא או רק כותב פתח את הקובץ – הוא ייחסם, כי אין לו עם מי לתקשר...
- המידע מגיע רק אחרי שהכותב מבצע `fflush()`
- כתיבה על תור אחרי סגירתו ע"י הקורא גורמת לשליחת הסיגנל SIGPIPE לכותב, ובאופן מחדלי מביאה להעפתו.
- בעת פתיחת התור באמצעות `open` ניתן להוסיף את הדגל: `O_NONBLOCK`.
כותב שיוסיף דגל זה ייכשל בפתיחה אם לא ממתין לו קורא.

קורא שיוסיף דגל זה, וינסה לקרוא מהתור כשאין בתור נתונים, לא יושהה, אלא יקבל חזרה אפס בתיים (לא ניתן יהיה לדעת האם לא קיים כותב, או שקיים כותב והוא לא כתב דבר).

ג. Message Queue:

מקורם של תורי הודעות הוא מ-System V והם נקראים XSI IPC (X / Open System Interface) כאשר X / Open הוא סטנדרט Linux הדומה ל-POSIX. + תורי ההודעות גמישים ופחות גולמיים מתור או PIPE רגיל: הרי בתור המידע מועבר בזרם של בתיים – כך שמוטל על השולח ועל המקבל – לתאם את צורת הפירוש. לעומת זאת, בתור הודעות – כל הודעה היא struct בו יש מערך תווים, ומס' המציין את סוג ההודעה. זו גם הסיבה לכך שאין אכיפה לסדר קריאת ההודעות אצל המקבל, כלומר: הוא יכול לקרוא רק הודעות מסוג מסוים. [המס' 0 מציין רצון לקבל את כל ההודעות האפשריות] תור ההודעות מנוהל ע"י ה-kernel כרשימה מקושרת, ואיברים חדשים נוספים לסוף הרשימה.

בניגוד לכלי IPC אחרים, תורי הודעות אינם ממוחזרים כאשר התהליכים שהשתמשו בהם הסתיימו, אלא הם נותרים במערכת [וכמובן צורכים זיכרון] עד למחיקתם המפורשת [או עד reboot]. כמו כן, הטיפול בתורי הודעות אינו כמו טיפול בקבצים, ולכן לא נוכל להשתמש בכלי הקבצים אלא ב-system calls מיוחדים לתורי הודעות.

ברוב גרסאות Unix ניתן לייצר כמה עשרות תורי הודעות סה"כ.

כל תהליך יכול גם לקרוא וגם לכתוב מאותו תור הודעות.

לכל תור הודעות יש ID משלו המונפק ע"י מעה"פ, מס' טבעי בסדר מספרי עולה. ה-ID מקביל ל-Inode של מערכת הקבצים. בעת יצירת התור – זהו הערך המוחזר מפונקציית היצירה.

תהליכים המעוניינים לפנות לתור ההודעות יצטרכו להכיר את ה-ID של שאר התהליכים המשתמשים בתור.

-?- כיצד אכן יכירו בהם?

לכל תור קיים מפתח חיצוני המועבר למעה"פ בעת ייצור התור, וכל התהליכים המשתמשים בתור צריכים להכיר אותו. המפתח החיצוני דומה לשם של קובץ. כך: המפתח החיצוני [המקביל לשם הקובץ] ייקשר למפתח הפנימי [ID הקובץ]. ובעת ייצור תור / התקשרות לתור קיים – יסופק המפתח החיצוני, מעה"פ תיצור מפתח פנימי ובשליחת וקבלת ההודעות – נשתמש במפתח הפנימי.

-!- ישנן מס' אפשרויות:

א. המייצר (בד"כ השרת הפנימי) ישמור את המפתח החיצוני בקובץ כלשהו

חיסרון: מחייב שימוש בקבצים

ב. היצרן וכל מי שרוצה להשתמש במפתח החיצוני – יסכמו ביניהם מראש על מפתח חיצוני אותו יספק היצרן לפקודת ייצור התור.

חיסרון: אם מישהו ייצר תור עם ערך המפתח שסוכם – פקודת הייצור של היצרן תיכשל.

ג. היצרן וכל מי שרוצה להשתמש במפתח החיצוני – יסכמו ביניהם מראש על קובץ ממערכת הקבצים וגם על תו מסוים שיציין את הפרויקט, ויפעלו באופן הבא:

- נזמן את הפונ' ftok() עם שם הקובץ הנ"ל והתו שסוכם – והיא תנפיק עבורם את המפתח. כך נקטין את הסיכון לשימוש חוזר במפתח קיים.


```

key_t key; // יצירת המפתח
if ((key = ftok("~/user/file", 'c')) == -1) { // בדיקה האם הייצור הצליח
    perror("ftok() failed");
    exit(EXIT_FAILURE);
}

```

```

int msgid; // כדי לייצר תור חדש או להתחבר לתור קיים נגדיר:
// נשלח כפרמטר את המפתח, האם ליצור, להיכשל אם התור קיים והרשאות:
if ((msgid = msgget(key, IPC_CREAT | IPC_EXCL | 0600)) == -1) {
    perror(...);
    exit(...);
}

```

כאשר לקוחות מעוניינים להיקשר לתור הקיים במערכת – נבצע:

```

msgid = msgget(key, 0)

```

בשליחת הודעות בתור: נגדיר את סוג ההודעה (מס' שלם חיובי), ותוכן ההודעה (במערך תווים). ההודעה נאזרת במבנה שנגדיר בשרת ובלקוחות:

```

struct my_msgbuf {
    long mtype;
    char mtext[MAX_TEXT];
};

```

אופן שליחת ההודעה:

```

struct my_msgbuf my_msg;
my_msg.mtype = 17;
scanf("%s", my_msg.mtext);
status = msgsnd(msgid, & my_msg, strlen(my_msg.mtext)+1, 0);

```

הפרמטר האחרון מציין את מס' הבתים בהודעה. 0 – משמע: ההודעה תישלח כשיהיה מקום בתור, ובינתיים השולח ייחסם. [יחזור 1- בכישלון, 0 בהצלחה]

אופן קבלת ההודעה:

```

// מקבל ההודעה יכול להגדיר את סוגי ההודעות שיקבל כך:
long int allowed_type;
// כאשר: 0 – כל הודעה שהיא, ערך X – הודעות ממס' X בלבד.
status = msgrcv(msgid, & my_msg, MAX_TEXT, allowed_type, 0);

```

בפרמטר האחרון נוכל לכתוב את אחד מהדגלים הבאים:

- IPC_NOWAIT – המורה לא להמתין להודעה, אלא אם לא קיימת הודעה – לחזור עם כישלון.

- IPC_EXCEPT – המורה שאם סוג ההודעות שציינו גדול מ-0 – נקבל כל הודעה שאינה מהסוג שציינו.
- MSG_NOERROR – המורה שאם יש בתור הודעה גדולה מידי – לא לחזור עם כישלון, אלא לקצץ מגודלה.
- נוכל לשים את הערך 0 שיסמן כי איננו מעוניינים באף אחד מהדגלים הנ"ל.

אופן שימושי אופייני בתורי הודעות:

השולח יוצר סדרות של הודעות, הקוראים קוראים אותן עד לכישלון (-1) או כשעובר X זמן – לפי הגדרת הקוראים. אז תור ההודעות נמחק ע"י הפונ':

`msgctl(msgid, IPC_RMID, NULL);`

נזכור כי תור שלא נמחק – ממשיך להתקיים!!

מבצעת:	הפונ':
יוצרת תור הודעות חדש	<code>msgget()</code>
שולחת הודעה לתור ההודעות	<code>msgsnd()</code>
קוראת הודעה מתור ההודעות	<code>msgrcv()</code>
מוחקת את תור ההודעות	<code>msgctl()</code>

נוכל לצפות בכל תורי ההודעות, ה-Shared Memories, והסמאפורים הקיימים במערכת, באמצעות הפקודה: `ipcs`

נוכל למחוק תור דרך ה-shell באמצעות הפקודה: `ipcrm -q <msg_id>`

2. Sockets:

כשנרצה להעביר מידע בין תהליכים השוכנים במכונות שונות – נוכל להשתמש ב'ממשק התושבת' – ממשק פרוטוקולי תקשורת. ה-socket הוא API ליצירת תוכנות רשת.

רקע בתקשורת:

כתובת IPv4 – מורכבת מ-32 סיביות ומשמשת ככתובת לוגית לציוד כלשהו ברשת האינטרנט, וכמובן שהיא ייחודית לכל רכיב.
כתובת IPv6 – מורכבת מ-128 סיביות, ומיוצגת בבסיס הקסא-דצימלי.
Port – ערוץ לוגי, שתוכנות רשאיות להעביר בו מידע באופן ישיר. קיימים מספרי פורטים שהם 'מוכרים' – מוגדרים כסטנדרט עולמי. לדוגמה: פורט 80 שייך ל-HTTP.
כל מחשב יזוהה ע"י כתובת IP ואילו התהליך שבו יזוהה ע"י מס' port.

תהליך העברת המידע דרך Socket:

- (1) מתבצע זיהוי של מחשבי הקצה – כתובות IP וסוג פרוטוקול תקשורת
- (2) הרשת מעבירה את ההודעה למחשב היעד [לפי ה-IP]
- (3) מעה"פ מעבירה את המידע ל-socket [לפי הפרוטוקול + מס' ה-port]
- (4) האפליקציה קוראת את המידע מה-socket.

נבדיל בין 3 סוגי 'תושבת':

א. תושבת זרם – Stream Socket:

מאפשרת העברת נתונים רציפה לאורך זמן, תוך שימוש בפרוטוקול TCP המבטיח סדר ותקינות מידע, בין היתר ע"י Triple-handshake. תושבת זו דומה לשיחת טלפון פשוטה, והמידע מועבר כזרם של בתים. החלוקה ל-packets (פקטות) – שקופה למשתמשים.

ב. תושבת דטהגרם – Datagram Socket:

משמשת להעברת חבילת מידע בודדת, ודומה לשליחת מידע בדואר. משתמשת בפרוטוקול UDP ודומיו.

ג. תושבת גולמית – Raw Socket:

משמשת לפניה לפרוטוקולים נמוכים, האפליקציה כותבת את הפרוטוקול. יש בזה סיכון מבחינת האבטחה – עוקפים מנגנוני ביטחון בפרוטוקולים קיימים.

אנו נתמקד בסוג הראשון – תושבת זרם, כאמצעי לתקשורת בין תהליכים. וכן נעסוק בעיקר במודל Server – Client (שרת-לקוח), כאשר השרת מורץ לנצח, ואילו הלקוח 'מזדמן'... הם ישתמשו ב-System Calls כדי לתקשר ביניהם. נכלול את הספריות:

```
#include <sys/types.h>
#include <sys/socket.h>
```

הלקוח ישתמש בפונקציות: create(), connect(), read(), write(), close().
השרת ישתמש בפונקציות: create(), bind(), listen(), accept(), read(), write(), close().

תהליך התקשורת ייראה כך:

Client side:

```
sd = socket()
connect(sd, dst)
write(sd, ...)
read(sd, ...)
close(sd)
```

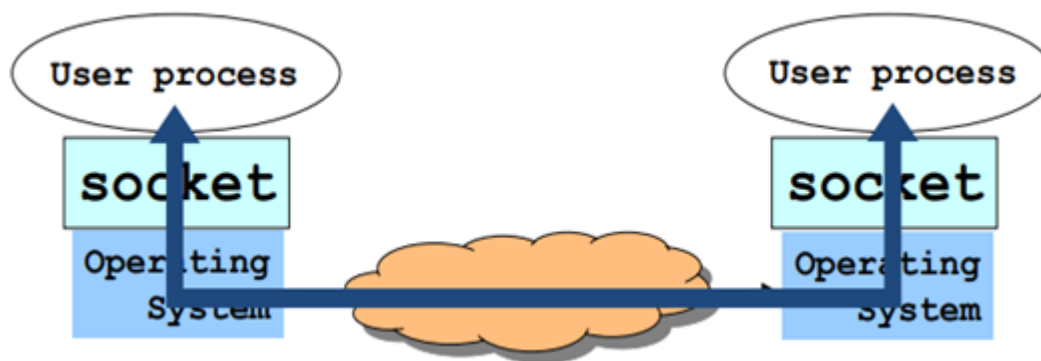
Server side:

```
sd = socket()
bind(sd, port)
listen(sd,...)
new_sd = accept(sd)
write(new_sd, ...)
read(new_sd, ...)
close(new_sd)
```

מבצעת:	מי מבצע?	הפונ':
יוצרת socket חדש. מקבלת: • ארכיטקטורת רשת • מודל תקשורת • סוג פרוטוקול לדוגמה: <pre>int my_socket = socket(AF_INET, SOCK_STREAM, 0);</pre>	שרת / לקוח	socket()
קריאת מידע מה-socket. מקבלת: • מצביע ל-socket שקורא • מצביע ל-buffer • גודל ה-buffer פונ' דומה לה נקראת recv()		read()
כתיבת מידע ב-socket. שונה מה- system call הרגילה write(). מקבלת: • מצביע ל-socket שכותב • מצביע ל-buffer • גודל ה-buffer פונ' דומה לה נקראת send()		write()
סגירת התקשורת עם ה-socket המרוחק. מקבלת: • מצביע ל-socket שנסגר		close()
חיבור בין ה-socket ל-port. מקבלת: • מצביע ל-socket. • כתובת מלאה: IP של המחשב + מס' port • מס' בתים כולל של הכתובת המלאה.	שרת	bind()
מגדירה את אורך תור בקשות בהמתנה. מקבלת: • מצביע ל-socket • מס' מקסימלי של בקשות בהמתנה.		listen()
קבלת בקשה מלקוח. מקבלת: • מצביע ל-socket • כתובת שולח הבקשה • אורך הכתובת בביתים		accept()
ניסיון להתחבר לשרת. מקבלת: • מצביע ל-socket • כתובת של השרת • מס' בתים של כתובת השרת	לקוח	connect()

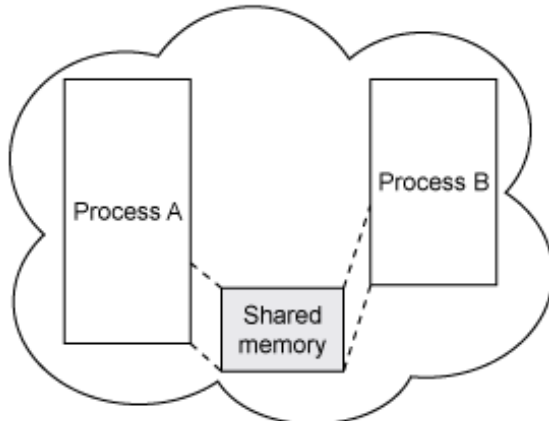
נוכל ייצר מצב בו שרת אחד מטפל בכמה לקוחות באופן מקבילי ב: Event Driven Model
 כמובן שהשרת ישתמש בפעולות מותאמות.

מבצעת:	מי מבצע?	הפונ':
עוברת על ערוצי ה-I/O הפתוחים, ומחזירה את הפנוי מבניהם. מקבלת:	שרת	select()
• כמות ערוצי ה-I/O • מצביעים לערוצי in, out, err • הגבלת זמן		
קריאת מידע מה-socket. לא תגרום לחסימה כשתקרא ל-accept() או ל-recv()	שרת / לקוח	readfds()



1. Shared Memory:

זיכרון משותף מאפשר לתהליכים שחולקים אותו:
 + לפנות לאותו קטע זיכרון בגישה ישירה – לקרוא או לכתוב על תא שברצונם.
 + **מהירות** קריאה / כתיבה – הפעולה לא עוברת תיווך של מערכת ההפעלה.
 האתגר נמצא בסנכרון הגישה לזיכרון המשותף: בד"כ נסנכרן באמצעות סמאפור או mutex כפי שלמדנו – לכל תהליך יש address space נפרד, ונחסמת הגישה לזיכרון שאינו של התהליך עצמו. לכן נצטרך לבצע פעולות להגדרת שטח מסוים כשטח המשותף ליותר מתהליך אחד.



תהליך היצירה מכיל:

- יצירת מפתח משותף [כמו המפתח ב-'message queue']

```
key_t key; // יצירת המפתח
if ((key = ftok("~/user/file", 'y')) == -1) { // בדיקה האם הייצור הצליח
    perror("ftok() failed");
    exit(EXIT_FAILURE);
}
```

- נגדיר יצירת שטח זיכרון משותף לפי המפתח:

```
[shread memory get ← shmget()]
int shared_mem_id;
shared_mem_id = shmget(key, SHM_SIZE, IPC_CREAT | IPC_EXCL | 0600);
if (shared_mem_id == -1) {
    perror(...);
    exit(...);
}
```

- תהליך שירצה להשתמש בשטח המשותף שהוקצה יבצע:

- הקצאת מפתח כמו שהובא בנקודה הראשונה
- הקצאת מצביע לזיכרון המשותף:

```
shared_mem_id = shmget(key, 0, 0600);
if (shared_mem_id == -1) {
    perror(...);
    exit(...);
}
```

כך מערכת ההפעלה הקצתה שטח משותף בזיכרון, ונתנה הרשאת גישה לתהליכים שביצעו את הפונק' `shmget()`. תהליכים שלא ביצעו אותה – לא שילבו עדיין את שטח הזיכרון המשותף ב-`address space` שלהם.

- כל תהליך הרוצה לשלב את הזיכרון המשותף בתחום הכתובות שלו יבצע:
[פעולת השיתוף נקראת `shmat()` ← `shared memory at`
הפרמטר השני מראה כי ניתן לשלב את קטע הזיכרון המשותף בכל מקום פנוי ב-`address space` של התהליך.
הפרמטר האחרון מראה שנבצע קריאה וכתיבה בזיכרון המשותף – בעזרת המצביע הזה. אם נרצה לבצע קריאה בלבד – נשלח `SHM_RDONLY`
`char* shared_mem_ptr = (char*) shmat(shared_mem_id, NULL, 0);`
`if (shared_mem_ptr == (char*) -1) {`
 `perror(...);`
 `exit(...);`
`}`

מעתה – נוכל לגשת לזיכרון המשותף כמו גישה פשוטה למערך!

- בשנרצה לנתק את שטח הזיכרון המשותף מהתהליך – ננתק את המצביע:
[הפונ' `shmdt()` ← `shared memory delete` עדיין לא שחררנו את שטח הזיכרון!!]

```
shmdt(shared_mem_ptr);
```

כדי לשחרר את שטח הזיכרון עצמו נבצע:

```
if (shmctl(shared_mem_id, IPC_RMID, NULL) == -1) {  
    perror(...);  
    exit(...);  
}
```

שטח הזיכרון ימוחזר לאחר שכל התהליכים שמשתמשים בו – הסתיימו.

זיכרון משותף שלא בוצע עבורו שחרור – ימשיך להתקיים: כך ייגזלו משאבים מהמערכת, וכן בניסיון הרצה נוסף של התוכנית – ההקצאה תיכשל. (כמו ב-`message queue`)

נוכל למחוק זיכרון משותף דרך ה-`shell` באמצעות הפקודה: `ipcrm -m <msg_id>`

מצורפת תוכנית לדוגמה:

תהליך 1 יוצר מחרוזות (קלט מהמשתמש) ומכניס אותן לזיכרון המשותף. תהליך 2 קוראת (מדפיסה) את תוכן המחרוזות הנ"ל.

בתוכנית זו – הסנכרון מתבצע ע"י ערכו של התא הראשון בזיכרון המשותף:

- כאשר ערכו הוא '-' ← אין מחרוזת בזיכרון המשותף
- כאשר ערכו הוא '+' ← יש מחרוזת בזיכרון המשותף

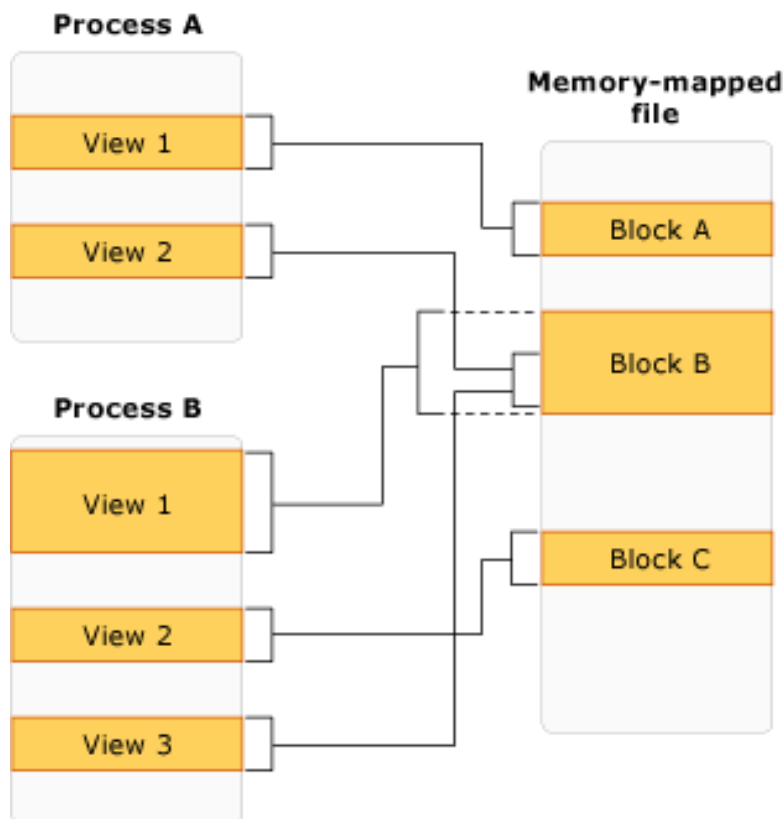
עדיף לבצע סנכרון ע"י סמאפורים או `mutex`!

הערות לגבי Shared Memory:

- א. לא ניתן להגדיל דינאמית את גודלו של הזיכרון המשותף!
- ב. נוכל להגדיר מבנה ספציפי ולהקצות זיכרון משותף שיכיל מערך של המבנים הללו.
`struct my_struct { ... };`
`struct my_struct* arr_structs = (struct my_struct*) shmat(...);`
- ג. כשנחזיק מבנה נתונים בזיכרון משותף – חובה לשמור את כל המידע עלו בשטח הזיכרון המשותף!

2. Memory Based Files

בשיטה זו תהליך עשוי למפות קטע ב-address space שלו – לקטע מתוך קובץ X. כך שבעת פניה לכתובת (ע"י מצביע) – ניגש לקובץ.
כשניגש בפעם הראשונה לדף בכתובת שבקטע הזיכרון הממופה – ניגש לחלק מסוים בקובץ ויתבצע ניהול page fault ← שיטען את החלק הרצוי ל-frame בזיכרון. פעולות I/O נוספות – יתבצעו בזיכרון הראשי כפעולות השמה [ללא חיוב פניה לדיסק].
נטפל בקובץ כמו טיפול במשתנה רגיל.
הקובץ שעל הדיסק יעודכן מידי פעם: לפי הנחיית המתכנת או בסיום המיפוי.
לכן, מספר תהליכים שרצים במקביל יכולים למפות את הקובץ למרחב הכתובות שלהם, כך שכל שינוי שיבצע אחד מהם – יופיע אוטומטית אצל כולם.



Higher Level Semantic – רמת תקשורת גבוהה

לא בחומר הנלמד.

Synchronization – סנכרון מידע שמועבר ביניהם

Pthread API

נוכל להשתמש בממשק pthread API לביצוע סנכרון בין תהליכים ו/או תהליכונים כפי שלמדנו בפרק 'מקבול וסנכרון'.

Semaphores

נוכל להשתמש בספריה semaphore.h לביצוע סנכרון בין תהליכים ע"י סמאפורים – כפי שלמדנו בפרק 'מקבול וסנכרון'.

Signals

סיגנל הוא כלי מאוד קדום ובסיסי להעברת מידע בין תהליכים, וכיום ממעטים להשתמש בו. נשתמש בסיגנלים בעיקר כאשר מעה"פ מאותתת לתהליך על שגיאה שביצע / אירוע שקשור אליו – כפי שלמדנו בפרק 'סיגנלים – Signals'.

מנגנון:	שיטה:	הגבלת תקשורת:
pipe	סדרתית	רק לתהליכים בעלי קשר של אב וצאצא, תקשורת חד-כיוונית
Named Pipe	סדרתית	אין הגבלה על הקשר בין התהליכים
Message queue	סדרתית	הכרות של ה-ID ע"י מפתח חיצוני.
Socket	סדרתית לפי מס' port	אין הגבלה על הקשר בין התהליכים
Shared Memory	ישירה	אין הגבלה על הקשר בין התהליכים
memory mapped files	ישירה	אין הגבלה על הקשר בין התהליכים
Pthread API	מבוסס על סנכרון והמתנה	
Semaphores		אין הגבלה על הקשר בין התהליכים
Signals		

מבנה מערכת האחסון המסיבי – Mass-Storage Structure

המשאב המרכזי לשמירת נתונים שאינם נדיפים (volatile non) במחשבים בימינו הוא הדיסק המגנטי (magnetic disk).

הדיסק מורכב ממספר דסקות/תקליטים (platters), אלו פלטות שטוחות מאלומיניום / זכוכית, מצופות חומר מגנטי, ומחוברות לציר אחד. טבעת רווח צרה חובקת את הציר בין הדסקות ע"מ לשמור על רווח קבוע ביניהן.), כ"א בצורת תקליט/CD בקוטר 1.8 - 5.25 אינץ'.

שני צדי הדיסקה = שני המשטחים מכוסים חומר מגנטי שקוטביותו מאחסנת את המידע.

בכל צד של כל דסקה, מעל כל משטח מותקן ראש קריאה כתיבה, (הראש מורכב משני רכיבים העשויים מסליל מתכת זעיר: 1 לקריאה ו-1 לכתיבה).

כל הראשים מתחברים לזרוע (arm disk) אחת, ולכן כולם נעים במקביל. תנועת הזרוע נשלטת ע"י אלקטרומגנט, וזמן הנעת הראש באמצעות הזרוע חשוב לזמן החיפוש.

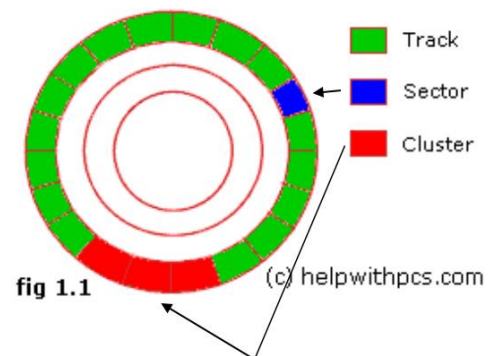
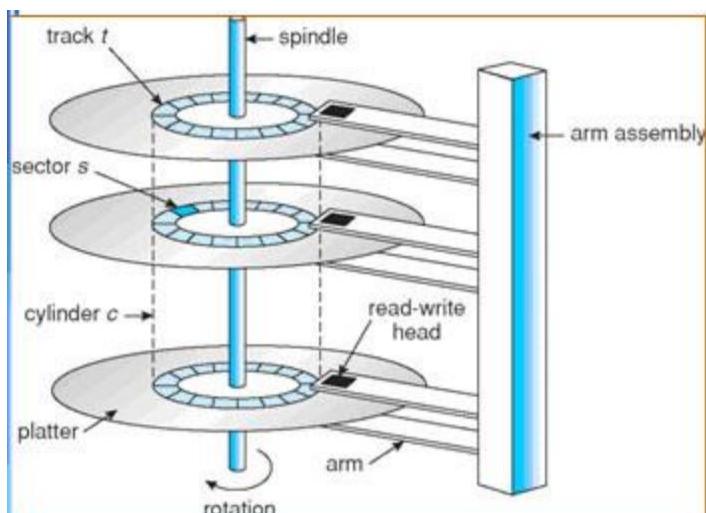
את המשטח מחלקים למסלולים (tracks): טבעות היקפיות סביב מרכז התקליט. בכל פלטה עשרות אלפי מסלולים.

כל מסלול מחולק לגזרות (sectors): יחידת המידע הקטנה ביותר שביכולת הראש לפנות אליו. (כל מסילה מחולקת ליחידות לוגיות קטנות יותר).

כל המסלולים שהרדיוס שלהם זהה נקראים גליל (cylinder)

מספרים אופייניים: 3-32 דסקות, עשרות עד מאות גלילים; עשרות עד כמה מאות גזרות במסלול, 512 בתים בנגזרת. (בעבר היו בדיסקט: דסקה יחידה, שני משטחים, 80 גלילים, 18 גזרות).

בעת פעולתו הדיסק מסתובב (כ- 60-200 סיבובים בשנייה).



גוש (Block) = מושג פחות חד משמעי מעמיתיו: ב-windows זוהי גזרת, וב-UNIX זהו גודל בסיסי של מעה"פ, בד"כ 16 גזרות. [לעיתים יוגדרו בלוקים פר פקודה]. כל גוש נמצא בתוך מסילה בודדת. מהלך זה יוצר פרגמנטציה.

כתובת block מכילה: מס' כוון, מס' גליל, מס' מסלול, מס' גזרת התחלתית.

מקבץ (Cluster) == היחידה הבסיסית שמע"פ בוחרת להקצות ולטפל בה בפעולת I/O. (בניגוד לגוש – הוא יהיה קבוע של מע"פ) גודל מקבץ = בין נגזרת אחת ל-128 נגזרות. במערכות קבצים שונות, מקבץ עשוי אפילו שלא לכלול סדרה רציפה של נגזרות.

נוהגים לחלק את מהירות הדיסק ל-2 מרכיבים:

1. זמן הצבה (time positioning) נקרא גם זמן גישה אקראית (random access time) המורכב משני גורמים:

א. זמן חיפוש – time seek = הזמן שנדרש להם הצבת הראשים בגליל המתאים + חביון הסיבוב

ב. latency rotational = הזמן שנדרש לשם סיבוב הדסקות כך שהנגזרת הדרושה תוצב מתחת לראשים. (25 מילי שניות).

2. קצב העברה – rate transfer = הקצב בו עוברים הנתונים מהכונן לזיכרון מהרגע בו הם התחילו לעבור בפועל. (מספר MB לשנייה)

הדיסק נקשר למחשב באמצעות פס. סוגי פסים נפוצים:

- Advanced Technology Attachment (ATA)
- Serial ATA (SATA).
- Universal Serial Bus (USB). SCSI

לפס מתחבר בצד הדיסק בקר הדיסק (disk controller), אשר מתפעל את הדיסק בפועל (בד"כ יהיה מצויד גם בחוץ המפונה 'מטמון הדיסק'). ובצד המחשב ה-controller host שמקבל הנחיות מהמעבד הראשי.

מבנה הדיסק

דיסק מספק מרחב כתובות ליניארי המאפשר פניה לגוש לוגי (= נגזרת) שהינו יחידת העברת הנתונים הבסיסית (בד"כ בנפח 512 בתים).

הדיסק ממפה את הכתובת הלוגית לנגזרת רצויה ע"ג הדיסק (גיאומטריית הדיסק, בתווית הדיסק) המיפוי הינו רציף: החל בנגזרת #0 במסלול החיצוני ביותר במשטח הראשון, עבור לנגזרות הבאות באותו מסלול, עבור למסלולים הבאים באותו גליל, עבור לגליל הבא; וכך הלאה.

היתרונות באחסון בדיסק:

1. קיבולת אחסון גדולה מהזיכרון הראשי.
2. מחיר נמוך.
3. האינפורמציה נשמרת גם לאחר כיבוי המחשב.

ביצד ניגש לדיסק?

כמו שפונים למערכי logical blocks. המערך ממופה ל-sectors של הדיסק.

ביצד נקרא/ נכתוב בדיסק?

1. הראש יתייצב על ה-track המתאים. (seek time)
 2. הדיסק יסתובב, כך שהראש יעמוד על ה-sector המתאים. (rotational delay)
 3. נעביר נתונים. (transfer time)
- נרצה לזרז את זמן מציאת הראש המתאים (1).

אלגוריתמים לתזמון זרוע הדיסק

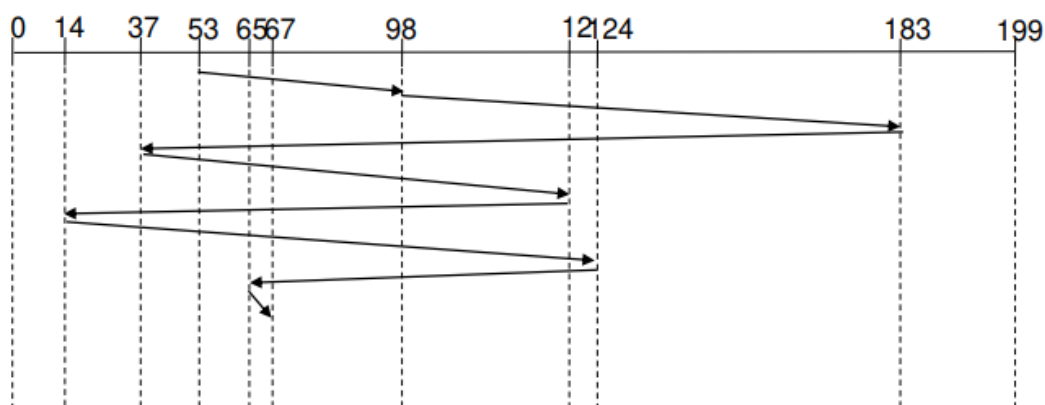
נדגים את ביצועי האלגוריתמים כולם – כאשר נקודת המוצא היא 53 וכן מחרוזת המיקומים הבאה:

98, 183, 37, 122, 14, 124, 65, 67

First Come First Served – FCFS

לפי סדר הגעת התהליכים לתור. חיסרון: גורם לתזוזות רבות בזרוע הדיסק.

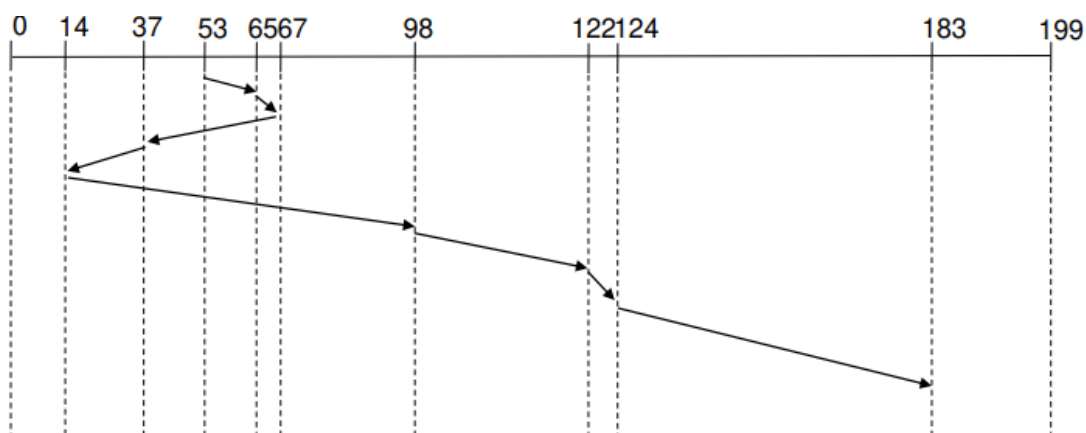
כמה גלילים יעברו ראשי הקריאה? $[98-53] + [183-98] + [183-37] + [122-37] + [...] + [65-67]$
סה"כ 640 גלילים.



Shorted Seek First – SSF

מוצא את הקרוב ביותר מבחינת מיקום. החיסרון: הרעבה למיקומים רחוקים.

כמה גלילים יעברו ראשי הקריאה? $[65-53] + [67-65] + [67-37] + [37-14] + [...] + [183-124]$
סה"כ 236 גלילים.

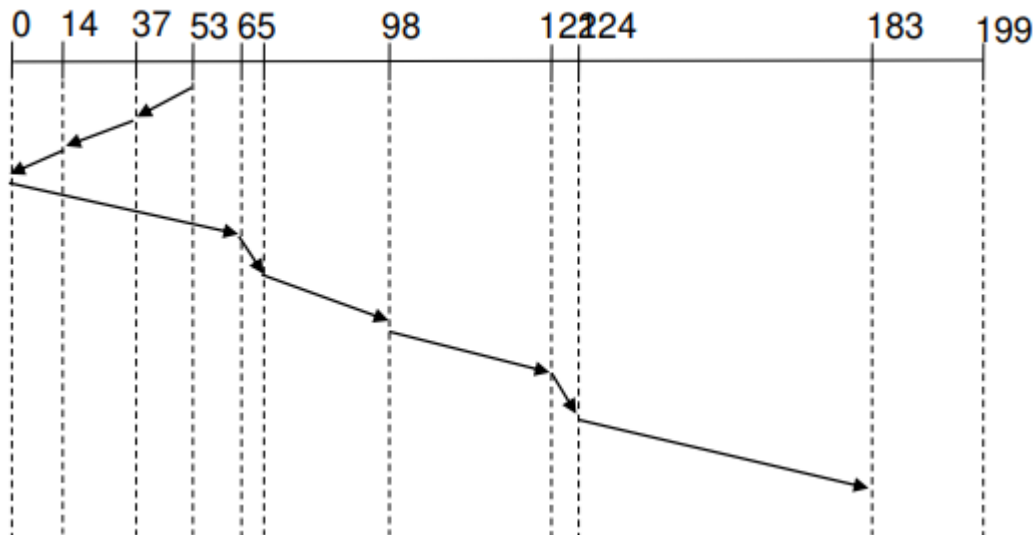


SCAN – אלגוריתם המעלית

חוזר למיקום 0 וממשיך משם לכיוון מיקום 0 של הדיסק. אם פוגש בדרך ל-0 מיקומים רצויים עוצר בהם, כך עד הקצה השני של הדיסק. לאחר מכן- כיוון הקריאה מתהפך לצד המקורי. חיסרון: פספוס אם הגיע בקשה למיקום X לאחר שעברנו אותו.

כמה גלילים יעברו ראשי הקריאה? $[183-124] + [...] + [65-0] + [14-0] + [37-14] + [53-37]$

סה"כ 236 גלילים.



Circular Scan – C-SCAN

מתחיל מהמיקום הנוכחי ומריץ את המיקומים שקרובים אליו אחרי – לכיוון המספרים הגבוהים יותר של הדיסק. לאחר מכן, פונה חזרה ל-0 וממשיך שוב בעליה. לא משרת בקשות בדרכו חזרה ל-0.

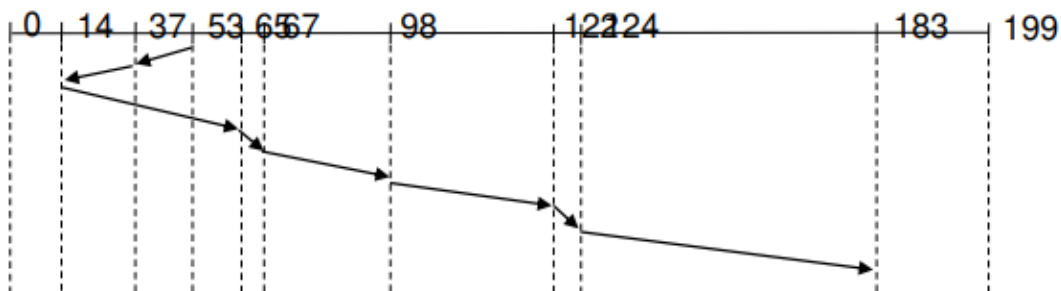
כך נוצר מצב שכביכול הגליל הראשון (0) הוא מייד אחרי הגליל האחרון (N) – לכן הסריקה נקראת 'סריקה מעגלית'.

LOOK

מתחיל מהמיקום הנוכחי, סורק פנימה, כל עוד יש בקשות בכיוון זה. ואז פונה החוצה למיקום הקרוב ביותר מהצד הנגדי החוצה.

כמה גלילים יעברו ראשי הקריאה? $[53-37] + [37-14] + [65-14] + [67-65] + [98- + \dots] + [\quad]$ 67]

סה"כ 208 גלילים.



C-LOOK

מתחיל מהמיקום הנוכחי, סורק פנימה, כל עוד יש בקשות בכיוון זה. ואז פונה החוצה למיקום הרחוק ביותר שצריך.

כמה גלילים יעברו ראשי הקריאה? $[53-37] + [37-14] + [183-14] + [183-124] + [124-122] + [122-98] + [98-67] + [67-65]$

סה"כ 326 גלילים.

דיסק בזיכרון – RAM DISKS

חלק מהזיכרון הראשי מוקצה לאחסון בלוקים של מידע, עם פקודות לקריאה וכתיבה. נאחסן בו נתונים שנזדקק להם תכופות (גישה מיידית).

מחולק ל-N בלוקים, כאשר כל בלוק בגודל זהה לבלוקים שבדיסק. כשהמתאם מקבל הוראת קריאה/כתיבה- הוא מבצע זאת ב-RAM DISKS, ומאוחר יותר- יבחר אותם לדיסק.

טיפול בשגיאות:

1. שגיאות תכנות: הבקר מאותת, והמתאם עוצר ומדפיס הודעה/ לא מבצע את הבקשה.
2. שגיאות בבדיקות ארעיות: חוזר על הפעולה, או שהבקר שומר מסילות למקרה זה, ומחליף בלוק מקולקל בבלוק חדש.
3. שגיאות בבדיקות קבועות: כשבבלוק ניזוק באופן פיזי.
4. שגיאות חיפוש: כשהזרוע נמצאת בצילינדר אחר. הבקר מתקן את הטעות/ מעדכן את המתאם שקורא לפקודה recalibrate (מזיזה את הזרוע ומאתחלת את מיקומי הצילינדרים).
5. שגיאות תוכנה בבקר: שבב ייעודי יידלק, והבקר יאפס את עצמו.

פיזור מידע-data striping

כדי לחלק באופן שווה את העומסים בין הדיסקים- נחלק את הקובץ למס' דיסקים פיזיים. נממש במערכי RAID

מערכי דיסקים – RAID

RAID – redundant array of independent disks: מערך יתיר של כוננים עצמאיים.

כלומר: איחוד כוננים קשיחים ליחידה לוגית אחת. כך נאפשר כתיבה וקריאה בו זמנית, ואף פיצול פעולה אחת למספר פעולות בו זמנית.

שיטות להקטנת אובדן מידע ב-RAID:

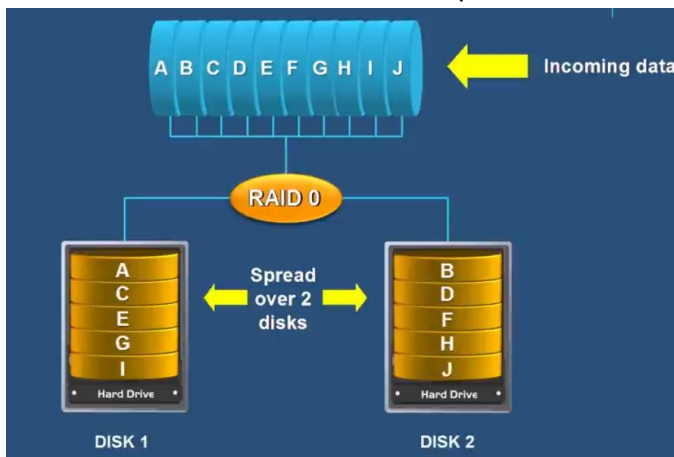
1. Mirroring – שכפול: שמירת כל המידע בשני עותקים. יוצר בזבוז מקום.
2. תוספת קוד- שמירת עותק בודד של המידע + קוד תיקון שגיאות, ע"מ לאפשר שחזור באובדן חלקי.
3. hot spare – רק ה-RAID יכול כונן קשיח נוסף שלא בשימוש תמידי. בעת תקלה- המידע מהדיסק המקולקל ישוחזר לדיסק הקשיח הנוסף.

מערכי ה-RAID הנפוצים ביותר:

1. RAID 0 - נקרא גם STRIPE

מורכב ממס' דיסקים המחוברים ברצף. חלק מהמידע נשמר בדיסק 1 והשאר בחלק נוסף. + ביצועים מהירים.

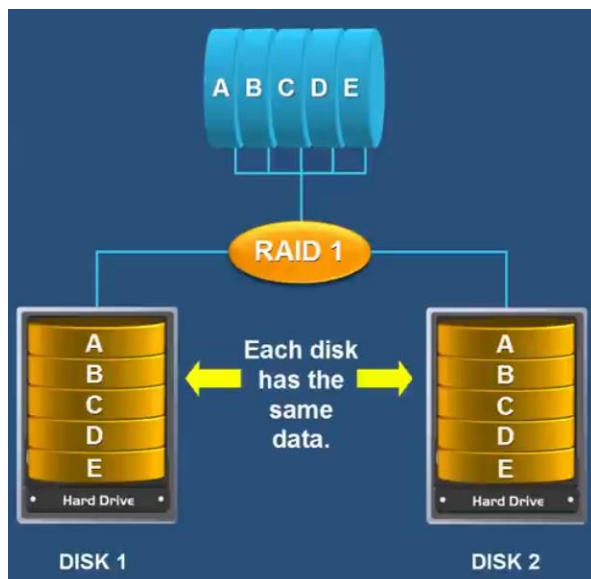
- בבשל מערכתי- המידע אובד עד להחלפת ושחזור הדיסק
- לא תומך ב-HS



2. RAID 1 - נקרא גם Mirror, ומורכב מ-2

דיסקים, ושומר את המידע פעמיים – ב-2 הדיסקים.

+ יתירות בשעת משבר
- מיועד לנפחים קטנים.



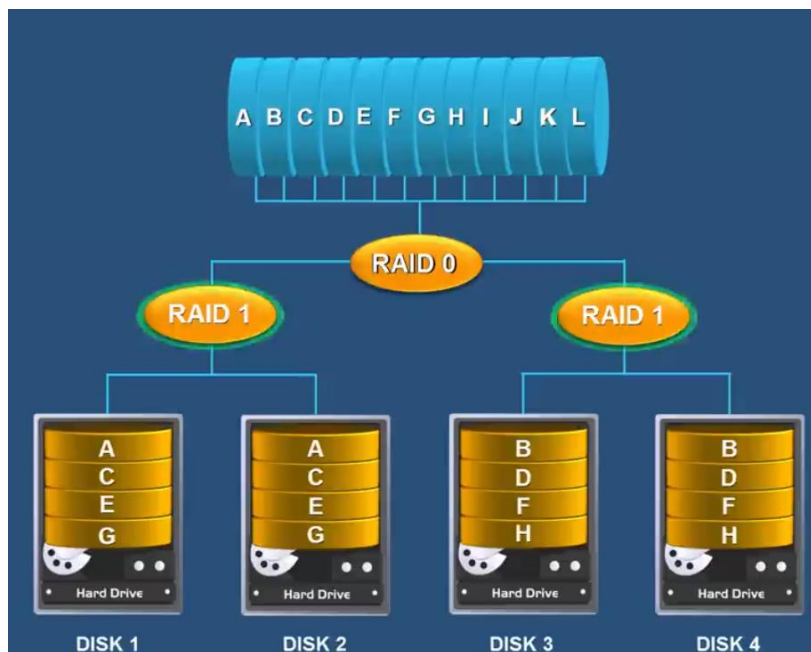
3. RAID 5

מורכב מ-3 דיסקים לפחות. נפח של דיסק אחד מבוזבז על ביט הזוגיות - Parity
+ יתירות, מהירות ביצועים
- בנפילת 2 דיסקים המערכת מושבתת



4. RAID 10

שילוב של RAID 0 ו-RAID 1



ביצד ניישם מערכי RAID?

1. בתוכנה (במעלה"פ): גובה זמן CPU בחישוב מקום במערך.
2. בחומרה (בבקר): יעיל יותר בקריאה וכתיבה
3. ניתן לשלב זיכרון מטמון, לערכים מבוקשים יותר.

ממשקי הדיסק להעברת נתונים:

- System Bus – פס המערכת.
- שפת הממשק
- בקר הדיסק. מחולק ל-3:
 1. בקר ה-DMA: מתקשר עם שאר המחשב
 2. בקר לוגי: מטפל בזיכרון המטמון.
 3. בקר פיזי: מניע את הזרוע המכוונת את הראשים בדיסק

שיקולים בבחירת ממשק: נפוצות הממשק, מחירו, אפשרויות הרחבה ושדרוג עתידיות.

הממשקים להעברת הנתונים:

1. ממשקי IDE / ATA: נפוצים ביותר.
IDE-Integrated Drive Electronic
ניתן לחבר 2 התקנים, ולעבוד איתם בנפרד.
העברת נתונים ב-80 פינים. ממשק SATA ב-7 פינים.
2. ממשקי SCSI: משוכללים ויקרים יותר.
SCSI- Small Computer System Interface
ניתן לחבר 15 התקנים בו זמנית. מהיר ויעיל.

ניהול הדיסק:

1. low level formatting - חלוקת הדיסק לסקטורים שהבקר יכול לכתוב ולקרוא בהם.
2. high level formatting - חלוקת הדיסק למחיצות לפי צילינדרים. (partition)
פירמוט לוגי- להכנת מערכת הקבצים.
3. אתחול ה-block root:
אחסון תוכנת ה-bootstrap loader ב-ROM
חלוקת הדיסק למחיצות לוגיות. (יוצר אבטחת נתונים)
ב-sector הראשון של הדיסק מאוחסן label בשם (volume table of content) -VTOC
מכיל מידע על בקר הדיסק, וטבלת חלוקת הדיסק- disk partition table.

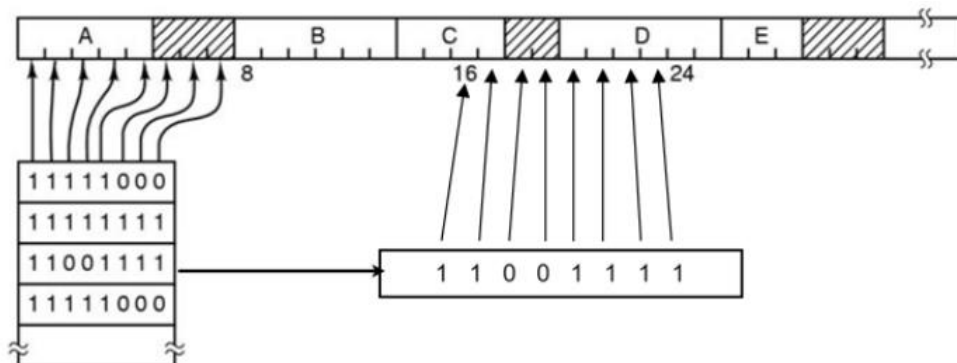
ניהול שטח הדיסק

כיצד ננהל את מאגר הבלוקים הפנויים?

1. Bit Map / Bit Vector

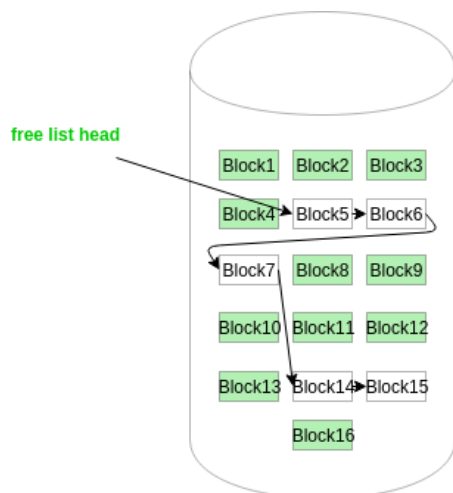
- נייצג את הדיסק באמצעות ווקטור של סיביות, כאשר כל סיבית מסמנת את מצב הבלוק. [1 = פנוי, 0 = תפוס].
- נשתמש בפקודות מכונה מיוחדות העובדות על ביטים (במיוחד כאלו המוצאות את הביט ה-1' הראשון מתוך רצף ביטים) ע"מ למצוא בלוק פנוי.

נשים לב כי כאשר מדובר בדיסק גדול מאוד – א"א להחזיק את המפה בשלמותה בזיכרון ← איטיות ב-I/O.



2. Linked List

- מערכת ההפעלה תשמור מצביע לרשימה מקושרת המכילה את הבלוקים הפנויים.
- מתאימה בעיקר להקצאות בבלוקים בודדים.
- נוכל לשמור לכל איבר ברשימה כמה בלוקים רצופים אחריו – פנויים.



File System – מערכת הקבצים

המטרות בניהול מערכת הקבצים:

1. הפשטת תכונות ספציפיות של התקן פיזי.
2. זמן ביצוע סביר
3. הפרדה בין משתמשים שונים
4. הגנה ואבטחה

המבנה שמע"פ מספקת:

- א. ממשק משתמש – תכונות הקובץ, פעולות, מבנה התיקיות.
- ב. מיפוי של מערכת הקבצים הלוגית – למצע האחסון הפיזי. (ע"י מבני נתונים + אלגוריתמי מימוש)

הדיסק הוא אחסון בגישה ישירה [← כדי לגשת ל-block בדיסק נצטרך זמן חיפוש + זמן חביון] וכמובן שניתן גם לשכתב מידע על מיקום בדיסק.

כדי לטפל בדיסק ביעילות ונוחות – נוכל לארגן אותו באמצעות מערכת קבצים אחת או יותר.

בד"כ מערכת הקבצים בנויה בשיטת השכבות. לדוגמה – מודל 4 השכבות:

- 1 I/O Control – שולטת בציוד ובחומרה, ומעבירה את הנתונים מהזיכרון לדיסק. שכבה זו מכילה את מנהלי ההתקנים – Device Driver ואת טפליות הפסיקה – Interrupt Handlers.

[שכבה זו מפורטת בנושא מערכות I/O]

- 2 Basic File System – שכבה זו אינה מבחינה בין ההתקנים השונים, לא מכירה קבצים בשלמותם, ולא מכירה את תכונות הדיסק. היא מעבירה בלוקים פיזיים לדיסק.

[שכבה זו מפורטת בנושא]

- 3 File Organization Module – שכבה זו מכירה את מיפוי הקובץ (הנמצא בבלוקים לוגיים) לדיסק (בלוקים פיזיים).

היא ממירה כתובת לוגית [block 17 בקובץ X] לכתובת פיזית [כונן 4, גליל 3827, משטח 3, נגזרת 12].

[שכבה זו מפורטת בנושא ארגון מערכת הקבצים]

- 4 Logical File System – שכבה זו מנהלת את מערכת הקבצים בשלמות, ללא עניין בתוכן הקבצים. היא מכילה את מבנה המדריכים, הרשאות, תאריכים חשובים לקובץ, בעלים, ומיפוי בין שמו הסמלי של הקובץ ל-ID הפנימי שלו.

שכבה זו שומרת לכל קובץ מבנה נתונים בשם (File Control Block) FCB, ובו הנתונים הנ"ל. ב-UNIX מבנה זה מכונה **inode**.

[שכבה זו מפורטת בנושא]

מע"פ שומרת את טבלת החלוקות שבשימוש, רשימת המדריכים אליהם פנינו לאחרונה, טבלת הקבצים הפתוחים במערכת, וטבלאות קבצים פתוחים לכל תהליך בנפרד.

אוסף נתונים קשורים, בעלי שם, השמורים ע"ג מצע אחסון. זהו מבנה נתונים מופשט ADT, והיא יחידת הפניה והטיפול הבסיסית בנתונים השמורים ע"ג הציווד. כלומר: כדי לשמור פיסת מידע כלשהי ע"ג ציווד – יש לאחסנה בקובץ.

המערכת מזהה את הקובץ בד"כ לפי ID, וישמרו מידע גם על טיפוס הקובץ.

ב-UNIX נשמור מעט מידע על טיפוס הקובץ, לעומת מעה"פ אחרות המרחיבות במידע נוסף.

מערכת הקבצים מורכבת מ:

- אוסף של קבצים (בד"כ מהדיסק).
- directory structure ארגון לוגי של הקבצים ומידע אודותם: כתובת הקובץ ע"ג הציווד, גודל הקובץ, תאריך יצירה, עדכון, פניה אחרונה, שם הבעלים, הרשאות וכו' (בתוך inode).

לעיתים תחזיק גם:

- חלוקות / כוננים / Partitions – המפרידים פיזית או אפילו רק לוגית את מבנה המדריכים לתתי קבוצות.

פיזית – כל קובץ שמור ביחידות מידע קטנות – block'ים.

לוגית – הקובץ מורכב מ'רשומות' = כל רשומה היא שורה בקובץ טקסט.

Packing – אריזה: כאשר מצרפים מס' רשומות לוגיות בתוך החוצץ, ע"מ להעבירן לדיסק כ-block אחד. חיסרון: עלולה להיווצר פרגמנטציה פנימית! אם לא נמלא את ה-block'ים בשלמותם (במיוחד לא את הגוש האחרון)...

כשתהליך **יוצר** קובץ חדש הוא פונה למערכת הקבצים הלוגית, היא מקצה FCB חדש, קוראת לזיכרון את קובץ המדריך המתאים (אליו נוסיף את הקובץ) ומעדכנת אותו בהתאם.

כשנכתוב נתונים בקובץ – מודל ארגון הקבצים יקצה בלוקים בדיסק לכתיבה, ויעדכן את ה-FCB בהתאם, וכמובן גם את רשימת הבלוקים הפנויים בדיסק.

כשתהליך **פותח** קובץ – נסרוק את המדריך שלו, נאתר את שמו הסמלי של הקובץ – ולפיו את מס' ה-ID הפנימי שלו, נשלוף מהדיסק את ה-FCB ולפי ההרשאות – נעדכן את טבלת הקבצים הפתוחים של התהליך. פתיחת קובץ תחזיר לנו את ה-file descriptor שהוא מס' הכניסה בטבלת הקבצים הפתוחים של התהליך. מכאן ואילך נשתמש רק ב-File Descriptor.

כשתהליך **סוגר** קובץ – הפעולה סימטרית לפתיחת קובץ.

זהו קובץ הכולל טבלה המכילה את שמות הקבצים הנכללים באותו מדריך. קובץ המדריך יישמר ע"ג הציוד ככל קובץ אחר. המדריך יכול עבור כל קובץ הנכלל באותו מדריך את שם הקובץ (מבחינת המשתמש), ה-ID עבור מעה"פ, ואולי נתונים נוספים אודות הקובץ.

כל ADT מכיל:

1. Name – המשתמש קובע. ישנם קבצים ששםם כולל נקודה וסיומת- סוג הקובץ.
2. Data – תוכן.
 - א. ללא מבנה- רצף של בתים. הגישה ע"י offset (הסטה) מתחילת הקובץ. [כך ב-UNIX]
 - ב. מבנה של אוסף רשומות- גישה ע"י מפתח.
- מספר הבתים / הרשומות – יכול להיות בגודל קבוע / משתנה, והתוכן – בינארי או טקסט.
3. Attributes – תכונות
4. Operations – פעולות

Name – שם הקובץ:

לכל מעה"פ יש חוקים משלה למתן שם לקובץ במערכת. וכן רוב מעה"פ תומכות בשם קובץ המכיל גם את סוגו (tar.c / letter.txt / page.docx) – זהו החלק המופרד ע"י נקודה ומכונה File Extension. לעיתים נאכוף סיומות ולעיתים נקל. אך חשוב להפריד את סוג הקובץ, לדוגמה כאשר קומפיילר מהדר קבצי אסמבלר ו-C.

ב-UNIX גודל שם הקובץ יכול להיות עד 255 תווים.

סוגי קבצים ב-UNIX

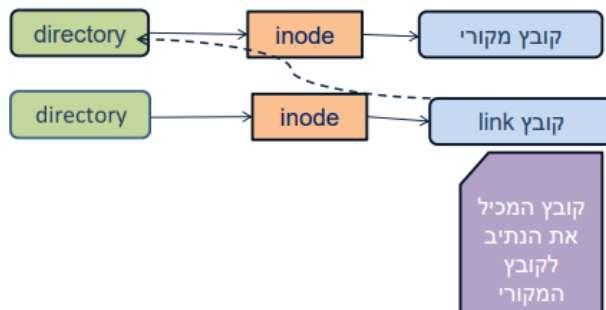
כל מעה"פ מחליטה מהם הנתונים החשובים ביותר עבורה, ואותם היא שומרת עבור כל קובץ. UNIX שומרת מעט מאוד מידע.

1. **Regular Files** – מידע של המשתמש. 2 סוגי קבצי משתמש:
 - א. קובץ Ascii – מכילים שורות טקסט ומאפיינים למעבר שורה. אין אכיפה לאורך השורות.
 - ב. קובץ בינארי – קבצים המיועדים לביצוע: רק תכנית מפעילה, ומעה"פ חייבת לזהות אותם. הם חייבים להכיל מבנה מסוים: כותרת, טקסט, נתונים, ציון מקום, וטבלת סמלים (לאיתור שגיאות).
2. **Directories** – קבצי מערכת, השומרים על מערכת הקבצים
3. **Character Device Files** – עיצוב I/O של התקני תווים (התווים נשלחים לחומרה והיא קוראת אותם כתו יחיד).
4. **Block Device Files** – עיצוב דיסקים.

[קבצים מסוג 3 ו-4 נשמרים בתיקיית /dev]
5. **Local Domain Sockets** – העברת הודעות
6. **Named Pipes** – קבצים זמניים להעברת פלט פקודות
7. **Symbolic Links** – קבצי קישור

ב-UNIX יש 2 סוגי קישורים:

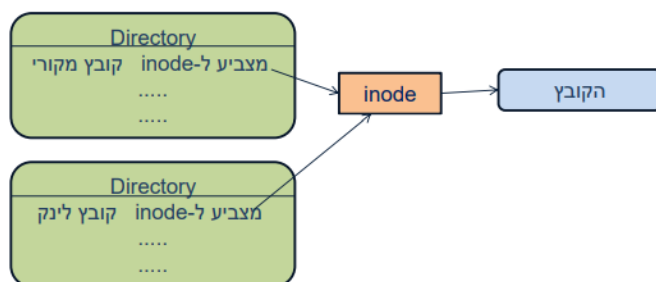
א. Soft Link – קישור לקובץ ראשי. הקובץ כולל inode המפנה לקובץ אחר. כאשר נפתח את הקובץ שהוא soft link – מעה"פ מאתרת את הקובץ המקורי מבצעת עליו את הפעולות. זהו רק 'שם משני' לקובץ המקור. אם נשנה או נמחק את הקובץ המקורי – הקישור יהיה 'יתום מתנדנד' (dangling).



כדי לייצר soft link נשתמש בפקודה: `ln -s source_file new_file` הפונקציה במעה"פ נקראת `symlink()` ומקבלת כפרמטר את קובץ המקור והקובץ החדש. נוכל לייצר קיצור דרך לתיקייה / קובץ / קובץ במכונה מרוחקת (NFS) / קובץ לא קיים.

ב. Hard Link – קישור לקובץ שווה ערך לקובץ הראשי, ממש 'שם ראשי' נוסף. כאן לא ניצור inode נוסף במערכת, אלא רק נשמור את מס' ה-inode של הקובץ המקורי. כך כל שינוי על אחד מהקבצים – משפיע על שניהם.

כדי לייצר hard link נשתמש בפקודה: `ln source_file new_file` הפונקציה במעה"פ נקראת `link()` ומקבלת כפרמטר את קובץ המקור והקובץ החדש. נוכל להציג את כל ה-hard links שבתקייה הנוכחית ע"י הפקודה `ls -li`



מחיקת קובץ `link`: תתבצע ע"י הפונקציה `unlink()` שתקבל כפרמטר את הניתוב לקובץ ה-`link`. נשים לב שהפקודה `ls -li` ב-UNIX תציג בתו הראשון בכל שורה בפלט את סוג הקובץ. סוגי הקבצים:

התו:	סוג הקובץ:
-	Regular Files
d	Directories
c	Character Device Files
b	Block Device Files
s	Local Domain Sockets
p	Named Pipes
l	Symbolic Links

Operations – פעולות על קבצים:

הערות:	מבצעת:	הפעולה:
נפתח כניסה מתאימה בטבלת הקבצים של התהליך ונטען את ה-inode לשם.	פתיחת קובץ	open()
סימטרית לביצוע open()	סגירת קובץ	close()
- הקצאת 'שם' פנימי למעה"פ - הוספת שורה במדריך בו יימצא הקובץ - הקצאת תא לקובץ במערך הקבצים של מעה"פ – ה-inode - הקצאת שטח אחסון במערכת הקבצים עבור הקובץ, ושמירת המידע היכן מיקומו כעת בדיסק	יוצרת קובץ חדש	create()
סימטרית לביצוע create()	מוחקת קובץ	delete()
- מקבלת מתאר לקובץ, ומצביע למידע שיש להכניס אליו. - מאתרת את כתובתו בדיסק ואת המקום בו התהליך הנוכחי כותב על הקובץ - מעבירה את הנתונים מהחוצץ לקובץ	כותבת לתוך קובץ	write()
סימטרית לביצוע write()	קוראת מתוך קובץ	read()
המצביע לקובץ מוזז למיקום חדש בקובץ. למעשה לא נעשית כאן כל פעולה על הקובץ, אלא על מבנה הנתונים המחזיק את המידע אודות הקובץ הרלוונטי לתהליך זה.	מזיזה את המצביע בתוך הקובץ	seek()
לא נתמכת בכל המערכות	מוסיפה נתונים לסוף הקובץ	append()
	משנה שם של קובץ קיים	rename()
התכונות קיימות ב-inode	שולפת את תכונות הקובץ	get_attributes()
	יכולה לשנות חלק מתכונות הקובץ	set_attributes()

ב-UNIX כל ה-I/O מתבצע באופן לוגי – לקובץ או מקובץ, גם כשמדובר בהתקני I/O שאינם קבצים (כמו טרמינל, מדפסת וכו').

ההבדל בין מערכות הקבצים השונות הוא בעיקר בצורת ההקצאה והגישה לקבצים בהתאם להתקנים שלהם.

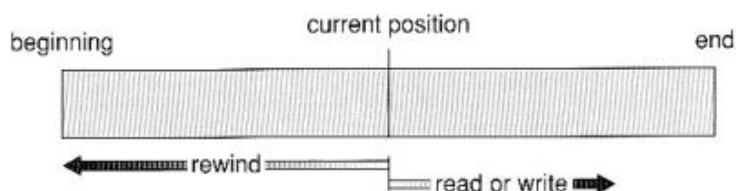
Access Methods – שיטות גישה לדיסק:

קיימות שיטות שונות לגישה למידע בקבצים, מעה"פ יכולה לספק יותר מדרך אחת.

1. גישה סדרתית – Sequential Access:

נסרוק את הרשומות בקובץ באופן סדרתי. כל פעולת קריאה קוראת נתון, ומקדמת את המצביע לקובץ לרשומה הבאה. כתיבה על קובץ סדרתי ניתן לבצע רק בסוף הקובץ, ועל כן כדי לעדכן את הנתונים בקובץ יש למעשה להעתיקו, תוך הכנסת העדכונים. לא ניתן לעדכן נתון בודד באמצע הקובץ.

סרט מגנטי מאפשר רק קבצים עם גישה סדרתית.



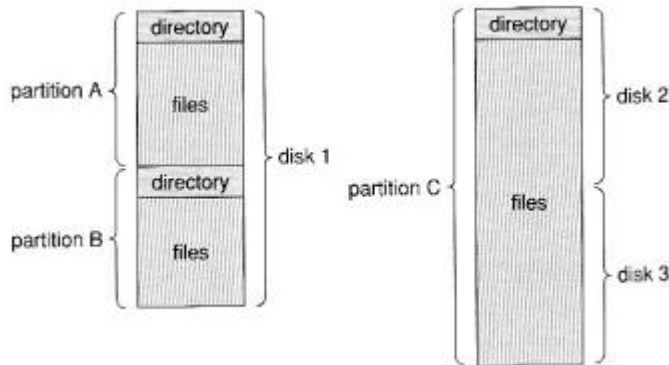
2. גישה ישירה / גישה אקראית – Random Access:

בניגוד לסרט, בו ניתן לפנות לרשומות רק באופן סדרתי, בדיסק ניתן לפנות ישירות לכל רשומה רצויה בקובץ. נוכל לפנות ישירות לרשומה מסוימת. כיצד? נחזיק מבנה נתונים נוסף (כמו עץ B או טבלת גיבוב) וכשנזין למבנה הנתונים את פרטי הרשומה המבוקשת – נקבל בחזרה את כתובתה בדיסק. עכשיו נוכל לפנות ישירות לכתובת בדיסק, ולשלוף את הרשומה.

ניהול קבצים – Directories ו-Partitions

כדי לארגן את הקבצים נוכל להשתמש ב-2 רמות:

1. **רמת החלוקה – volume / partition** [חלוקה של הדיסק הפיזי לדיסקים לוגיים].
זוהי סדרה רציפה של גלילים בדיסק המוקצית לכל partition. בכל חלוקה קיים בלוק המכיל מידע אודות החלוקה: מס' גלילים השייכים לחלוקה, מס' בלוקים, מעקב אחר בלוקים פנויים, מצביעים ל-FCB ועוד.



יתרונות:

- כאשר קיימות מס' מעה"פ – כל אחת תקבל partition משלה.
- partition אחד יחושב כ'מרחב השחלוף' או למטרת גיבוי.
- קל לפירמוט / גיבוי / תחזוקת נתונים.
- יורסים בד"כ לא גולשים בין partition'ים.
- נחזיק תוכניות יחד עם הקבצים שלהן ב-partition אחד, וכך זמן החיפוש יהיה מהיר.
- בכל partition תהיה מערכת קבצים קטנה שתנהל אותו.
- כאשר partition מכיל כמות מעטה יחסית של track'ים (מסלולים) – נקבל נפח גדול + זמן חיפוש קטן. בחלק הפנימי של הדיסק נאחסן את הגיבוי – משתמשים בו מעט...

חסרונות:

- מבני הנתונים של מעה"פ – כפולים ← בזבוז שטח בזיכרון!
- כשנפנה לעיתים קרובות לקבצים השוכנים ב-2 חלוקות שונות – ה-Optimizer של הדיסק לא יוכל למקם אותם בסמיכות זה לזה, וכן נצטרך לפנות ל-2 מבני נתונים של מעה"פ, ולא רק ל-1.
- מכיוון שלא נוכל לאחד שטחים בין חלוקות שונות ← נקבל הרבה יותר פרגמנטציה!
[נרצה לשמור קובץ בגודל X שאין עבורו שטח פנוי בחלוקה 1, אבל יש ב-2 נפרדות 😞]
- כשנעביר קובץ בין חלוקות – נצטרך גם להזיז את הנתונים פיזית בדיסק ממקום למקום.
- לעיתים ישנה הגבלה על פניה לקבצים בחלוקות שונות. [ב-UNIX ניתן לעשות hard link רק על קבצים באותה החלוקה].

סוגי Partitions:

- חלוקה 'גולמית' – Raw: חלוקה זו אינה מכילה מערכת קבצים. נשתמש בשיטה זו במרחב השחלוף, ב-DB שמנהל את הדיסק והגישות אליו, ובאפליקציות המשתמשות במערכות קבצים לא קונבנציונליות.
 - חלוקה 'אפויה' – Backed: חלוקה זו מכילה מערכת קבצים.
- נמשיך לארגן כל חלוקה בפני עצמה – ע"י שימוש ב-Directories (מדריכים):

2. רמת המדריך – directory

ברוב מערכות הקבצים – כל מדריך הוא קובץ בעצמו. הוא מכיל כניסות במס' הקבצים שהוא מפנה אליהם.

ישנם 2 מבנים עיקריים למבנה מדריך:

- א. כל מדריך מכיל: שם קובץ, תכונות, וכתובת אחסון הקובץ בדיסק. [Windows]
- ב. כל מדריך מכיל: שם קובץ, מצביע ל-DS אחר – בו נשמרים תכונות הקובץ, וכתובת האחסון בדיסק. [Unix]

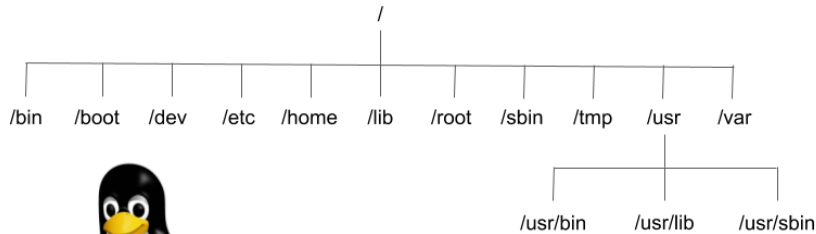
פעולות על מדריכים:

הערות:	מבצעת:	הפעולה:
	יוצרת ספריה חדשה	<code>create()</code>
ניתן למחוק ספריה רק כאשר היא ריקה.	מוחקת ספריה	<code>delete()</code>
• נוכל לקרוא את כל הקבצים שבספריה • לפני ביצוע הפעולה יש לפתוח את הספרייה.	פתיחת ספריה	<code>opendir()</code>
כדי לפנות מקום בטבלה הפנימית – נסגור ספריה בשנסיים לקרוא בה.	סגירת ספריה	<code>closedir()</code>
נוכל להשתמש גם ב-system call הרגיל <code>read()</code> רק אם המתכנת מודע למבנה הפנימי של הספריות.	קבלת הכניסה הבאה בספריה הפתוחה	<code>readir()</code>
	שינוי שם הספרייה	<code>rename()</code>
אפשר גם soft link ע"י <code>symlink()</code>	יצירת hard link למדריך	<code>link()</code>
	הסרת קישור לספרייה	<code>unlink()</code>

מודלים למבנה מערכת הקבצים

עץ ספריות:

בד"כ מבנה המדריכים נראה כעץ פשוט. כך נראית מערכת הקבצים של Linux:



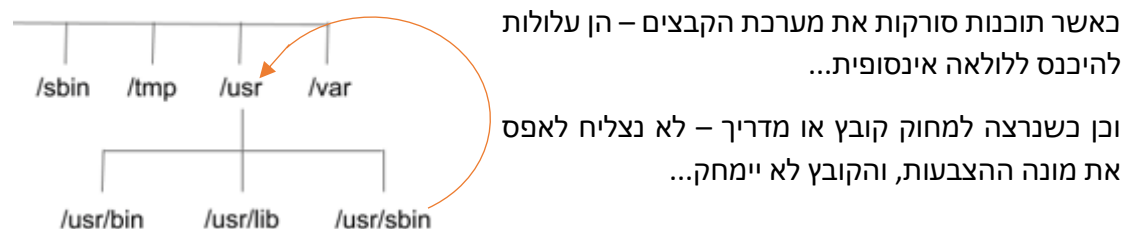
Linux file system

אך במצב בו נרצה שיותר ממדריך אחד יכיל הפניה לקובץ – נצטרך לדאוג שהמדריכים יכילו הפניה לאותו הקובץ ע"י hard link / soft link וכך המדריך הופך להיות shared directory.

כך נקבל גרף חסר מעגלים (DAG).

כאשר מס' מקומות מפנים לאותו הקובץ – עלולים לצוץ קשיים: יש לקובץ יותר משם 1, לעיתים הוא יגובה פעמיים, ונבצע פעולות כפולות...

נשים ❤️: כשגרף המדריכים מכיל קישורים (רכים או קשים) – נוצרת אפשרות שיהיו בגרף מעגלים... ← מורכבות מיותרת!



כאשר תוכנות סורקות את מערכת הקבצים – הן עלולות להיכנס ללולאה אינסופית...

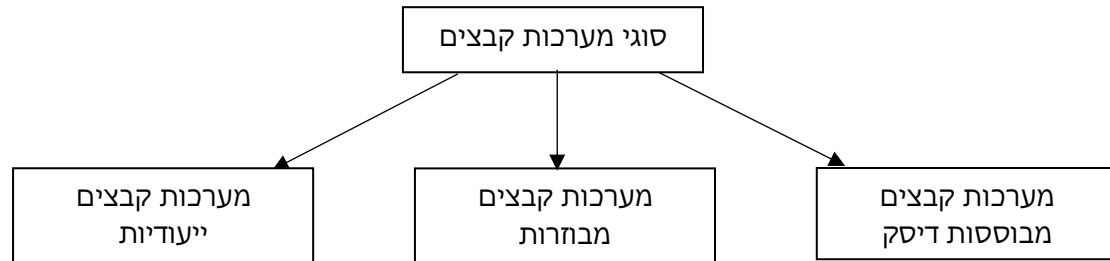
וכן כשנרצה למחוק קובץ או מדריך – לא נצליח לאפס את מונה ההצבעות, והקובץ לא יימחק...

פתרונות:

- 1) נאפשר ומעגליות, ונשתמש ב-garbage collector [אוסף אשפה]
- 2) נאפשר מעגליות רק באמצעות soft link ולא נתעניין בהם במשימות שונות.
- 3) בסריקת מערכת הקבצים נתעלם מקבצי קישור.

File System Implementation – מימוש מערכת הקבצים

הקבצים במחשב שוכנים בדיסק האחסון המשיני. בנושא זה נביר את אופן שמירת המידע במערכות מחשבים מתקדמות בדיסקים מגנטיים.



מערכות קבצים מבוססות דיסק

ב-UNIX המסורתית משתמשים ב-UFS.

ב-Linux משתמשים ב-Ext2fs. קיימות גם מערכות קבצים נוספות כמו: JFS, ReiserFS ועוד.

ב-Windows משתמשים ב-FAT / FAT32 / NTFS.

קיימות גם מערכות קבצים להתקני זיכרון הבזק – כמו YAFFS ו-JFFS2.

כל מערכת שמתעדת מידע ע"מ למנוע ששגיאות ו/או השחתה של מערכת הקבצים – מכונה 'Journaling File System' – מערכת קבצים מתועדת.

מערכות קבצים מבוססות

אלו מערכות קבצים הפועלות כלקוח לפרוטוקול גישה לקובץ מרוחק, וכך נותנות גישה לקבצים השמורים בשרת.

עבור רשת האינטרנט קיימות מערכות קבצים כמו NFS, AFS, SMB, FTP ועוד.

כמו כן, מערכות קבצים אלו משמשות לדיסקים המשותפים בין מס' מחשבים ו/או שרתים, כך שהן מסונכרנות בגישתן למערכת הקבצים – למניעת התנגשויות בכתיבה.

לדוגמה: GFS, GPFS, CFS ועוד.

מערכות קבצים ייעודיות

אלו מערכות קבצים המנהלות מבני נתונים השייכים ל-kernel. מטרתן לאפשר לתהליכי מערכת ומנהלי מערכת לבצע ניטור ובקרה על הגרעין בפשטות, וכן לאפשר מימוש מנגנוני IPC.

proc (IPC), sockfs (זיכרון משותף ב-IPC), shm (pipes ב-IPC), pipefs (קבצי התקנים) devfs (ניטור ובקרה הגרעין)

ארגון מערכת הקבצים

מענה"פ מחלקת את הקבצים לבלוקים בגודל קבוע, ושומרת אותם בדיסק, אך לא בהכרח ברצף. אם נתבקש לבצע הקצאה ברצף ← תיגרם פרגמנטציה חיצונית + בעיה בהגדלת קובץ.

לעומת זאת, כאשר נמפה בלוקים גדולים – נקטין את תקורת המיפוי ואת גודלו של מבנה הנתונים, ונשפר את ביצועי הדיסק. גם כאן נשים לב כי בלוקים גדולים ← יגרמו לפרגמנטציה פנימית בבלוק האחרון של הקובץ, שלא תמיד יתמלא עד סופו.

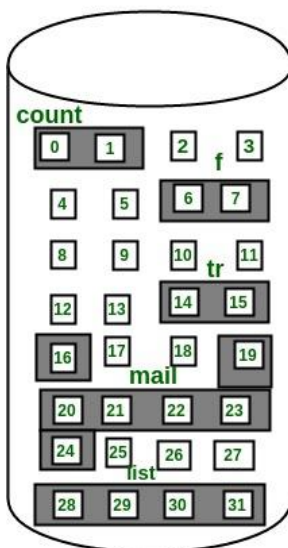
כשנרצה להקצות שטח בדיסק – לשמירת קבצים נרצה לשמור על מהירות שליפה גבוהה, וכמובן לנצל את שטח האחסון ככל שניתן.

שיטות להקצאת בלוקים בדיסק:

1. הקצאה רצופה:

- בשיטה זו נאחסן כל קובץ כרצף בלוקים בדיסק.
- + קל ליישום (נדע רק את הכתובת הראשונה)
- + קריאת הקובץ ע"י פעולה אחת – הקטנת זמן קריאה
- לא בר ביצוע: א"א לדעת את גודל הקובץ לפני שנוצר
- יצירת פרגמנטציה חיצונית בדיסק
- מציאת חור גדול מספיק עלולה להיות בעייתית
- מתאים רק למערכות הדורשות I/O מהיר, וכשגודל הקבצים ידוע מראש.

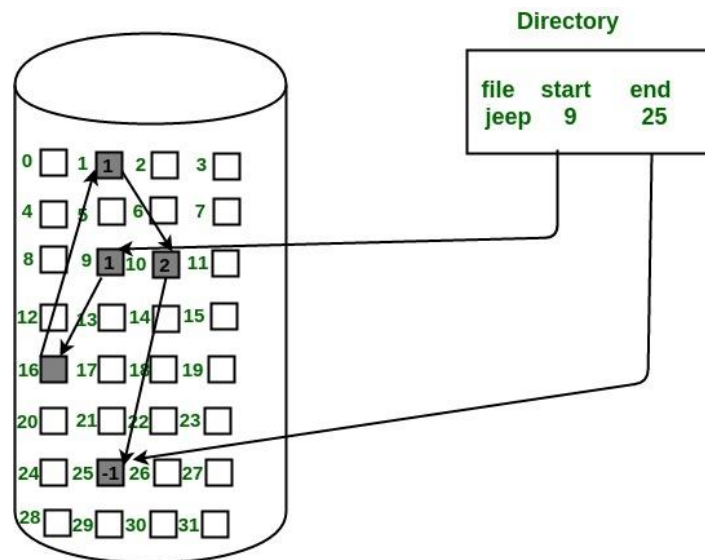
Directory



file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

2. רשימה משורשרת:

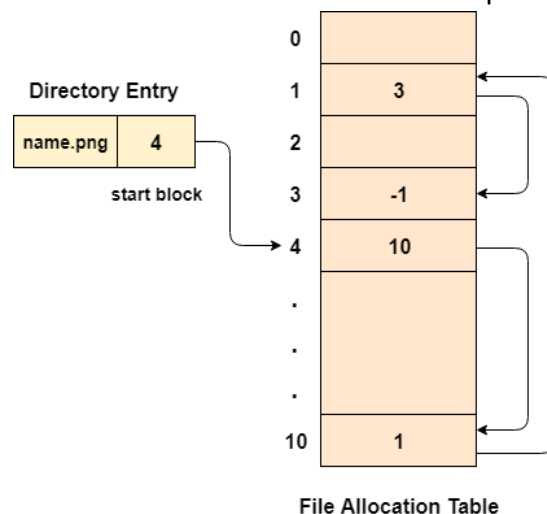
- אחסון קובץ ברשימה משורשרת של בלוקים. כל חוליה בשרשרת מכילה מצביע לבלוק הבא ברשימה ונתונים על הבלוק הנוכחי.
- + קל להגדלה דינאמית
- + אין בזבז מקום בדיסק – ללא חורים
- + יעיל יותר – הספרייה שומרת רק את כתובת ראש הרשימה
- גישה איטית לקובץ: נצטרך לסרוק סדרתית את כל הרשימה כדי להגיע לבלוק X
- א"א לשמור נתונים בחזקות 2 כי המצביע תופס מספר בתים בעצמו
- בעיית אמינות – איבוד מצביעים כתוצאה מתקלה בדיסק או באג במענה"פ
- פשרה: נקצה בלוקים ב-Clusters קבוצות סמוכות, ונשלם בפרגמנטציה פנימית



3. FAT:

זהו שיפור לרשימה משורשרת – בתחילת חלוקת הדיסק תהיה טבלה של מצביעים לבלוקים בדיסק. כל שורה בטבלה מיועדת לבלוק בדיסק, ותוכנה – מצביע לבלוק הבא. אם הבלוק לא הוקצה – ערכו יהיה '0'.

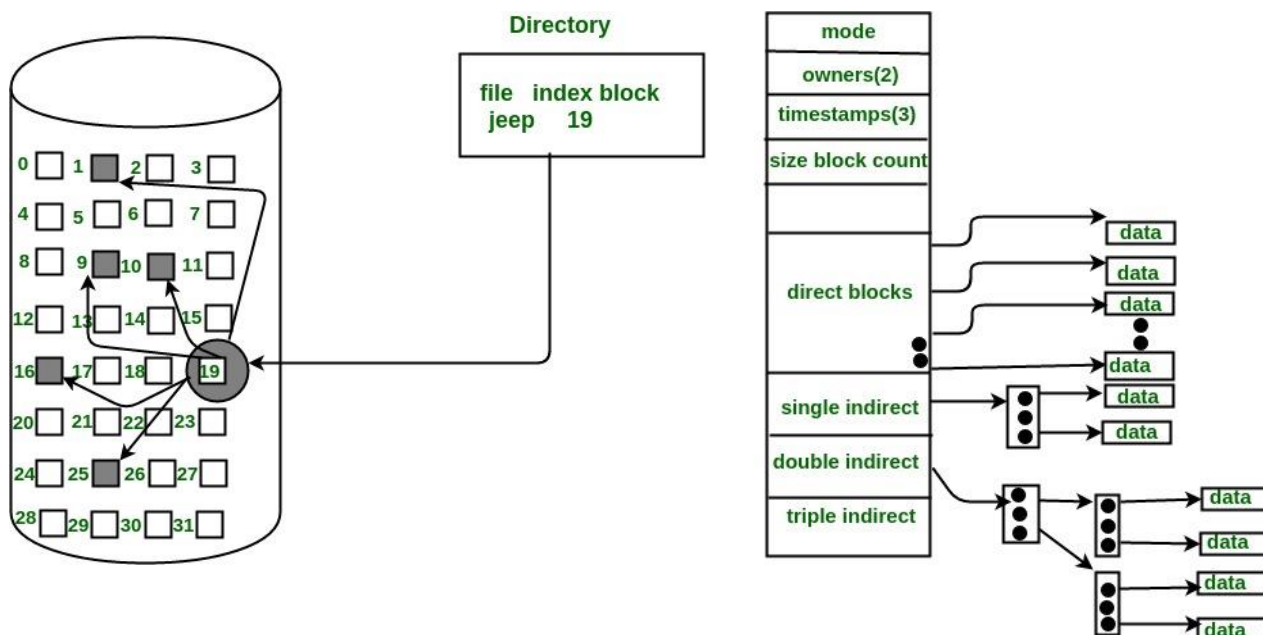
+ בהנחה שהטבלה נטענת לזיכרון – גישה ישירה לבלוק הקובץ בדיסק
- טבלה גדולה: מכילה מצביעים לכל הבלוקים בדיסק



4. I-node:

טבלת אינדקסים עבור כל קובץ, ומרוכזת בבלוק אחד. ה-inode יצביע על בלוק האינדקס. [דומה מבחינה רעיונית לטבלת דפים היררכית].

- + אין קיטוע חיצוני
- + אין דרישה לרציפות זיכרון
- + קל לביצוע: סדרתית ורנדומלית
- קיטוע פנימי
- הגבלת גודל הקובץ
- הפתרון: עץ גושי אינדקס – ע"י רשימה מקושרת
- יהיו מצביעים ישירים – direct, ומצביעים למצביעים – indirect. עד 3 רמות.
- + הגדלת קובץ, כאשר ה-inode נשאר קטן.
- גישה איטית לקובץ. ככל שיש יותר רמות הצבעה ← הכפלה / שילוש של גישות לדיסק.



5. NTFS:

שיטה המיועדת ל-Windows ולניהול התקני DiskOnKey. כאן נשתמש רק ב-clusters ולא ב-blocks. בקבצים קטנים – נקבל פרגמנטציה פנימית גדולה, לכן – עבורם נשתמש ב-FAT. ה-FCB מכונה כאן MTF וגם הוא נחשב 'קובץ'. מכיל את תחילת רצף הנתונים, מידע על הקובץ, מערך מצביעים לבלוקים המכילים את תוכן הקובץ. כשנרצה להגדיל את הקובץ – נוכל לשמור טבלת אינדקסים נוספת ב-MTF. מבנה הנתונים הראשי הוא MFT (Master File Table) – המממש Hash Table.

חסרונות:	הגדלת קובץ:	סדרתית:	ישירה:	
פרגמנטציה חיצונית	קשה	גישה אחת	גישה אחת	הקצאה רציפה
גישה סדרתית איטית	הוספת חוליה לרשימה	N גישות	i גישות	רשימה משורשרת
טבלה גדולה	הוספת שורה בטבלה	N גישות	גישה אחת	FAT
פרגמנטציה פנימית	שימוש במס' רמות של מצביעים	$N + N^2 + N^3$ גישות	גישה אחת / תלוי ברמת המצביע	I-node
מערכת קבצים גדולה	הוספת טבלת אינדקסים ב-MFT	N גישות	גישה ע"י key	NTFS

Virtual File System – מערכת הקבצים הווירטואלית

מערכת ההפעלה מחזיקה שכבת וירטואליזציה לצורך טיפול במצבים רבים, כמו: קבצים השוכנים במס' התקנים נפרדים / מציאת מערכת קבצים מתאימה לכל התקן / טיפול במערכות קבצים מרוחקות.

מערכת ההפעלה חייבת לתמוך במס' מערכות קבצים, וכמובן להסתיר מהמשתמש את ההבדלים שביניהן – כך שייראו כמערכת אחת אחידה.

- ? - כיצד מערכת ההפעלה מבצעת זאת?

- מערכת הקבצים תמומש בצורה היררכית.

המבנה המקובל הוא בן 3 שכבות:

א. File System Interface – FSI: זהו ממשק מערכת הקבצים למשתמש [דרך תוכנה כמו

סייר הקבצים או Shell] ומספק פעולות על קבצים, משתמשת ב-File Descriptors.

- למדנו על שכבה זו בנושא 'פעולות על קבצים', וניגע בו עוד בהמשך בפרק "

ב. Virtual File System – VFS: מערכת הקבצים הווירטואלית, ותפקידה לדאוג לממשק

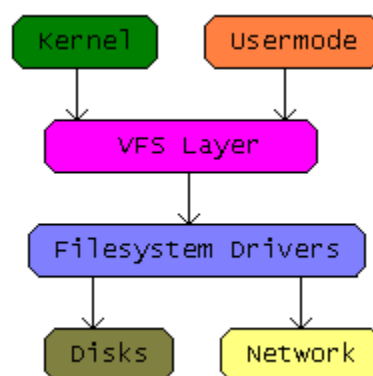
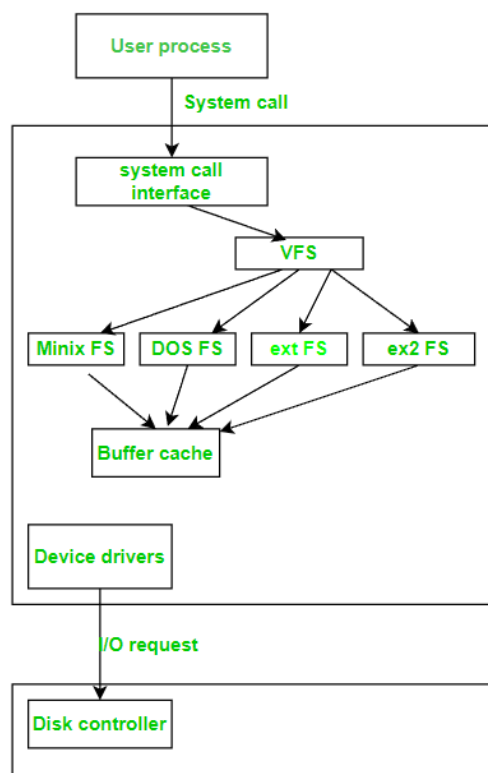
המשתמש להיראות אחיד, למרות השוני בין מערכות הקבצים השונות, גם במערכות

קבצים השייכות לרשת (כמו NFS).

- נראה את המימוש של שכבת ה-VFS כאן למטה.

ג. File System – FS: מערכת הקבצים עצמה.

- למדנו על שכבה זו בפרק 'מערכת הקבצים' ובפרקים שאחריו.



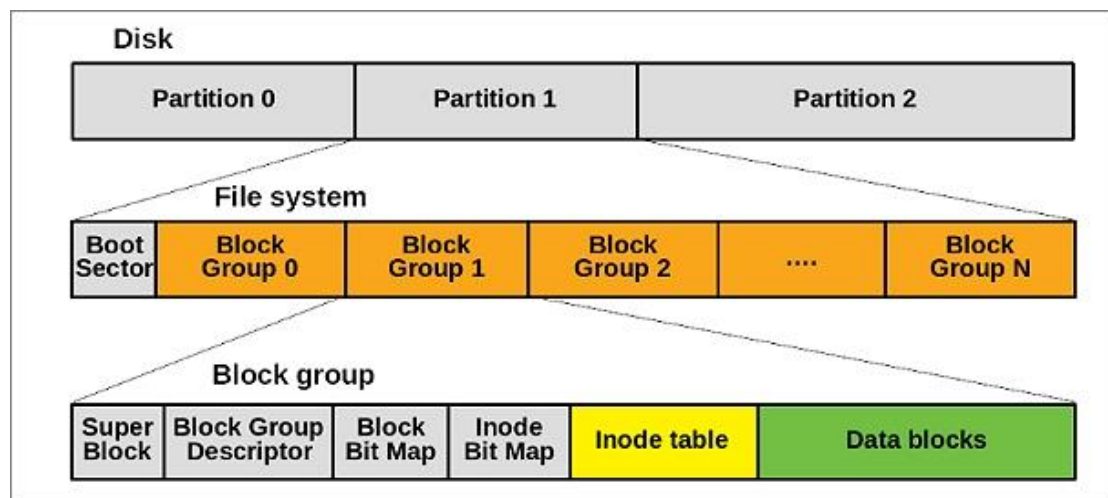
- פרמטרים המשותפים בין מערכות קבצים שונות – ייכתבו פעם אחת בלבד.
- פרמטרים המותאמים לכל מערכת בנפרד – נכתבים לכל מערכת בנפרד.
- מערכת הקבצים הווירטואלית – VFS – מתרגמת את הבקשות הכלליות לקריאות המתאימות למערכת הקבצים בה שוכן הקובץ.

- ? - ממה מורכבת מערכת הקבצים הווירטואלית?

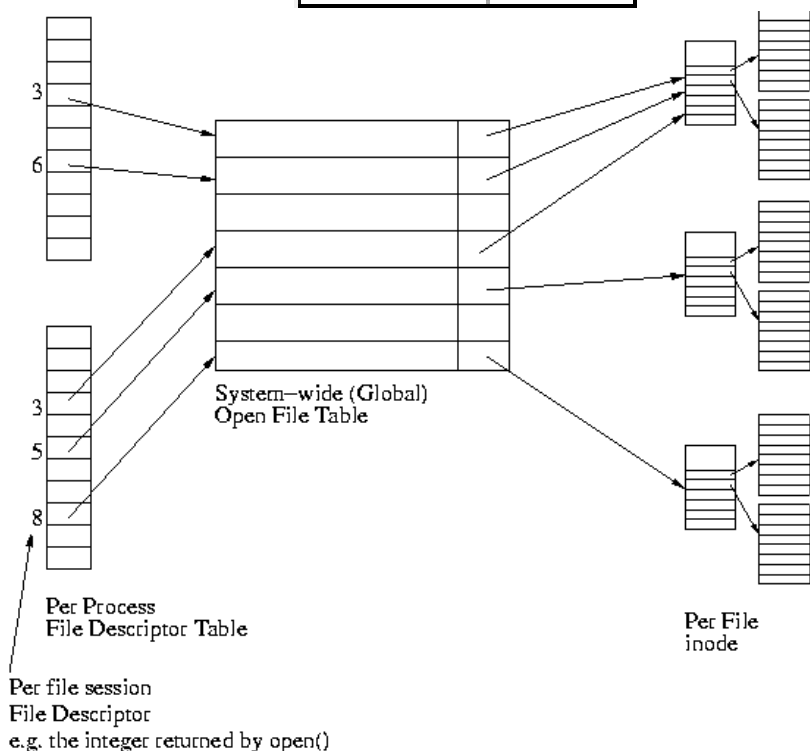
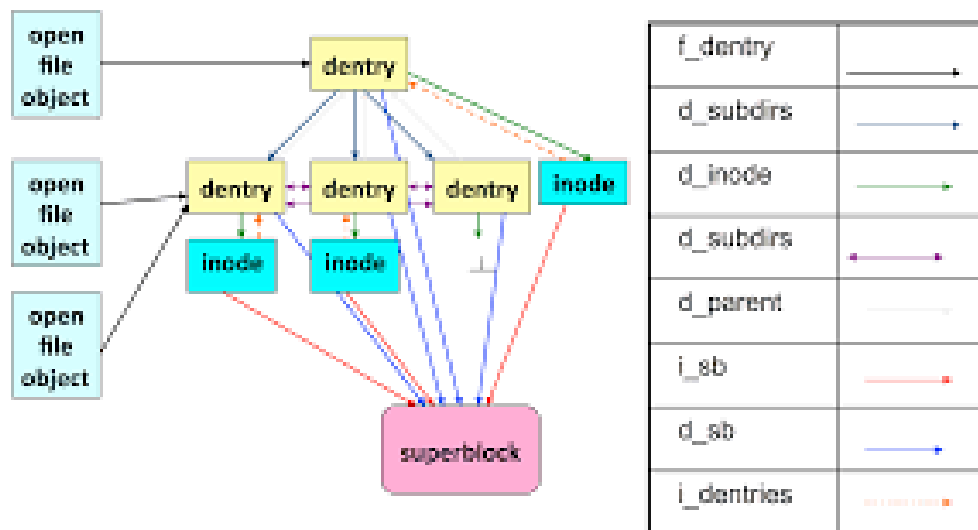
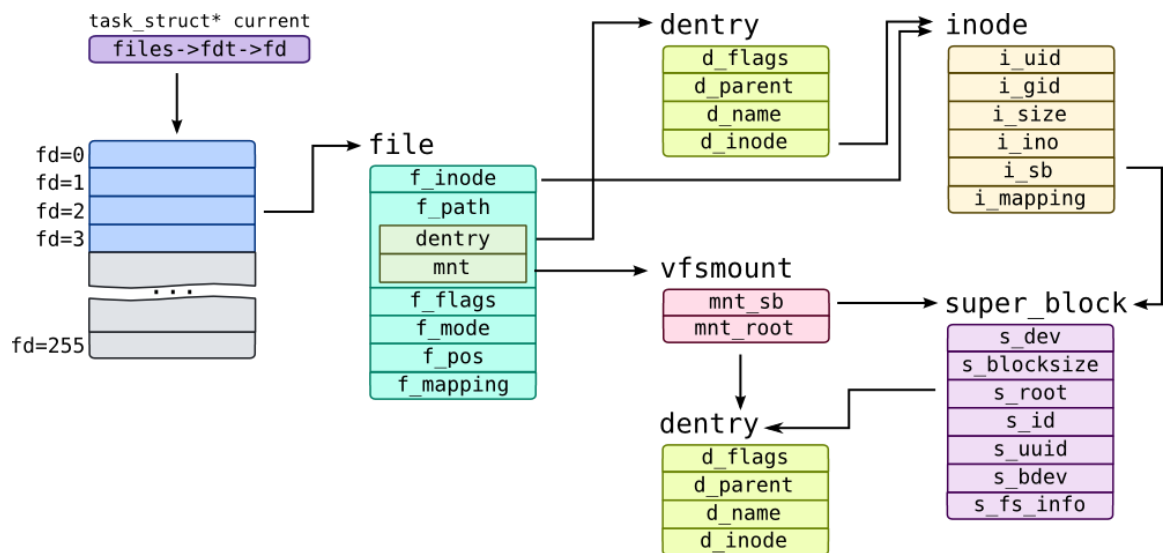
- ! - מערכת הקבצים מכילה את המרכיבים הבאים, ויודעת לייצר את האבסטרקציה הרצויה.

1. File – קובץ פשוט
2. File Descriptor – מתאר הקובץ
3. Inode – ייצוג הקובץ [למדנו ב'ארגון מערכת הקבצים']
4. Dentry – כניסה ב-Directory (לכל ניתוב יש כניסה מתיקיה מסוימת). בד"כ יישמר רק ב-RAM אך גם ב-cache כדי להאיץ את החיפוש.
5. Super Block – בלוק השומר מידע על כל מערכת הקבצים.

נדגים את המבנה על מערכת הקבצים Ext2fs של Linux:



1. File – קובץ פשוט
2. File Descriptor – מתאר הקובץ
3. File Descriptor Table Per Process – טבלה השומרת מצביעים ל-File Descriptor לכל קובץ שהתהליך פתח.
4. File Descriptor Table – טבלה השומרת את ה-File Pointers לכל הקבצים הפתוחים במערכת באופן כללי. כל כניסה בטבלה זו נקראת File Object ומכילה את דגלי הסטטוס של הקובץ, ה-offset הנוכחי שלו, ומצביע ל-VNODE המתאים לו.
5. Inode – ייצוג הקובץ [למדנו ב'ארגון מערכת הקבצים']
 - Dentry – ה-kernel יוצר Dentry לכל כניסה ב-Directory. לדוגמה, עבור הניתוב "/var/letter" ייווצרו Dentry 3:
 - עבור התיקיה הראשי '/'
 - עבור התיקיה tmp כי היא בכניסה של התיקיה הראשית
 - עבור הקובץ letter כי הוא בכניסה של התיקיה tmp
6. Super Block – מכונה כאן גם Partition Table, וזוהי טבלה המתארת את כל החלוקות שנטענו.
7. VNODE Table – טבלה השומרת לכל קובץ מידע על ה-VFS ואת ה-inode שלו.



עגינת מערכת הקבצים – Mounting

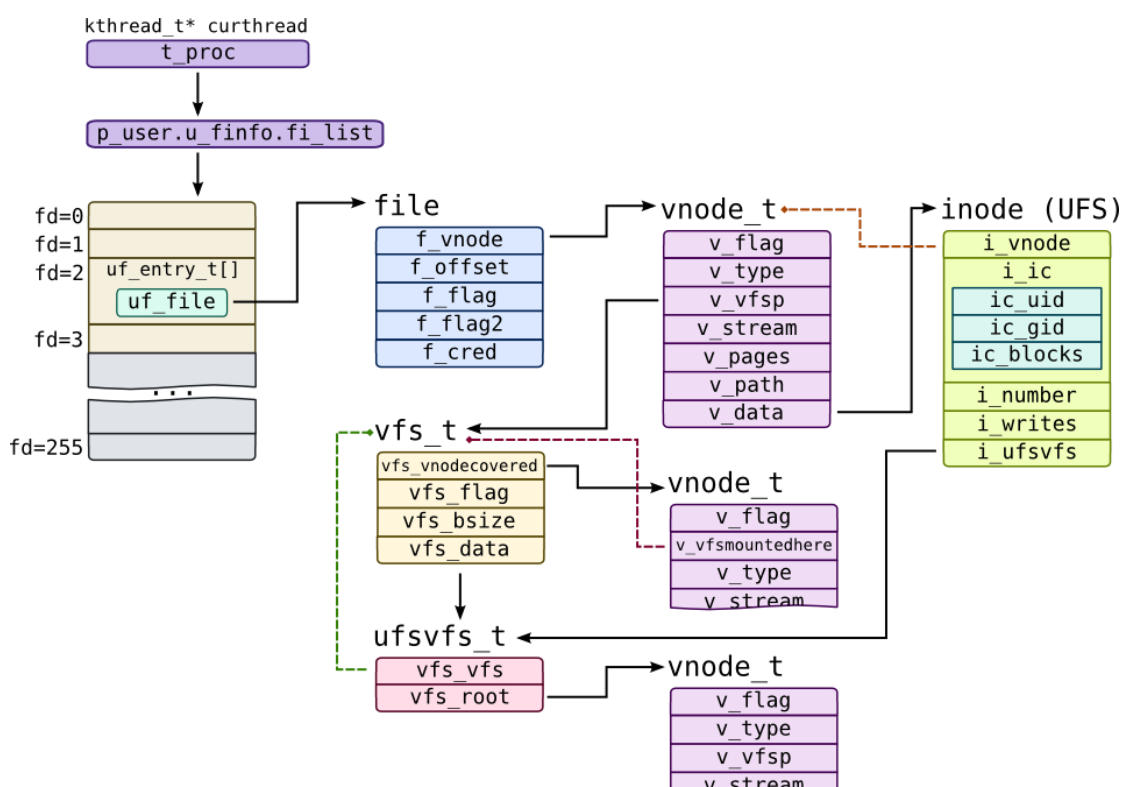
כדי לאפשר 'עגינה' – הטענה של מערכת קבצים X על גבי מערכת קבצים Y – נצטרך לשמור כל מערכת קבצים ב'מחיצה' נפרדת.

מחיצת ה-root מכילה את מערכת הקבצים הבסיסית של מערכת ההפעלה, וטוענת אותה יחד עם מערכת ההפעלה. שאר המערכות יעוגנו באופן אוטומטי או באופן ידני.

תמיד נעגן מערכת קבצים לתוך תיקיה ריקה. אם היו קבצים בתיקיה בה עיגנו את מערכת הקבצים – הקבצים ייחסמו לגישה עד להסרת מערכת הקבצים שעוגנה.

ה-struct – מבני הנתונים המאפשרים לבצע עגינה:

1. **VFS** – רשימה מקושרת השומרת את מערכות הקבצים שעוגנו. ראש הרשימה שומר את מערכת הקבצים של ה-root.
2. **VFS_OPS** – מבנה קבוע (לכל מערכת קבצים) השומר את הפעולות שניתן לבצע במערכת קבצים זו, ואת המצביעים לפונקציות. [פונ' עגינה וכדו']
3. **VNODE** – מבנה לכל קובץ, השומר את סוג הקובץ, מס' הגישות, המידע שב-inode, וכן מצביע לאיבר ב-VNODE_OPS המתאים לו.
4. **VNODE OPS** – מבנה לכל מערכת הקבצים, המכיל את מימוש הפונקציות שבה.



נוכל לעגן (אפשר לומר גם 'להרכיב' - Mounting) מערכת קבצים – ידנית או אוטומטית. התיקיה בה נבצע את ההרכבה מכונה 'mount point' – נקודת הרכבה.

לכל מערכת קבצים מוגדר 'Root Directory' – תיקיית שורש.

מערכת הקבצים שמורכבת לתוך ה-root של ה-VFS – נקראת מערכת הקבצים הראשית.

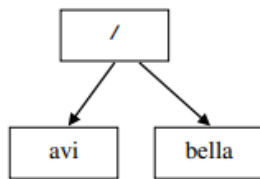
ההרכבה מתבצעת ע"י קריאת מערכת `mount()` והסרת מערכת קבצים ע"י קריאת מערכת `unmount()`. ניתן לבצע זאת גם ע"י פקודות `mount` ו-`unmount` בהתאמה.

טעינת מערכת הקבצים

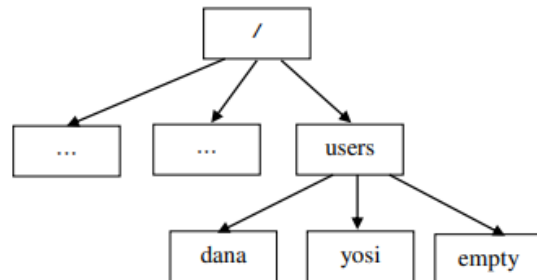
כדי להשתמש במערכת קבצים – נצטרך לטעון ולהעלות אותה, כמו פתיחת קובץ לפני השימוש בו. פעולת הטעינה מקבלת 2 פרמטרים: שם הציוד, ונקודת הטעינה (mount point) – המיקום במערכת הקבצים הקיימת, בו נציב את מערכת הקבצים החדשה. הפעולה בודקת שמערכת הקבצים בציוד אכן תקינה, ואז מוסיפה במדריך המתאים הפניה למערכת החדשה שנטענה.

נניח שנתונות:

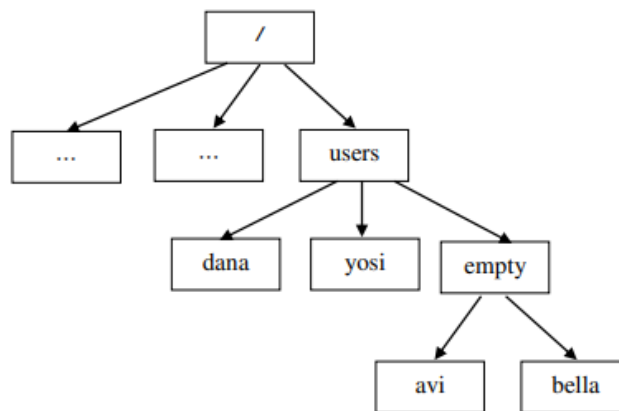
מערכת קבצים בדיסק הלוגי:



מערכת קבצים קיימת:



כאשר נטען את מערכת הקבצים מהדיסק הלוגי – מבנה העץ ישתנה:



כאשר מערכת הקבצים מאורגנת כעץ מדריכים, יש צורך במהלך התהליך לציין את שם הקובץ:

נוכל לייצג את שם הקובץ ב-2 אופנים:

א. ניתוב אבסולוטי – Absolute Path Name:

ב. ניתוב יחסי – Relative Path Name:

נכתוב את הניתוב באופן יחסי לספריית העבודה הנוכחית. כל מדריך יכול הצבעה למדריך של עצמו ['. ']. והצבעה למדריך האבא שלו ['. .. ']

פעולות על קבצים

1. `path_name()` - איתור inode של קובץ לפי `path` (נתיב)

2. `Open()` – פתיחת קובץ

לפני השימוש בקובץ - חייב לפתוח אותו.
פעולת ה-`Open()` בודקת גם את חוקיות הפעולה למשתמש מבחינת הרשאות וכדו'.
הפעולה:

- מזמנת את `path_name()`
- מוסיפה כניסה לטבלת הקבצים הפתוחים של מערכת ההפעלה ובה ה- `File Descriptor` של הקובץ. [טבלת הקבצים הפתוחים של התהליך תכיל מצביע לכניסה המתאימה בטבלת הקבצים הפתוחים של מערכת ההפעלה.]
- יוצרת `File Object` חדש ומעדכנת את התכונות שלו.
- מאתחלת ב-`VNODE` את הפעולות שניתן לבצע על הקובץ - לפי ה-`inode` שלו. כך כל הפעולות העתידיות יבוצעו דרך ה-`VNODE`.
- זימון הפעולה `Open()` של ה-`File Object` החדש
- מילוי הכניסה בטבלת הקבצים הפתוחים - במצביע ל-`File Object` החדש.
- לאחר פתיחת הקובץ, כל הפעולות על הקובץ שנפתח יתבצעו ע"י שימוש ב- `File Descriptor` שלו.
- המטרה היא למנוע חיפוש של הקובץ בעץ הספריות, ולאפשר מהירות כאשר הקריאות עוקבות זו לזו – נביא תכונות וכתובות מהדיסק ל-RAM.

3. `Close()` – סגירת קובץ

הפעולה:

- מוצאת את ה-`File Object` בתא של ה-`File Descriptor` של הקובץ הרצוי ומסמנת אותו כ-`NULL`.
- מפעילה את הפונ' `flush()` של ה-`File Object` [באם היא מוגדרת]
- כותבת את ה-`Buffers` לדיסק
- משחררת את ה-`File Object`
- מחזירה את קוד השגיאה של הפונ' `flush()`

4. `Create()` – יצירת קובץ

הפעולה:

- מוצאת שטח אחסון בגודל הרצוי ומקצה אותו עבור הקובץ
- מקצה `inode` חדש מטבלת ה-`inode`'ים
- הוספת כניסה בטבלה של ה-`Directory`

5. `Delete()` – מחיקת קובץ

- מוצאת את הכניסה המתאימה ב-Directory
 - מוחקת אותה
 - משחררת את הזיכרון בדיסק + עדכון הבלוקים הפנויים
- ! - במקרה של hard link המחיקה תוריד את כמות הקישורים, וב-0 תמחק לגמרי.

6. Read() – קריאה מקובץ-

- מקבלת את מתאר הקובץ – File Descriptor + כמות בתים רצויה לקריאה
 - מאתרת את כתובת הקובץ – לפי ה-inode + מיקום המצביע על הקובץ
 - מעבירה את הנתונים לחוצץ
 - מקדמת את המצביע
- הפעולה מחזירה את כמות הבתים שקראה. אם המס' שחזר קטן מכמות הנתונים שביקשנו לקרוא – יכול להיות שכמות הנתונים בקובץ קטנה. אם מדובר בקריאה מה-Shell – נוכל לקרוא רק עד שורה אחת. וכן כשמדובר בקריאה מהרשת – לעיתים נקבל פחות בתים מהמס' שביקשנו בעקבות סוגי Buffering והעברת נתונים.

7. Write() – כתיבה לקובץ

- מקבלת את מתאר הקובץ – File Descriptor + מצביע למידע שנרצה לכתוב
- מאתרת את כתובת הקובץ – לפי ה-inode + מיקום המצביע על הקובץ
- מעבירה את הנתונים לחוצץ
- מקדמת את המצביע

8. Seek() – הזזת מצביע

- הפונ' מאתרת את הקובץ במערכת הקבצים, ומשנה את ערכו של מצביע המיקום – File Position Pointer שנמצא ב-File Object. כלומר: מתבצע עדכון של מבנה הנתונים.

9. Append() – כתיבה בסוף הקובץ

[פונ' זו אינה נתמכת בכל המערכות].

10. Get_Attributes() – גישה לתכונות הקובץ

נוכל לשלוף את התכונות המשויכות לקובץ כדי לבדוק שינויים וכדו'.

11. Set_Attributes() – שינוי תכונות הקובץ

נוכל לשנות חלק מתכונות הקובץ, כמו: שם, הרשאות גישה, דגלי מצב וכדו'. לדוגמה: Rename() – לשינוי שם הקובץ. [שינוי השם נעשה ב-Directory].

שיתוף קבצים

הרשאות

ביצועים

התקדמות הטכנולוגיה

- ככל שפערי המהירות בין הדיסק לזיכרון הולכים וגדלים – מחיר הגישה היחסי לדיסק עולה.
- ככל שגודלם של הזיכרונות הולך וגדל – בד"כ נבצע קריאות מה-cache ← רוב הגישות לדיסק מתבצעות למטרת כתיבה ← חשוב לשפר ביצועי כתיבה יותר מביצועי קריאה.
- כשמבצעים I/O ביחידות רצופות – העבודה יעילה יותר.
- רוב העבודה של מערכות קבצים מורכבת מפעולות קטנות ומפוזרות על הדיסק ← פגיעה בניצולת הדיסק.
- לדיסק מסוג SSD (flash) יש מאפיינים שונים מדיסק מגנטי:
 - כתיבה מחדש לאותו האזור = פעולה בעייתית! נצטרך למחוק את האזור כולו, אך המחיקה איטית, ומתבצעת ביחידות של 64KB ומעלה.
- כמו כן, המחיקה פוגעת בהתקן – א"א למחוק אזור מסוים יותר ממיליון פעמים.
- כיום בקרים מודרניים מזיזים את המידע ממקום למקום כדי להימנע משחיקה של אזורים הנמצאים בשימוש תדיר.
- בד"כ נשתמש במערכת קבצים מתועדות (log)

מערכות קבצים מתועדות

כד לשמור על אמינות ה-meta-data – מערכות קבצים מתועדות – מתעדות את הפעילות שלהן, בקבצי log סדרתיים – לפני שהם נכתבים לדיסק – ע"מ לתעד את הטרנזקציות שמבוצעות.

בכל שורה נכתוב: transaction id, data_item name, old value, new value. בפתיחת וסגירת הטרנזקציה יופיעו רשומות מיוחדות, והבלוקים שהשתנו – ייכתבו מאוחר יותר לדיסק, יכול להיות שיתבצעו לא לפי הסדר.

נוכל למחוק מה-log עדכונים שכבר בוצעו. וכמובן לבטל עדכונים שביצענו.

לאחר נפילה של המערכת, נוכל לעבור על קובץ ה-log ולבצע הרצה של כל שורה. אם השורה האחרונה לא מלאה – נתעלם ממנה.

מערכות קבצים מתועדות מוכרות גם כ-JFS (Journaled File System) ונמצאות ב: NFS, ZFS ועוד.

+ נוכל לכתוב את ה-metadata באופן א-סינכרוני

+ נוכל להתאושש מנפילה באופן יעיל [תלוי במס' השינויים שלא בוצעו]

- נצטרך לבצע כתיבות נוספות לדיסק לקובץ ה-log.

ישנן מערכות קבצים מתועדות הנשמרות כ-log בלבד, ולא כמו JFS המיועדת להתאוששות. במערכות אלו, ה-inode'ים מפוזרים בין הרשומות בקובץ ה-log, ולא במקום קבוע בדיסק. בלוקים המכילים data של קובץ – מפוזרים ג"כ ב-log.

כל כתיבה יוצרת רשומה ב-log: או שיצרנו בלוק חדש, או שעדכנו בלוק קיים.

כשנכתוב לדיסק – נבצע זאת ביחידות של סגמנטים מה-log וכך נרוויח ניצולת גבוהה של הדיסק.

נשים לב מערכת קבצים מבוססת לוג היא ציקלית – אסור לראש להשיג את הזנב. ישנו תהליך מיוחד המקצר את הזנב, ומאפשר לראש לגדול. הוא מבצע את האלגוריתם הבא:

- עובר על כל הרשומות שבזנב, ובודק אם ה-inode אינו תואם למפה או שמס' הבלוק לא מופיע ב-inode – נוכל למחוק אותו מהרשימה, כלומר: להעביר את המצביע לזנב לרשומה שלפניו.
- אחרת – נכתוב את המידע מחדש בראש קובץ ה-log
- מבצעים עדכון של המפה או של ה-inode הלוונטי
- נקדם את המצביע לזנב לרשומה שלפניו

בעת פתיחת קובץ – נצטרך למצוא את ה-inode של הקובץ במקום האחרון ב-log בו הוא נמצא. נחזיק מפת inode'ים, שתמפה את מס' ה-inode למיקומו האחרון בדיסק. [לשפר את יעילות מציאת מיקום ה-inode].

הערות:

1. במערכות LFS שומרים את **מפת** ה-inode'ים גם בדיסק וגם בזיכרון הראשי [לא את כל ה-inode'ים]
2. במערכות שנבנות עבור זיכרון מסוג flash – לא נשמור את המפה בדיסק, ע"מ להימנע משחיקה של אזורים בזיכרון. אלא המפה תיבנה בעת ביצוע אתחול למערכת הקבצים.

דוגמא : מפת inodes

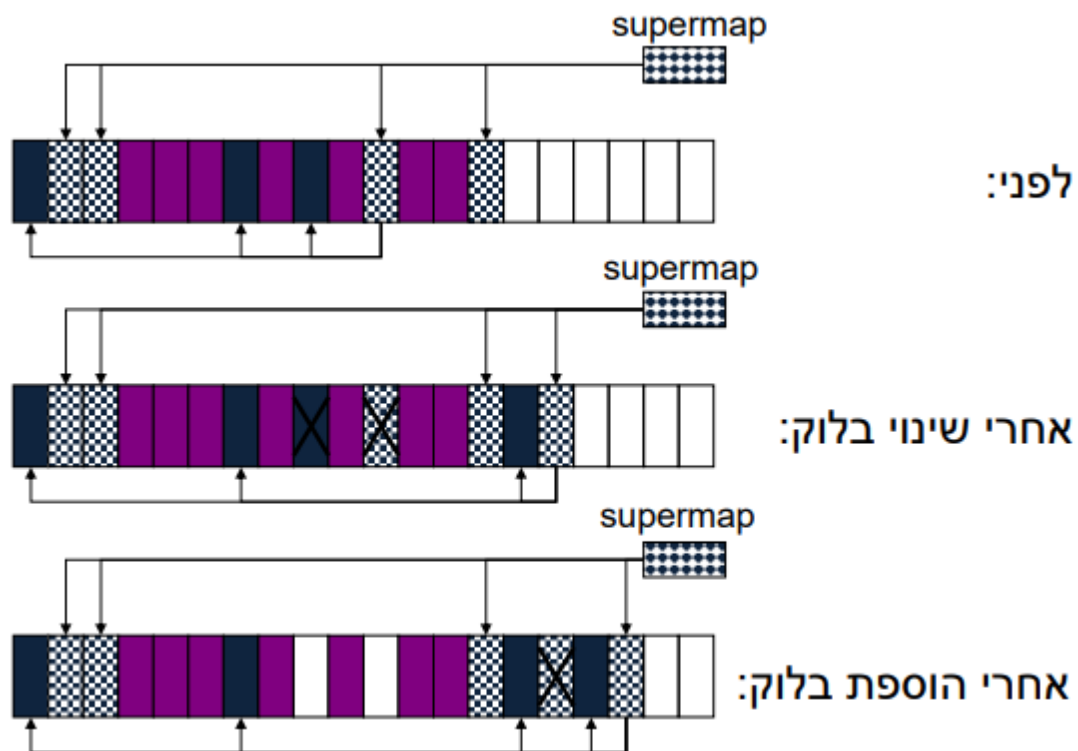
מערכת קבצים במבנה לוג		מפת inodes	
1	inode 1 = ..., 334, 1256	1	1
2	inode 2 = ..., 13456, 6790	2	20119
.	...	.	
.	...	.	
.	...	.	
20119	inode 2 = ..., 13456, 6790, 20120	.	
20120	NEW DATA	.	
		.	
		.	
		.	

יתרונות במערכות מבוססות log:

- א. כתיבה סדרתית לדיסק מתבצעת ביחידות גדולות
- ב. נבצע פחות תזוזות ראש בדיסק
- ג. נפחית את רמת ה-overwrite בזיכרון flash

חסרונות במערכות מבוססות log:

- א. בניית מפת ה-inode איטית
- ב. כתיבת מפת inode בדיסק יוצרת כתיבה לא סדרתית
- ג. בהעברת רשומה לראש ה-log נבצע מחיקה מהזנב וכתיבה לראש – מאיט את הביצועים, עבודה קצת מיותרת כי המידע הראשוני שהיה ברשומה לא צריך להימחק
- ד. קשה לדעת כמה מקום פנוי בהתקן



מערכות I/O

למחשב מחובר ציוד מגוון ביותר: דיסק, מקלדת, מסך, מדפסת, עכבר, וכו'.

לכן אופן הטיפול בציוד – מגוון, לפחות ברמה הנמוכה.

תת המערכת (subsystem I/O) של מעה"פ הוא חלק הגרעין האחראי על ביצוע ה-I/O בפועל, באופן שיתר מעה"פ יזכה לממשק אחיד ככל האפשר.

השאיפה: לכמוס (encapsulate) את הפרטים וההיבטים הייחודיים של רכיבי הציוד השונים.

האמצעי: שימוש במנהלי התקן (drivers device) אשר יספקו ממשק אחיד. יצרן חומרה, לכשייצר חומרה חדשה, אחראי גם לכתוב לה מנהל התקן, או לדאוג שהחומרה תוכל להיות מנוהלת ע"י מנהל התקן קיים.

מנהל ההתקן יתקשר עם בקר הציוד (אשר מופקד על חומרת הציוד). ב-UNIX, ראינו את המושג 'קובץ ציוד' – אשר מאפשר טיפול אחיד במגוון סוגי ציוד, כך שה-I/O לסוגי הציוד השונים מתבצע כמו שה-I/O לקובץ רגיל, או מקובץ רגיל (יש לפתוח את קובץ הציוד, ואחר לכתוב עליו, והנתונים נשלחים לציוד).

את מרבית מרכיבי הציוד ניתן לחלק לשלוש מחלקות:

- א. ציוד אחסנה (storage), כדוגמת דיסק או סרט.
 - ב. ציוד שידור (transmission), כדוגמת מודם או כרטיס רשת.
 - ג. ציוד לממשק אנוש, כדוגמת מסך, מקלדת, עכבר. הציוד מתחבר למערכת דרך נקודת חיבור = פורט, ומתקשר למערכת ע"י שליחת סיגנלים/איתותים כדוגמת פסיקות.
- המידע המועבר בין הציוד לזיכרון נשלח על פס (=bus) קבוצת מוליכים המעבירים מתח ונתונים מ/לרכיב ציוד אחד או יותר + פרוטוקול (=אוסף חוקים: למי מותר 'לומר' מה ומתי על המוליכים, כיצד יש 'לומר' (פיזית) וכיצד יש 'להבין' את הנאמר 'ולהגיב' לו).

נהוג להבחין בין מספר סוגי פסים:

- א. פסים **לוקליים** (=bus local) לציוד פנימי, כדוגמת כרטיס גרפי.
- ב. פסים **חיצוניים** (=bus external) לציוד חיצוני, כדוגמת דיסק קשיח או סורק.

שיטת אבחנה אחרת:

- א. פס **מקבילי** מעביר מילה מפורקת לסיביות במקביל על מספר קווים.
- ב. פס **סדרתי** מעביר את הנתונים סדרתית, סיבית אחרי סיבית (מחייב קווים נוספים להעברת מתח, ואותות בקרה דו-כיווניים).

עם השנים ההבדל בין הפסים לבין חיבורי הרשת השונים (Ethernet), ובין הפס הפנימי לחיצוני הולכים ומטשטשים.

ניהול קלט פלט I/O

מטרות:

1. יעילות ואבסטרקציה – ממשק אחיד והסתרת החומרה.
2. אבטחה – אין השתלטות של תהליך על התקן.
3. מקבילות – מיצוי כל התקנים.

מבנה המערכת:

1. User Applications – האפליקציות ברמת המשתמש

הגישה ל-I/O מתבצעת רק באמצעות system call. וכן חלק מפונקציות הספרייה ממשק ה-API. [יש פונקציות שמבצעות דברים נוספים מלבד ה-system call. לדוגמה: printf() מבצעת גם חציצה ופירמוט מידע חוץ מ-write()]. גישה סינכרונית: בקשת I/O מעבירה את התהליך למצב blocked עד ביצוע הבקשה. גישה אסינכרונית: התהליך ממשיך לרוץ, ובמקביל מידי פעם מנטרת את ערוצי ה-I/O.

2. OS – מערכת ההפעלה.

מטרה ראשית: הסתרת החומרה למשתמש, למרות השוני בין ההתקנים. מיושם ע"י העקרונות הבאים:

- 1) device independence – עצמאות ההתקנים. כל התקן מטפל בנתונים שלו.
- 2) uniform naming – שמות ההתקנים לא תלויים בהתקנים עצמם. [ב-UNIX נפנה להתקן דרך קובץ המייצג אותו, ונמצא בתקייה /dev יש לקובץ מספר ראשי המייצג את ה-Driver שאחראי על ההתקן, ומספר משני המייצג את ה-Device עצמו].
- 3) טיפול בשגיאות נעשה ברמת ההתקן, ואם לא יצליח – מעלה לטיפול ברמה גבוהה יותר.

תפקידי המימוש בתוכנת ה-OS:

- א. זיהוי פעולות משותפות להתקנים, וסיפוק ממשק אחיד למשתמש.
- ב. מיפוי שמות סמליים (מחרוזת או מספר שלא תלויים בהתקן עצמו) להתקנים, לאיתור המתאם המתאים.
- ג. הגנה על הגישה להתקן:
 - פקודות I/O המדליקות את ה-I/O bit – מותרות רק ב-kernel mode.
 - מנגנון הגנה על הזיכרון שומר על הבקרים המגיבים ל-I/O.
 - כדי לגשת להתקן חיצוני – מתבצע system call.
 - הקריאה הנ"ל מעבירה את ה-CPU ל-kernel mode כדי לתקשר עם הבקר.
- ד. אספקת גודל בלוק לוגי אחיד לשכבות העליונות.
- ה. caching – הטמנה: שמירת בלוקים שנקראו מהדיסק- במטמון.
- ו. buffering – שמירת מידע בחוצץ, ואחר כך כתיבה / עדכון בדיסק עצמו. כך נאפשר גישה להתקן לפי הקצב שלו, וכן ייווצר מצב בו בטוח נקבל בלוק שלם.
- ז. ניהול הבלוקים הפנויים לכתיבה ע"י אלגוריתמים.
- ח. ניהול תורות עבור התקנים המטפלים בכל תהליך בנפרד.
- ט. טיפול בשגיאות שלא נפתרות במתאם לפי המצב- שליחה לפתרון המשתמש או סיום פעולת המערכת.

Daemons:

- מפקחים על תקלות בהתקנים.
 - דואגים לתהליכים מורעבים.
 - spoolers קובעים תור להתקנים ייעודיים.
- יש לממש מנגנון spooling על ההתקנים. [ב-UNIX נשמור את המידע ב: /var/spool]

3. Device Driver – מנהל ההתקן.

נחשב כמתרגם בין מערכת ההפעלה לבקר. מעה"פ מספקת ממשק תוכנה אחיד לכל ההתקנים, ה-driver מתרגם לכל בקר את הפקודות עבורו, והיחיד שמכיר את מבנה הבקר ומבנה ההתקן.

מקבל בקשה מופשטת ממעה"פ, ובודק אם היא ברת ביצוע:
אם כן – קובע סדר פעולות שהבקר יבצע, ושולח אותן אליו דרך ה-CPU. הוא נחסם עד שפעולת הבקר מסתיימת.
הוא מכיל שגרות שמבצעות:

- ביצוע פסיקות
- אתחול ההתקן
- התחברות (open) / התנתקות (close)
- קריאת נתונים (read) / כתיבת נתונים (write)
- הזזת מצביע (seek)
- העברת פקודות למנהל או להתקן (ioctl)
- שגרות נוספות...

4. Interrupt Controller – מנהל הפסיקות.

החומרה אחראית על תפעול ציוד, פס או פורט.

הבקר עשוי להכיל מעבד וזיכרון (כפי שציינו בהקשר של בקר הדיסק).

ישנם רכיבי ציוד הכוללים בתוכם בקר מובנה, למשל: כונן הדיסקים. בקר הדיסק יהיה אחראי, כמובן, על העברת הנתונים בין הדיסק לזיכרון. הוא עשוי לאתר (ולהסתיר) נגזרות פגומות, לבצע prefetching, buffering, caching. הבקר מופעל ע"י המעבד.

ההפעלה עשויה לחול בשני אופנים שונים:

1. פקודות I/O גורמות לשליחת איתותים ונתונים על הפס לבקר שקולט ופועל לפיהם.
2. I/O ממופה זיכרון (I/O mapped memory):
אוגרי הבקר ממופים למרחב הכתובות של המעבד. עת המעבד פונה לכתובת x בזיכרון, הוא למעשה פונה לאוגרי הבקר, וכך ביכולתו לכתוב תכנית/הוראות לבקר (כך למשל מבוצע פלט למסך ב-PC). אופנות זאת של ק"פ צריכה להיתמך ע"י בקר הציוד, על-כן באותו מחשב ייתכן שיהיו רכיבי ציוד שיתמכו בה, ואחרים שלא.
היתרון ב-I/O ממופה זיכרון: מבחינת המעבד יש אחידות בביצוע פעולות I/O שונות, ובין לבין גישה לזיכרון. לכן הדבר מפשט את חומרת המעבד.
החסרון ב-I/O ממופה זיכרון: קורא התוכנית (תוכנית האסמבלר) עשוי שלא להיות ער לכך שמבוצע I/O.

לבקר האחראי על ק"פ יהיו בד"כ אוגרים כדוגמת הבאים:

1. **אוגר הסטטוס:** יעודכן ע"י הבקר. יכול דגלים (סיביות) המציינות את מצבו של הבקר:
האם עסוק (busy) בביצוע פקודה? האם חלה שגיאה (error)? האם האוגר in-data מכיל נתונים שניתן להעביר לזיכרון?
2. **אוגר הבקרה:** יעודכן ע"י המעבד, ויכיל את: הפקודה שעל הבקר לבצע, ודגלים אודות הפקודה.
לדוגמה: ready-command המורה לבקר שהוכנה לו (ע"י המעבד) פקודה חדשה לביצוע, write המורה שהפקודה היא כתיבה, seek להזזת ראשים.
באמצעותו גם יקבע המעבד לבקר לאיזה מהדיסקים עליהם הוא מופקד עליו לפנות. (לעתים האוגר יופרד לזוג אוגרים: אוגר הבקרה ואוגר פקודה).
3. **אוגר ה-in-data:** יכול נתונים שהבקר קרא מהדיסק, והמעבד מוזמן לקרוא לזיכרון.
4. **אוגר ה-out-data:** יכול נתונים שהמעבד הכין לבקר, על-מנת שהבקר יעתיק אותם לדיסק.
(לעתים השניים האחרונים ימוזגו לאוגר אחד).
5. **אוגר מסלול ואוגר נגזרת:** מציינים את כתובת המסלול / הנגזרת עליהם ניצב הבקר.

כיצד נדע האם הבקר זקוק ל-CPU? 2 גישות:

א. Polling – תשאול:

מידי פעם המחשב שואל את המעבד האם זקוק לו.
+ זמן טיפול קצר. (כשפרקי הזמן בין I/O רחוקים)
- overhead: המעבד עסוק בתשאול תמידי.

נציג עתה סכמה של העברת נתונים בשיטת ה-polling:

הרעיון הבסיסי הוא 'לחיצת ידיים' (handshaking): המעבד והבקר עובדים בשיתוף פעולה הדוק, ומתקדמים צעד צעד בביצוע התהליך.

נניח שבאמצעות פקודות קודמות המעבד הפנה את הבקר לדיסק המתאים, ובאותו דיסק למסלול ולנגזרת הרצויה. עתה המעבד מעוניין לפלוט בית לכתובת עליה ניצבים הראשים הקריאה/כתיבה.

- א. המעבד מבצע: כל עוד busy דולק: המתן.
- ב. המעבד מבצע: הדלק סיבית write.
- ג. המעבד: העבר בית לכתובת out-data.
- ד. המעבד: הדלק סיבית ready-command.
- ה. הבקר: מבחין שסיבית ready-command דולקת, ומדליק את סיבית busy.
- ו. הבקר: קורא את אוגר הבקרה ומגלה שהפקודה לביצוע היא כתיבה.
- ז. הבקר: מעביר את הנתונים הדרושים מהאוגר. לדיסק data-out.
- ח. הבקר: מנקה את הסיביות: ready, error, bus-command.

מגבלת השיטה:

בסעיף א' המעבד מבצע המתנה עסוקה או polling עד שהבקר יתפנה.

מנגד, אם המעבד יפנה לביצוע פעולות אחרות, יש חשש שמידע שנקרא ע"י הבקר יידרס ע"י מידע אחר שהוזן (שכן גודל החוצץ של הבקר מוגבל). לדוגמה: במידה והבקר אחראי על מקלדת ממנה מוזנים נתונים.

מסקנה: במצבים בהם הסכמה הזאת אינה מתאימה נשאף לפתרון חלופי, כזה המבוסס על פסיקות. על כן נחזור לדון מעט בפסיקות.

ב. פסיקות – Interrupts

Interrupts – בעת הצורך הבקר מדליק את קו הפסיקה.

+ תגובה מיידיית ל-I/O

- overhead - בשלבי הטיפול בפסיקה.

= פסיקה

1. איתות א-סינכרוני מהחומרה המורה למעה"פ 'אני זקוקה להתערבותך' (שכן גמרתי לבצע I/O שהורית לי לבצע, נתקלתי בתקלה, קרה משהו חדש: המשתמש הזיז את העכבר).

2. איתות סינכרוני מהתוכנית המורצת, המורה למעה"פ 'אני זקוקה להתערבותך, בפרט לשרות ממך'.

(כדי לבצע I/O, לשלוף נתונים שברשותך בלבד / כי קרה משהו: התוכנית פנתה לכתובת לא חוקית.)

בשני המקרים: מעה"פ לוקחת פיקוד על המעבד (מהתוכנית שרצה עד קבלת הפסיקה) ומריצה טפליט פסיקה (interrupt handler), אשר 'תפתור' את הבעיה שנוצרה.

כדי לייצר פסיקה סינכרונית, פסיקת תכנה, ע"י התוכנית המורצת, תבצע התוכנית פקודת אסמבלר מיוחדת (מין הסתם הפקודה תבוצע ע"י פונ' הספרייה שמזמנת התוכנית, אחרי הכנת הארגומנטים הדרושים לטפליט הפסיקה).

נדון עתה בפסיקת חומרה:

המנגנון הבסיסי, שהיה נהוג בעבר, לטיפול בפסיקות חומרה פעל באופן הבא:

- אחד החוטים המתחברים למעבד נקרא 'קו בקשת פסיקה' (interrupt-request line)
- בתום ביצוע כל פקודה ופקודה בודק המעבד קו זה. כשהוא מגלה שאחד הבקרים שלח איתות על הקו, הוא שומר מידע בסיסי על מצבו (כגון ה: PC, לא מתבצעת החלפת הקשר), ופונה לביצוע פונ' טיפול בפסיקות.
- הפונ' מאתרת את מקור הפסיקה, מטפלת בה, ולסיום מבצעת פקודת חזרה-מפסיקה אשר מחזירה את המעבד לביצוע התוכנית שנקטעה.

עם השנים המנגנון הבסיסי הנ"ל הלך והשתכלל:

- א. נוספה היכולת לדחות טיפול בפסיקות שאינן קריטיות, בעת ביצוע קוד מסוים.
- ב. מנגד, נוספה היכולת להתעניין בפסיקות גם כשמענה"פ מטפלת בפסיקה קודמת. כלומר: נוצרה היררכיה של פסיקות.
- ג. נוספה יכולת לזהות, ולו בקרוב, את מקור הפסיקה, כך שלא יהיה צורך לבדוק את כל בקרי הציוד.

לכן, כיום הפסיקות מתווכות ע"י בקר הפסיקות – Programmable Interrupt Controller, המכונה PIC (מקבל את הפסיקות, מעדכן אוגרים ייעודיים, ומעביר איתות למעבד על אחד משני קווים אפשריים, כפי שנתאר בהמשך).

המעבד, על-פי קו הפסיקה ותוכן האוגרים מחליט כיצד ומתי לטפל בפסיקה.

הזכרנו שני קווים באמצעותם יאותת הבקר למעבד על פסיקה:

1. קו לפסיקות שאינן ניתנות למיסוך כגון. NonMaskable Interrupts = MNI. כגון: פסיקות שעון, page fault.
2. קו לפסיקות הניתנות למיסוך Maskable IRQ = Interrupts. כגון: הבקר סיים להעביר נתונים, התוכנית המורצת מבקשת שרות ועל-כן יוצרת פסיקה.

פסיקות הציוד מטופלות ע"י מנהל ההתקן של הציוד המתאים. כל פסיקה מאופיינת ע"י מספר פסיקה = מספר תא בווקטור הפסיקות. בתא ישמר מצביע לפונ' הדרושה במנהל ההתקן.

עם עליית המחשב, ה-BIOS (עליו נדבר בהמשך) מאתר את רכיבי הציוד, טוען את מנהלי ההתקנים הדרושים, ומאתחל את וקטור הפסיקות.

פסיקה הנשלחת ע"י הציוד עשויה לגרום להרצת מספר טפלויות פסיקה. לדוגמה: נניח שהבקר שלח פסיקה שכן הוא גמר לקרוא נתונים (לחוצץ של מ.ה.):

- יש להעביר את הנתונים לשטח הזיכרון של תהליך המשתמש.
- יש להתחיל את פקודת ה-I/O הבאה, הממתינה בתור לדיסק.

המשימה השנייה דחופה יותר, כדי למקסם את ניצולת הדיסק.

המשימה הראשונה יכולה להמתין, כאשר המשמעות היא עיכוב תכנית המשתמש.

מענה"פ תריץ את טפלויות הפסיקה לביצוע המשימה השנייה. כאשר זו תסיים – היא תשלח פסיקה. כשמענה"פ תוכל – היא תריץ גם את הטפלויות לביצוע המשימה הראשונה. כשהאחרונה תסיים תשלח פסיקת תכנה שתביא להרצת המתזמן אשר יעביר את התהליך לתור המוכנים להרצה.

גישה ישירה לזיכרון – Direct Memory Access (DMA)

הציוד (כמו הדיסק) מעביר כמות רבה של נתונים.

תפעולו בשיטת ה-polling יחייב את המעבד בהקצאת כמות זמן ניכרת (נזכור שהמעבד מבצע המתנה עסוקה לבקר).

לכן נעדיף שיטה בה בקר הציוד לא 'יעיק' כ"כ על המעבד.

הפתרון: בקר DMA אשר יהיה מופקד על ביצוע ה-I/O מול בקר הציוד, ויחסוך טרחה זאת מהמעבד.

המעבד יכין לבקר ה-DMA על גבי חוצץ בזיכרון את תיאור ה-I/O שיש לבצע: כתובת בזיכרון, כתובת בדיסק, כמות בתים שיש להעביר, ק' או כ'. כתובת החוצץ תכתב על אוגר של בקר ה-DMA.

בקר ה-DMA יתקשר עם בקר הציוד, ויהיה מופקד על העברת הנתונים (דרך הפס).

-?- כיצד תבוצע העברת הנתונים:

!- באחת מהאפשרויות הבאות:

א. בקר ה-DMA מייצר פקודת קלט כנ"ל (אך במקום שהפקודה תיווצר ע"י המעבד היא מיוצרת ע"י הבקר).

ב. כשבקר הציוד קרא נתונים מהדיסק, נתונים שברצונו להעביר לזיכרון, הוא שולח איתות על קו ייעודי (DMA-request).

ג. בקר ה-DMA תופס את הפס (לזיכרון), שם על קווי הכתובת של הפס את הכתובת אליה יש להעביר את הנתונים, ושולח לבקר הציוד איתות (על קו DMA-acknowledge).

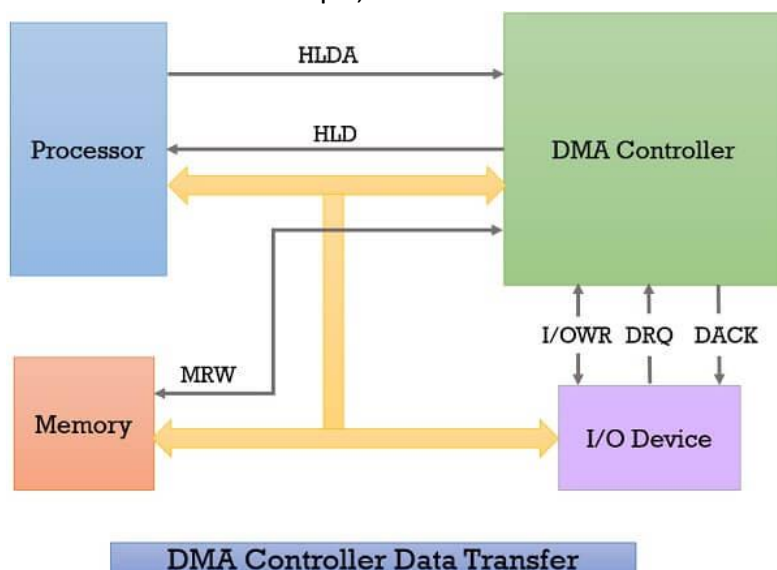
ד. בקר הציוד שולח את הנתון על הפס, ואחר מוריד את DMA-request

כשהתהליך מושלם בקר ה-DMA שולח פסיקה למעבד, כדי להודיע על סיום המשימה.

המצב בו בקר ה-DMA מעביר נתונים על הפס נקרא 'גנבת מחזורים' (stealing cycle) כאשר באותם מחזורים המעבד אינו יכול להשתמש בפס.

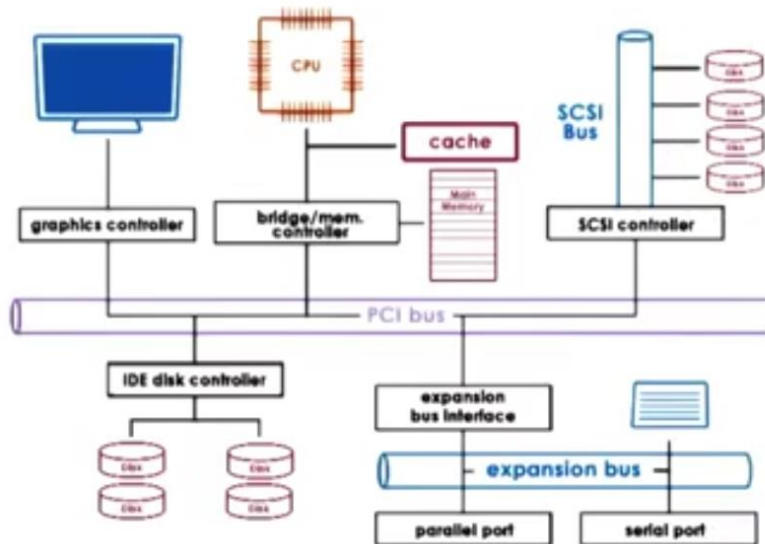
וריאנט של DMA – נקרא **DVMA** Direct Virtual Memory Access, ובו הנתונים מועברים לכתובת מדומה שיש למפות לכתובת פיזית.

היתרון: ניתן למפות שני רכיבי ציוד לכתובות מדומות, וכך להעביר נתונים ישירות ביניהם.



5. Device Controller – בקר / מתאם התקן.

זהו ממשק פרימיטיבי לניהול ה-I/O של ההתקן. יוצר יעילות של כלל המערכת, בכך שמפנה את ה-CPU לתהליכים אחרים. הבקר מחובר להתקן באמצעות ה-bus. (במחשבים גדולים-מודל IPC), ויכול להתחבר למס' התקנים.



מכיל:

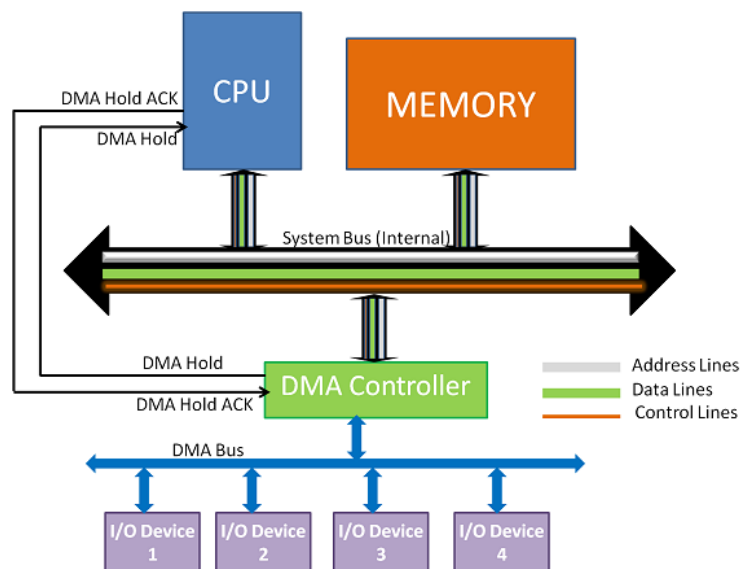
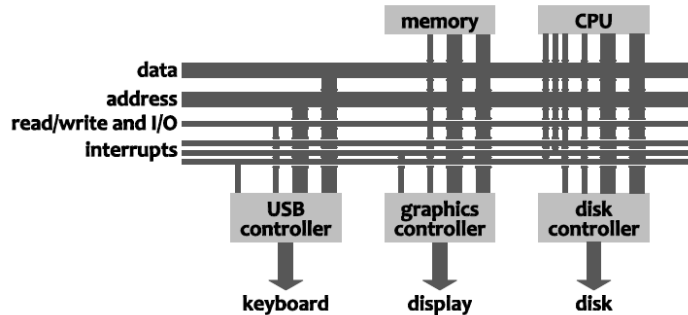
- א. רגיסטר Commands – הפקודות לביצוע.
- ב. רגיסטר Data – מידע על האופרטורים בפקודה או תוצאות מההתקן.
- ג. רגיסטר Status – שומר מידע על סטטוס ההתקן, עבור ה-CPU.
- ד. מעבד – שערים חשמליים לפענוח הפקודה, ו-ALU לביצוע.
- ה. Buffer – שומר את המידע שהגיע מההתקן. בסיום העברת המידע, יעורר פסיקה ל-CPU.
- ו. שבבים מיוחדים עבור החומרה.

אופני התממשקות הבקר, ה-CPU והזיכרון:

1. Memory Mapped IO – נפוצה מאד. מרחב כתובות ה-I/O משותף לגישה לזיכרון.
 2. Port Mapped IO – שימושי לאינטל x86.
 - מרחב כתובות נפרד- I/O ולזיכרון.
 - ל-CPU יש פקודות מיוחדות (in-out).
 - הגישה לכתובת ההתקן- ע"י port. (קל לתרגום כתובת)
- התהליך עצמו: התוכנית כותבת לכתובת שהבקר מאזין להם, וה-CPU שולח IO request. הבקר מקבל את הבקשה, שולח מידע ומתקשר עם התוכנית ע"י פסיקה.

ה-bus מורכב מ-4 ערוצים:

1. **data** – עליו נמצאים הנתונים.
2. **address** – עליו הכתובת שה-CPU רוצה לקרוא ממנה.
3. **read/write** – מה המצב הנוכחי? קריאה / כתיבה?
4. **Interrupts** – מייצר פסיקה בשביל ה-CPU.



6. Device – התקן.

סוגי התקני I/O:

1. **Block Device**: התקנים השומרים מידע בבלוקים, ועבור כל בלוק שמורה כתובת פרטית.
2. **Character Device**: התקנים שמקבלים או שולחים רצף תווים, בלי התייחסות למבנה הבלוק.
3. **מקרים מיוחדים**:
 - א. **NIC** - כרטיס רשת: לא מעביר מידע בצורת תווים או בלוקים, אלא כ-package (גוש) גמיש.
 - ב. **Clock** - השעון. (שעיקר מטרתו ליצור פסיקות בפרקי זמן).

BIOS-ו Bootstrapping

BIOS = Basic Input/Output System

זהו קוד השמור בזיכרון בלתי נדיף המורץ ב- PC IBM ודומיו.

עם הדלקת המחשב ע"י ה-CPU, הוא מונחה לפנות לכתובת קבועה מראש בה שוכן ה-BIOS.

תפקידו: להכין את המערכת כך שניתן יהיה לטעון ולהריץ את מעה"פ.

ה-BIOS מאתחל:

- א. את clock ומתזמן ומתאם את פעולת רכיבי המערכת.
- ב. המעבד וזיכרונות המטמון.
- ג. ה-chipset הכולל את בקרי הזיכרון וה-I/O.
- ד. הזיכרון
- ה. הציוד המותקן על ה-PCI (כך שרכיבי הציוד יוכלו לחלוק את הפס).
- ו. הבקר הגרפי שעל לוח האם.
- ז. בקרי הדיסקים הקשיחים (IDE, SATA)
- ח. בקרי ק"פ נוספים (למקלדת, עכבר, ציוד USB)
- ט. וקטור הפסיקות וטפלויות הפסיקות.
- י. מנהלי ההתקנים שיש לטעון.

לאחר מכן טוען ה-BIOS את ה-boot loader של מעה"פ, ומריץ אותו.

התהליך נקרא (=Power-on Self-Test) שכן במקורו רובו יועד לבדיקת המערכת, ומיעוטו לאתחולה.

לעתים ה-BIOS מאפשר בחירה של מערכת ההפעלה שתטען.

כדי לאתחל ציוד נוסף (RAID, SCSI, כרטיס גרפי וכו') כוללים מרכיבים אלה BIOS משלהם, אשר יזומן ע"י ה-BIOS 'הראשי'.

בעבר ה-BIOS שימש גם כמנהל התקנים אחרי עליית מעה"פ.

בשנים האחרונות נוספו ל-BIOS פונ' נוספות:

- א. ניהול המתח = management power : העברת המערכת למצב 'שינה' עת לא נעשה בה שימוש לפרק זמן כלשהו.
- ב. ניהול מערכת הקירור.
- ג. hot plugging = hot swapping אפשרות לניתוק וטעינת ציוד עת המחשב דולק.

מכיוון שה-BIOS שוכן בזיכרון לקריאה בלבד, אז: מצד אחד הוא חסין לוויורוסים, ומצד שני קשה לעדכנו...

הפתרון: ה-BIOS יכול את המינימום שבמינימום, ויטען את תכנת האתחול מכתובת קבועה בדיסק. תכנת האתחול תטען את יתר מ.ה. מכל מקום שהוא בדיסק.

boot-disk/system-disk = דיסק הכולל תכנת אתחול של מעה"פ.

בחלונות 2000 קוד האתחול מצוי בסקטור #0 הנקרא בקוד Master Boot Record (MBR). קוד זה נטען ע"י ה-BIOS. הקוד כולל את טבלת החלוקה של הדיסק, ואת שם חלוקת ה-boot ממנה יש לטעון את מעה"פ.

ה - MBR קוראת את הסקטור הראשון בחלוקת ה-boot, ולפי המידע המצוי בו את יתר מעה"פ.

סיווג מערכות

וירטואליזציה

עקרונות הווירטואליזציה:

1. דימוי תשתיות חומרה אמיתיות- ע"י הקצאת משאבי חומרה מתשתית רחבה יותר.
 2. הרכבת מכונה וירטואלית ממספר משאבים פיזיים
- VM- וירטואליזציה של החומרה. היא מספקת שקיפות, כל מעה"פ שרצה על מכונה וירטואלית בטוחה שכל משאבי המערכת עומדים לרשותה.

יתרונות הווירטואליזציה:

- + איחוד: אפשרות להריץ מספר מכונות וירטואליות (מעה"פ שונות) על אותה פלטפורמה ← ניצול חומרה.
- + נייחות: קל להעברה בין חומרות שונות ← יוצר זמינות ואמינות
- + פיתוח תוכנה באופן יעיל
- + אבטחה

מימוש הווירטואליזציה: Hypervisor / VMM

ציפיות מ-VMM

1. שיריץ ממש כמו על מעה"פ ישירה
2. בידוד בין המכונות
3. מהירות כמו שמעה"פ ישירה הייתה מבצעת

סוגי Hypervisor:

1. Bare – Metal - ממש במקביל
2. Hosted - מעה"פ אורחת במכונה וירטואלית.

שיתוף קבצים

231-242

הרשאות

245-246

ביצועים

251-267

מערכות קבצים מתועדות

מערכות קבצים מבוזרות

325